

MANUEL PROFESSIONNEL · ÉDITION 2026

M08 — Python pour l'analyse de données

Huit chapitres · 30 h · fil rouge vente.csv

Analyse de données appliquée & Business Intelligence
8 partie(s) assemblée(s) · document de production

Table des matières

MODULE M08.C01 — INSTALLER SANS STRESS : PYTHON, PIP, ENVIRONNEMENTS VIRTUELS, VS CODE ET JUPYTER, EXÉCUTER SON PREMIER SCRIPT	7
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — la conversation avec la machine	8
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 Les 4 couches — ce qu'on installe, et pourquoi	9
5.2 Installer Python 3.13 — la case qui sauve	10
5.3 Installer des paquets — pip, la boutique	10
5.4 L'environnement virtuel — la trousse scellée	11
5.5 VS Code — l'atelier de travail	12
5.6 Jupyter et le noyau — la table à dessin qui se souvient	12
5.7 Le premier script — premier.py	13
5.8 Les 5 erreurs d'installation les plus courantes	13
6. Exemple concret — le script qui répond à une question métier	14
7. Démonstration pas à pas — 5 étapes sur le fil rouge	14
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « le rapport d'installation » (15 min, /10)	16
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P1 — « L'audit d'environnement » (1 h)	17
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C02 — LES FONDATIONS SANS DONNÉES : VARIABLES, TYPES, OPÉRATEURS, ENTRÉES/SORTIES, COMMENTAIRES, ERREURS	20
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — l'étiquette, la famille, la porte	21
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 La variable — l'étiquette qui se re-colle	22
5.2 Les 4 types de base — et la conversion à la porte	22
5.3 Les opérateurs — l'arithmétique du comptoir	23
5.4 input() et print() — la porte et la vitrine	23
5.5 Les commentaires — le « pourquoi », pas le « quoi »	24
5.6 Lire une erreur — l'anatomie, en 3 protestations réelles	24
5.7 None et les valeurs « fausses » en condition	25

6. Exemple concret — le calculateur de remise	26
7. Démonstration pas à pas — 5 étapes sur le fil rouge	14
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « le ticket de caisse » (15 min, /10)	28
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P2 — « La caisse » (1 h)	29
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C03 — DÉCIDER ET RÉPÉTER : IF/ELIF/ELSE, BOUCLES FOR/WHILE, BREAK/CONTINUE, COMPRÉHENSIONS	32
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — le carrefour, le tapis, le « tant que »	33
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 if/elif/else — l'échelle du comptoir	33
5.2 La boucle for — le tapis roulant des 5 magasins	34
5.3 La boucle while — le « tant que » du crédit	35
5.4 break et continue — la porte de sortie et le « passe »	36
5.5 Les compréhensions — le tapis avec filtre, en une ligne	36
5.6 zip — deux listes, un seul parcours	37
6. Exemple concret — le contrôle qualité express	38
7. Démonstration pas à pas — 6 étapes sur le fil rouge	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « le registre à trois modes » (15 min, /10)	41
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P3 — « Le rapport de qualité complet » (1 h)	43
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C04 — RANGER : LISTES, TUPLES, CHÂÎNES, DICTIONNAIRES, ENSEMBLES, INDEX, TRIS	45
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — la case, l'enveloppe, l'étagère, le tamis	46

4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 La liste — la case numérotée	47
5.2 Le tuple — l’enveloppe scellée	47
5.3 La chaîne — le format français en méthodes	48
5.4 Le dictionnaire — l’étagère étiquetée	48
5.5 L’ensemble — le tamis	49
5.6 Index, tranches, tris	50
6. Exemple concret — les 5 magasins dans 4 contenants	51
7. Démonstration pas à pas — 6 étapes sur le fil rouge	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « les 5 questions des magasins » (15 min, /10)	54
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P4 — « L’annuaire des magasins » (1 h)	55
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C05 — ORGANISER : FONCTIONS, ARGUMENTS, PORTÉE, DOCS-TRING, MODULES, TRY/EXCEPT, FICHIERS	58
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — la machine à café	59
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 La fonction — le return est la sortie, pas le print	59
5.2 Les arguments — positionnels, nommés, par défaut	60
5.3 La portée — la cuisine et le comptoir	61
5.4 La docstring — la notice sur la machine	62
5.5 Modules et paquets — l’import en tête	62
5.6 Le try/except — le filet sous la fuite prévue	63
5.7 Lire une traceback — les 3 couches	63
6. Exemple concret — le nettoyeur de montants	65
7. Démonstration pas à pas — 6 étapes sur le fil rouge	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « le nettoyeur de montants » (15 min, /10)	68
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P5 — « Le registre de caisse » (1 h)	70
14. Boîte à outils du chapitre	18

15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C06 — NUMPY UTILE ET SUFFISANT : TABLEAUX, VECTEURISATION, STATISTIQUES, NAN, WHERE, BROADCASTING	73
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — le convoi, la barrière, le camion vide	74
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 Le tableau — le convoi homogène	75
5.2 La vecteurisation — la barrière levée	75
5.3 Les statistiques — min, max, percentiles, médiane	76
5.4 nan — le camion sans marchandise	76
5.5 np.where — le contrôle qui étiquette	77
5.6 Le broadcasting — la règle d'alignement	77
5.7 La mesure — la boucle de C03 face à la ligne	78
6. Exemple concret — les 4 calculs du fil rouge	79
7. Démonstration pas à pas — 6 étapes sur le fil rouge	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « les 4 statistiques du comptoir » (15 min, /10)	82
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P6 — « Le rapport de colonne » (1 h)	83
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C07 — PANDAS I : SERIES, DATAFRAME, READ_CSV/READ_EXCEL, LOC/ILOC, FILTRES, NOUVELLES COLONNES, DESCRIBE, VALEURS MANQUANTES	86
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — le classeur d'indexation	87
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 read_csv — l'import des 8 CSV réels	88
5.2 head, shape, dtypes — la première lecture	88
5.3 L'index — la réduction, pas une colonne	89
5.4 loc vs iloc — le tableau des 4 cas	90
5.5 Les filtres — le WHERE en masque	91
5.6 Les nouvelles colonnes — le SELECT ... AS	92
5.7 describe() — la carte d'identité statistique	92

5.8 Les valeurs manquantes — l’audit du dossier	92
5.9 read_excel — le même objet, un autre format	93
5.10 Les 4 écritures obsolètes — 1 ^{re} save des 12	94
6. Exemple concret — l’audit du dossier en 8 lignes	95
7. Démonstration pas à pas — 6 étapes sur le fil rouge	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « la vue des 120 premières ventes » (15 min, /10)	98
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P7 — « L’audit du dossier » (1 h)	100
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C08 — PANDAS II : GROUPBY/TRANSFORM/MERGE, PIVOT_TABLE, DATES, APPLY, EXPORT EXCEL — ET L’ANNEXE MIGRATION 2.3 → 3.X	103
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — les classeurs et le classement	104
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 groupby + agg — le GROUP BY en une ligne	105
5.2 transform — la moyenne du groupe, remise sur chaque ligne	106
5.3 merge — le JOIN de M07, et la table vide qui rend tout NaN	106
5.4 pivot — la vue mensuelle de M07, reconstruite sans SQL	107
5.5 Les dates — l’index qui porte un sens, et l’alias 'M' mort	108
5.6 apply — la fonction de C05 sur une colonne, et son prix mesuré	108
5.7 L’export Excel — le classeur multi-feuilles, le livrable E3	109
5.8 La leçon des centimes — 1 à 2 FCFA sur 7,9 milliards	109
5.9 L’annexe migration 2.3 → 3.x — les 12 écritures, avant/après	110
6. Exemple concret — les 10 questions de M07.P, exécutées en pandas	111
7. Démonstration pas à pas — 6 étapes du bout en bout	112
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « l’année 2025 du magasin 4 » (15 min, /10)	114
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P — « Le script qui fait le travail de trois matinées » (2 h)	116
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19

Module M08.C01 — Installer sans stress : Python, `pip`, environnements virtuels, VS Code et Jupyter, exécuter son premier script

Outils : Python 3.13.14, `pip` 26.1.2, `venv` (natif Python 3.13), VS Code (cité), Jupyter (noyau exécuté). **Durée indicative :** 4 h. **Niveau :** N2 → N3. **Prérequis :** M05.C04 (vous avez vu du pandas en M05) ou grande aisance Excel ; M07 (aisance SQL) — le parallèle SQL/pandas s'active à C07, ici il est seulement annoncé.

L'idée du chapitre. En M05 vous avez vu pandas produire un chiffre ; en M07 vous l'avez fait produire par le SGBD. En M08, c'est **vous** qui tenez la machine. Mais avant la première ligne de code, il faut que la machine **vous écoute** : un interpréteur installé proprement, des paquets au bon endroit, et un projet isolé du reste de la machine. C01 pose les **4 couches** de l'environnement (la machine, l'interpréteur, les paquets, le projet), installe chaque couche, et termine par **votre premier script** exécuté deux fois (terminal, puis notebook) — avec les **5 erreurs d'installation** qui font abandonner plus d'apprenants que n'importe quelle notion de Python. Le chapitre ne vous apprend pas encore à programmer : il vous apprend à **ne pas être bloqué par l'environnement** quand vous voulez programmer.

Matériel de l'atelier — Python 3.13.14 · `pip` 26.1.2 · pandas 2.2.3 (environnement d'écriture) · numpy 2.3.5 · Jupyter (noyau exécuté). Toutes les commandes de ce chapitre ont été exécutées dans l'atelier le 19/09/2026 ; les sorties publiées sont **celles de l'atelier** (Linux). Sur votre machine Windows, la commande est presque toujours identique — chaque écart (`python` vs `python3`, `.venv\Scripts\activate` vs `source .venv/bin/activate`) est signalé au passage.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Distinguer les **4 couches** de l'environnement Python (machine, interpréteur, paquets, projet) et dire quelle couche cause quel symptôme.
2. Installer Python 3.13 **sans casser le système** (la case « Add to PATH », la vérification en ligne de commande).
3. Installer des paquets avec `pip` — et **figer** la liste avec `requirements.txt`.
4. Créer et activer un **environnement virtuel** — la boîte à outils scellée du projet.
5. Exécuter votre premier script dans un **terminal**, et les mêmes cellules dans **Jupyter** — en comprenant ce que le noyau se souvient, et ce qu'il oublie.
6. Diagnostiquer les **5 erreurs d'installation** les plus courantes par leur sortie exacte.

2. Pourquoi cette notion est importante

C'est le chapitre le plus court du module en heures (4 h) et le plus long en « risques » : l'installation est le **premier point d'abandon** de tout parcours Python. Un apprenant qui perd trois jours sur un « `pip` installe dans le mauvais Python » perd plus que n'importe quelle notion de boucle ou de dictionnaire ne lui en coûtera — parce que le doute empoisonne tout ce qui vient après (« c'est peut-être moi qui n'y arrive pas », alors que c'est le PATH qui est vide).

Deuxième raison, professionnelle : l’environnement virtuel n’est pas une précaution de débutant, c’est **l’habitude du métier**. Le script que vous livrerez en M08.P (« le travail de trois matières ») devra tourner sur la machine du collègue : sans `requirements.txt` et un `venv`, il tourne chez vous et nulle part ailleurs. C01 installe cette habitude **avant** la première ligne de code utile, pas après le premier incident.

Enfin, l’enjeu de version : vous verrez plus bas qu’une installation fraîche de `pandas` installe la version **3.0.6**, tandis que l’atelier (et ce manuel) s’appuie sur la **2.2.3** — les deux coexistent, et le manuel les documente toutes les deux (la décision est au §1.7 du plan M08). Comprendre pourquoi les versions cohabitent maintenant, c’est ne pas paniquer plus tard quand une sortie ne correspond pas.

3. Explication simple — la conversation avec la machine

Prenons l’analogie que vous connaissez déjà :

- **Excel** est un programme qui sait faire des additions et des tris : vous lui parlez dans **sa** langue (les formules), il fait le reste.
- **Python** est un **traducteur généraliste** : il ne « sait » rien par lui-même, il **lit vos instructions ligne par ligne** et les exécute dans l’ordre. Votre fichier `.py`, c’est une liste numérotée d’ordres ; l’interpréteur, c’est la personne qui les lit à voix haute, un par un.
- **pip** est la **boutique** : elle installe des paquets — des boîtes d’instructions déjà écrites par d’autres (`pandas` sait faire des tableaux, `numpy` sait faire des calculs en bloc). Sans la boutique, Python ne sait faire que l’arithmétique et quelques fonctions de base.
- **L’environnement virtuel** (`venv`) est la **trousse à outils scellée** du projet : la boutique livre les boîtes dans la trousse, pas dans le placard commun de la maison. Le projet d’à côté n’y touche pas ; vous pouvez jeter la trousse sans vider le placard.
- **Jupyter** est une **table à dessin** découpée en cases (les cellules) : chaque case est exécutée à part, mais le **noyau** — le traducteur qui tourne derrière la table — **se souvient** de ce que les cases précédentes ont défini. C’est formidable (pas besoin de tout relancer) et piégeux (l’ordre d’exécution compte, pas l’ordre d’écriture — nous le prouverons à §5.6).

La conversation, à partir d’aujourd’hui, est celle-ci : vous écrivez des ordres (`.py` ou cellules), l’interpréteur les lit, et soit il répond (sortie, fichier, graphique), soit il **proteste** (erreur — et la protestation est lisible, nous l’entraînerons à C05). Tout le reste de ce chapitre sert à ce que cette conversation **puisse commencer**.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
interpréteur interpreter	— le programme qui lit le Python et l’exécute ligne à ligne.	croire qu’il « traduit en avance » : non, il exécute au fil, ce qui explique les erreurs de ligne N alors que le bug est à la ligne N-3.
paquet — package	un bloc de code réutilisable installé séparément (<code>pandas</code> , <code>numpy</code>).	confondre paquet (la boîte, ce que <code>pip</code> installe) et module (ce qu’on importe) : un paquet contient un ou plusieurs modules.
environnement virtuel — virtual environment	une copie isolée de Python + ses paquets, réservée à un projet.	installer dans le Python système « pour cette fois » : le projet de demain casse celui d’aujourd’hui.
noyau — kernel	le moteur d’exécution derrière Jupyter, qui garde en mémoire les variables entre les cellules.	croire qu’une cellule « voit » ce que la cellule écrite au-dessus a fait : seul l’ordre d’exécution compte.

Français — English	Définition en une phrase	Piège à éviter
PATH	la liste d'endroits où Windows/Windows cherche les exécutables quand vous tapez <code>python</code> .	croire que l'installation a « marché » parce que le raccourci existe : seul <code>python --version</code> dans le terminal fait foi.
exigence requirement	— la ligne <code>paquet==version</code> d'un <code>requirements.txt</code> qui fige une installation reproductible.	figer « trop » : <code>requirements.txt</code> liste ce que le projet a besoin , pas tout ce que le <code>venv</code> contient par effet d'entraînement.

5. Cours approfondi

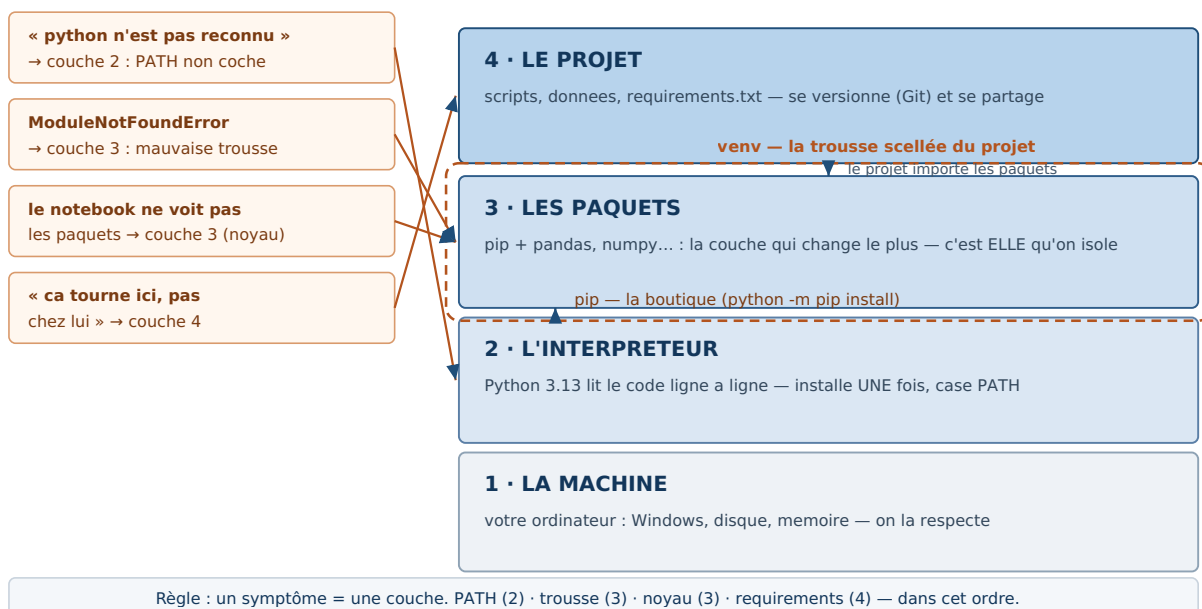
5.1 Les 4 couches — ce qu'on installe, et pourquoi

L'environnement Python se pense en **4 couches empilées** (figure du chapitre) :

Les 4 couches de l'environnement Python

Chaque symptôme d'installation se rattache à UNE couche — la nommer, c'est diagnostiquer.

Le diagnostic par couche



1. **La machine** — votre ordinateur (Windows ici), sa mémoire, son disque. On ne l'« installe » pas, on la **respecte** : on ne met pas de `venv` sur le disque C: plein, on ne met pas 40 Go de paquets inutiles sur la machine de la formation.
2. **L'interpréteur** — le runtime Python 3.13. Une seule installation suffit pour tout l'ordinateur ; c'est la couche qu'on installe **en dernier recours** (elle est fragile : deux versions mélangées = paquets perdus).
3. **Les paquets** — `pip` et ce qu'il installe (`pandas`, `numpy`, ...). C'est la couche qui change le plus vite ; c'est **elle** qu'on isole.
4. **Le projet** — vos scripts, vos données, votre `requirements.txt`. C'est la couche qui se versionne (Git) et se partage.

Définition. interpréteur — interpreter — le programme qui lit le code Python et l'exécute ligne à ligne, sans phase de « compilation » préalable visible : le fichier `.py` est lu, exécuté, relu si vous le modifiez. C'est la couche 2, installée une fois pour toutes.

Chaque symptôme d'installation se rattache à **une** couche : « `python` n'est pas reconnu » = couche 2 (PATH) ; « `ModuleNotFoundError` » = couche 3 (paquet absent **ou** mauvais venv) ; « le notebook ne voit pas mes paquets » = couche 3 mal pointée (mauvais noyau) ; « ça tourne ici, pas chez lui » = couche 4 (requirements non figés). Apprendre à nommer la couche, c'est diviser le temps de diagnostic par quatre.

5.2 Installer Python 3.13 — la case qui sauve

Windows-first, comme le lecteur du manuel :

1. Télécharger l'installateur officiel depuis **python.org** (3.13.x — la version de référence du module ; la date de vérification est le 19/09/2026).
2. Lancer l'installateur. **Première case, en bas de l'écran** : « **Add python.exe to PATH** » — **la cocher**. C'est la case n° 1 des erreurs d'installation : sans elle, le terminal ne trouvera jamais `python`.
3. « Install Now ». L'installateur termine en quelques minutes.

La **vérification qui fait foi** (point de contrôle, identique sur Windows et Linux) :

```
C:\Users\vous> python --version
Python 3.13.14
```

Dans l'atelier, l'interpréteur s'adresse en `python3` (convention Linux) ; la sortie publiée est celle du 19/09/2026 :

```
$ python3 --version
Python 3.13.14
```

Attention. Sur un Windows qui a déjà eu Python installé par une autre application, `python` peut pointer vers l'**App Installer** de la Microsoft Store (qui ouvre le Store au lieu d'exécuter). Symptôme : `python --version` ouvre une fenêtre du Store. Remède : installer la version officielle, et vérifier que `python -m site` répond par un chemin `C:\Users\...\AppData\Local\Programs\Python\...` et non par une page du Store.

5.3 Installer des paquets — `pip`, la boutique

`pip` est le gestionnaire de paquets de Python. Sa version et son lieu d'installation, vérifiés dans l'atelier le 19/09/2026 :

```
$ pip3 --version
pip 26.1.2 from /usr/local/lib/python3.13/site-packages/pip (python 3.13)
```

Trois choses à lire dans cette sortie : la version de `pip` (26.1.2), **le dossier** où `pip` installe (`.../site-packages` — c'est là que logent les paquets), et **le Python** auquel `pip` est attaché (`python 3.13` — la couche 3 sert la couche 2, et une seule).

Installer, c'est une ligne :

```
$ python3 -m pip install pandas numpy
```

(`python3 -m pip` plutôt que `pip3` : ça garantit que c'est **ce** Python qui installe — le réflexe professionnel, surtout quand plusieurs Pythons coexistent.)

Dans l'atelier, la sortie réelle du 19/09/2026 (`pandas` déjà présent, version figée) :

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/site-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/site-packages (2.3.5)
```

Conseil professionnel. Figez toujours la version dans `requirements.txt` avec `==` (`pandas==2.2.3`), jamais sans : un « `pip install -r` » qui installe la dernière version disponible le 14 mars 2027 peut casser un script qui tournait depuis le 19 septembre 2026. La version figée est une **date de péremption connue** ; la version flottante, une inconnue.

5.4 L'environnement virtuel — la trousse scellée

Créer la trousse, c'est une ligne ; dans l'atelier (exécuté le 19/09/2026) :

```
$ python3 -m venv /tmp/venv_m08
```

Sur Windows, même chose : `python -m venv .venv` (le venv vit **dans** le dossier du projet, par convention). L'activer, c'est **changer de Python temporairement** :

```
# Linux (atelier)      $ source .venv/bin/activate
# Windows              .venv\Scripts\activate
```

Une fois activé, le préfixe du terminal change — c'est le **seul** signe visible que vous êtes « dans » (sur Windows, `(.venv)` apparaît au début de chaque ligne).

La démonstration de l'isolation, exécutée dans l'atelier — le venv vierge **ne connaît que pip** :

```
$ /tmp/venv_m08/bin/pip list
Package Version
-----
pip      26.1.2
```

Et pandas n'y est **pas** — l'erreur est publique, avec sa sortie exacte :

```
$ /tmp/venv_m08/bin/python -c "import pandas"
File "<string>", line 1, in <module>
    import pandas
ModuleNotFoundError: No module named 'pandas'
```

Définition. environnement virtuel — virtual environment — une copie isolée de l'interpréteur + ses paquets, limitée à un projet : les installations dans le venv n'affectent pas le Python système ni les autres venvs. C'est la couche 3, isolée de la couche 2.

Installer **dans la trousse**, puis vérifier — sortie réelle de l'atelier le 19/09/2026 :

```
$ /tmp/venv_m08/bin/pip install pandas
$ /tmp/venv_m08/bin/python -c "import pandas; print('pandas', pandas.__version__, 'importe sans erreur')"
pandas 3.0.6 importe sans erreur
```

Dans les faits. Une installation **fraîche** de pandas installe la version **3.0.6** (la dernière publiée au 19/09/2026) ; l'environnement d'écriture de ce manuel est **figé** sur la **2.2.3** — la version sur laquelle les M01-M07 ont été mesurés. Les deux coexistent sans se gêner (c'est tout l'intérêt du venv). Le module enseigne les patterns **valides sur les deux**, et signale les 12 écritures qui ne le sont plus (C07 §9 et C08 §8) ; un apprenant sur pandas 3.x ne sera donc jamais « en faux » par rapport au manuel — c'est la décision de conception du plan M08 (§1.3).

Et figer la trousse, c'est l'**exigence** — `pip freeze` liste ce qui est installé ; on n'en garde que ce que le projet a **besoin** :

```
$ /tmp/venv_m08/bin/pip freeze
numpy==2.5.3
pandas==3.0.6
python-dateutil==2.9.0.post0
six==1.17.0
```

`python-dateutil` et `six` sont des **effets d'entraînement** (`pandas` en a besoin, vous ne les avez pas demandés) ; un `requirements.txt` propre les liste quand même — sinon `pip install -r` ne reconstruirait pas l'environnement.

Définition. exigence — requirement — la ligne `paquet==version` d'un fichier `requirements.txt` : la liste figée des paquets (et versions) qu'un projet a besoin, qui permet à `pip install -r requirements.txt` de reconstruire **exactement** le même environnement sur une autre machine.

5.5 VS Code — l'atelier de travail

VS Code est l'éditeur du manuel (cité, non exécuté dans l'atelier — l'atelier travaille en ligne de commande, ce qui est le même moteur). Ce qu'il faut savoir en 4 points :

1. **Ouvrir un dossier** (pas un fichier) : c'est le dossier qui est le projet — le terminal intégré s'ouvre **dedans**, et c'est là que le `venv` doit vivre.
2. **Le terminal intégré** (`Ctrl+`) : il hérite du dossier du projet ; activer le `venv` **une fois** dedans, et tous les `python` du terminal pointent vers la trousse.
3. **Exécuter un script** : bouton « Run » en haut à droite (ou `F5`) — VS Code lance `python nom_fichier.py` et affiche la sortie en dessous.
4. **Le pointeur de Python** : en bas à droite, VS Code affiche le Python utilisé. S'il indique le Python système alors que vous êtes dans un `venv`, cliquez dessus et choisissez le `venv` — c'est la couche 3 mal pointée, variante éditeur.

Attention. Le terminal intégré et l'éditeur peuvent **ne pas se souvenir l'un de l'autre** : activer le `venv` dans le terminal n'active pas automatiquement l'interpréteur du bouton « Run ». Vérifiez le pointeur en bas à droite **après** chaque activation.

5.6 Jupyter et le noyau — la table à dessin qui se souvient

Jupyter (JupyterLab ou le notebook classique) exécute du Python **par cellules**. Le moteur derrière est le **noyau** : un interpréteur qui tourne en permanence et **garde la mémoire** de tout ce que les cellules exécutées ont défini.

Définition. noyau — kernel — le processus d'exécution derrière Jupyter : il exécute chaque cellule demandée et **conserve l'état** (variables, imports) entre les exécutions. Réinitialiser le noyau, c'est oublier tout l'état — les cellules gardent leur texte, plus leur effet.

La preuve par l'exécution — un notebook de 4 cellules, exécuté dans l'atelier le 19/09/2026 (`jupyter nbconvert --execute`, le même moteur que JupyterLab utilise pour les cellules) :

```
# cellule 1
print("Bonjour depuis Jupyter.")
```

```
sortie : Bonjour depuis Jupyter.
```

```
# cellule 2
prix = 1500
remise = 0.10
print("Prix net :", prix * (1 - remise))
```

```
sortie : Prix net : 1350.0
```

```
# cellule 3 – écrivez AVANT la cellule 4, exécutée dans l'ordre écrit
print("x =", x)
```

```
erreur : NameError: name 'x' is not defined
```

```
# cellule 4
x = 42 # je la crée MAINTENANT, dans l'ordre d'exécution
```

La cellule 3 a échoué **parce que la cellule 4 n'avait pas encore été exécutée** — pas parce qu'elle était « en dessous » dans l'ordre d'écriture. C'est la leçon du noyau : **l'ordre d'exécution fait la mémoire, pas l'ordre du papier**. Re-lancer la cellule 3 après la cellule 4, et elle s'exécute — sans rien avoir changé d'autre.

Dans les faits. L'atelier exécute les notebooks par `jupyter nbconvert --execute` (même noyau, sortie capturée pour les chapitres) ; sur votre machine, vous lanceriez `jupyter lab` (ou `jupyter notebook`) dans le **dossier du projet, venv activé** — le pointeur de noyau de JupyterLab doit indiquer le Python du venv, sinon c'est la même erreur de couche 3 que \$5.5, variante notebook.

5.7 Le premier script — `premier.py`

Le plus petit script utile du monde : il salue, il demande, il répond.

```
# premier.py – mon premier script
print("Bonjour, je suis un script Python.")
nom = input("Comment vous appelez-vous ? ")
print(f"Enchanté, {nom}.")
```

Exécuté dans le terminal de l'atelier le 19/09/2026 (la réponse `Aïcha` fournie par `echo` — le même effet que de la taper) :

```
$ echo "Aïcha" | python3 /tmp/premier.py
Bonjour, je suis un script Python.
Comment vous appelez-vous ? Enchanté, Aïcha.
```

Trois choses dans ces 3 lignes : un `print` (sortie), un `input` (entrée — et il renvoie **toujours** une chaîne, même `42`), et une **f-string** (le `f"... {nom} ..."` qui insère la valeur — le formatage que vous ferez toute la formation, montants compris, règle M01). Le même script, collé dans une cellule Jupyter, s'exécute identiquement — sauf que le noyau **se souvient** de `nom` pour la cellule suivante.

5.8 Les 5 erreurs d'installation les plus courantes

Le tableau de diagnostic du chapitre — chaque ligne est un **cas réel** rencontré par des débutants ; les deux premières sont citées (elles ne se produisent que sur Windows mal installé), les trois dernières ont leur **sortie exacte publiée** (exécutées dans l'atelier).

#	Symptôme	Cause (couche)	Remède
1	python : le terme ... n'est pas reconnu (Windows)	PATH non coché à l'installation (couche 2)	réinstaller en cochant « Add python.exe to PATH », ou ajouter le chemin du Python manuellement ; vérifier par python --version.
2	python --version ouvre le Store (Windows)	l'alias Microsoft Store précède le vrai Python (couche 2)	python -m site doit montrer un chemin AppData\Local\Programs\Python ; sinon désactiver l'alias App Installer (Paramètres → Applis → Alias).
3	ModuleNotFoundError: No module named 'pandas' alors que « pandas est installé »	paquet dans le Python système, venv actif sans pandas (couche 3) — exécutée : sortie publiée au \$5.4	pip list dans le venv actif ; si pandas n'y est pas, pip install pandas. Le paquet est « installé » — mais dans l'autre trousse.
4	Le notebook dit ModuleNotFoundError, le terminal, lui, importe sans problème	noyau Jupyter attaché au mauvais Python (couche 3)	en bas à droite de JupyterLab : choisir le noyau du venv ; sinon python -m ipykernel install --user --name monvenv.
5	pip install « réussit », mais le projet ne voit pas le paquet	pip et python sont deux Pythons différents (couche 2/3)	toujours python -m pip install ... : le -m pip force l'installation dans le Python qu'on exécute. Vérifier par python -m site avant et après.

6. Exemple concret — le script qui répond à une question métier

La question, en langage de comptoir : « l'article coûte 1 500 FCFA, la remise est de 10 %, quel est le prix net ? » En Excel (M03), c'est une cellule `=1500*(1-10%)`. En SQL (M07), c'est une colonne calculée dans le `SELECT`. En Python, c'est **un script** — exécuté dans l'atelier le 19/09/2026 :

```
# prix_net.py
prix = 1500
remise = 0.10
print(f"Prix net : {prix * (1 - remise):,.2f} FCFA")
```

```
$ python3 /tmp/prix_net.py
Prix net : 1 350.00 FCFA
```

La valeur est celle du socle du chapitre ($1\,500 \times 90\% = 1\,350$, clé `m08p_demo_prix_net`) — elle servira de **référence de relecture** : si votre machine renvoie autre chose, l'environnement ne tourne pas comme prévu, et c'est le chapitre entier qu'il faut rejouer. Le parallèle SQL, déjà : en M07, le même calcul était `SELECT prix_vente_ht * (1 - 0.10) AS prix_net FROM produit` ; — la formule est la même, le lieu change (une ligne de table vs une ligne de script).

7. Démonstration pas à pas — 5 étapes sur le fil rouge

De zéro à « mon premier script tourne », les 5 étapes, avec la sortie réelle de chaque commande (exécutées dans l'atelier le 19/09/2026).

Étape 1 — Vérifier l'interpréteur (couche 2) :

```
$ python3 --version
Python 3.13.14
```

Interprétation : la machine a un Python 3.13 — la version de référence du module est installée.

Étape 2 — Vérifier la boutique (couche 3) :

```
$ pip3 --version
pip 26.1.2 from /usr/local/lib/python3.13/site-packages/pip (python 3.13)
```

Interprétation : `pip 26.1.2` sert ce Python 3.13, et installe dans `site-packages` — les deux couches se correspondent.

Étape 3 — Sceller la trousse du projet (couche 3 isolée) :

```
$ python3 -m venv .venv
$ source .venv/bin/activate      # Windows : .venv\Scripts\activate
(.venv) $ python --version
Python 3.13.14
```

Interprétation : le préfixe `(.venv)` confirme l'activation ; le `python` du terminal pointe désormais vers la trousse.

Étape 4 — Livrer les paquets dans la trousse :

```
(.venv) $ python -m pip install pandas numpy
(.venv) $ python -c "import pandas, numpy; print(pandas.__version__, numpy.__version__)"
```

Interprétation : `import` sans erreur = les paquets sont lisibles par ce Python. Sur une machine neuve en 2026, cette installation livrerait `pandas 3.0.6` (dernière publiée) — c'est voulu : le chapitre C08 enseigne les deux versions.

Étape 5 — Exécuter le premier script (couche 4) :

```
(.venv) $ echo "Aïcha" | python premier.py
Bonjour, je suis un script Python.
Comment vous appelez-vous ? Enchanté, Aïcha.
```

Interprétation : la conversation est ouverte — sortie lisible, entrée acceptée, variable en mémoire.

8. Erreurs fréquentes

1. « **J'ai installé Python, `python` ne répond pas.** » → couche 2 : la case `PATH`. Vérification : `python --version` (Windows) / `python3 --version` (Linux). Si le terminal n'a pas de réponse, rien d'autre ne marchera — ne passez pas aux paquets.
2. « **`pip install pandas` dit OK, mais `import pandas` échoue.** » → deux Pythons différents : le `pip` d'un côté, le `python` de l'autre. Toujours `python -m pip install ...`.
3. « **Ça tourne ici, pas chez mon collègue.** » → couche 4 : pas de `requirements.txt` (ou versions non figées). `pip freeze > requirements.txt`, partage, `pip install -r requirements.txt`.
4. « **Le notebook a oublié mes variables.** » → le noyau a été réinitialisé (bouton « Restart ») — ou la cellule a été exécutée **avant** la définition. L'ordre d'exécution fait la mémoire (§5.6).
5. « **J'ai installé 3 versions de pandas dans 3 venvs, je ne sais plus où j'en suis.** » → c'est **normal** : c'est le but. Chacun des venvs a sa trousse ; le préfixe du terminal et le pointeur de Jupyter disent lequel est actif. Le problème serait l'inverse (tout dans le Python système).
6. **SyntaxError dès la première ligne, sur du code « qui vient du manuel ».** → le copier-coller a mangé des caractères (guillemets typographiques « » au lieu de "). Recopier la ligne à la main : si elle passe, c'était le collage, pas le code.

9. Bonnes pratiques professionnelles

1. **Jamais d'installation dans le Python système** pour un projet : venv d'abord, install ensuite. L'exception acceptée : les paquets d'outillage (pip lui-même, virtualenv).
2. **requirements.txt dès le premier paquet** — pas « quand le projet sera stabilisé » : le fichier qui n'a pas été tenu à jour pendant un mois est un fichier faux.
3. **Le venv ne se versionne pas** : il se **reconstruit** (`python -m venv .venv + pip install -r requirements.txt`). Le `.venv` va dans le `.gitignore` — le partage d'un venv binaire entre machines est une source de pannes silencieuses (chemins absolus dedans).
4. **La version figée est une date, pas une opinion** : `pandas==2.2.3` dit « tel que vérifié le 19/09/2026 ». Monter de version est une **décision** (on relit l'annexe de migration, C08 §8), pas un `pip install -U` par réflexe.
5. **Le nom du script est le nom de sa sortie** : `prix_net.py` produit des prix nets ; un script qui fait trois choses a trois noms en attente (C05 : fonctions).

Conseil professionnel. Dans une équipe, la première question qu'on pose avant d'exécuter un script reçu : « quel `requirements.txt`, quelle version de Python ? » La réponse doit prendre moins de dix secondes (le fichier est au dossier). Si elle en prend plus, le script n'est pas prêt à tourner — et c'est un défaut de livraison, pas de votre exécution.

10. Exercice guidé — « le rapport d'installation » (15 min, /10)

Énoncé. Écrire un script `controle.py` (6 lignes maximum de code) qui affiche un rapport d'installation du projet, **exactement** dans cet ordre :

1. la version de Python ;
2. la version de pandas ;
3. la version de numpy ;
4. le chemin du Python actif ;
5. le message `Installation OK` — ou `INSTALLATION INCOMPLETE` si un import manque.

Barème. Import de `sys` (1 pt) · 4 versions/chemins affichés (4×1 pt) · le `try/except` qui attrape le paquet manquant (2 pts) · le message final conditionnel (2 pts). Le rapport attendu sur l'environnement de l'atelier (mesuré le 19/09/2026) :

```
Python 3.13.14
pandas 2.2.3
numpy 2.3.5
Python actif : /usr/local/bin/python3
Installation OK
```

11. Exercices autonomes

E1 — L'écho. Écrire `echo.py` qui demande une chaîne et l'affiche en MAJUSCULES. (2 min)

E2 — Le venv chronométré. À partir d'un dossier vierge : créer le venv, l'activer, installer pandas, écrire `requirements.txt`. Chronométrer : l'objectif professionnel est **moins de 3 minutes** sur une connexion correcte (la première installation de pandas prend plus longtemps — c'est le téléchargement, pas l'installation). (10 min)

E3 — L'exigence cassée. Dans un venv, exécuter `python -m pip install "pandas==1.5.3"` — lire le message d'erreur ou d'avertissement **ligne par ligne** : qui parle (pip), quoi (conflict of dependencies), et pourquoi. (10 min — l'objectif est de **savoir lire** le message, pas de le faire disparaître.)

E4 — Le noyau qui oublie. Dans un notebook : cellule A `total = 100`, cellule B `print(total)`, cellule C « réinitialiser le noyau », cellule D `print(total)`. Prédire les 4 sorties **avant** d'exécuter, puis vérifier. (5 min)

12. Correction détaillée

Exercice guidé. Le script qui fait l'affaire (les 5 lignes de code, docstring comprise) :

```
"""Rapport d'installation du projet : versions et chemin du Python actif."""
import sys

print(f"Python {sys.version.split()[0]}")
for paquet in ("pandas", "numpy"):
    try:
        mod = __import__(paquet)
        print(f"{paquet} {mod.__version__}")
    except ImportError:
        print(f"{paquet} ABSENT")
print(f"Python actif : {sys.executable}")
print("Installation OK" if __import__("pandas") and __import__("numpy")
      else "INSTALLATION INCOMPLETE")
```

Deux points de correction : `sys.version.split()[0]` prend le numéro seul (3.13.14) sans le texte qui suit ; et le `try/except ImportError` est **spécifique** — jamais `except:` tout nu (C05 §6), sinon une vraie panne (disque plein, venv corrompu) serait avalée et le rapport dirait « OK » à tort.

E1. `nom = input("Chaîne ? "); print(nom.upper())` — la méthode `str.upper()` est vue d'office en Python (pas de `pip` : c'est le langage de base, couche 2).

E2. La séquence : `python -m venv .venv` → `source .venv/bin/activate` → `python -m pip install pandas` → `pip freeze > requirements.txt`. Le `pip freeze` **après** l'installation (sinon le fichier est vide) ; la flèche `>` **écrase** un fichier existant — utiliser `>>` pour ajouter.

E3. Le message de pip commence par le sujet (ERROR: Cannot install ...), nomme les deux paquets en conflit, et termine par la **raison** (depends on numpy>=...). La lecture se fait **de haut en bas** : le message est en dernier, la cause en premier — la même discipline que la traceback (C05 §7).

E4. Sorties prédites : A — rien (pas de `print`), B — `100`, C — rien (le noyau réinitialise), D — `NameError: name 'total' is not defined`. La cellule D est la preuve : le texte de B est intact, **l'effet** de A est parti avec le noyau.

13. Mini-projet M08.P1 — « L'audit d'environnement » (1h)

Énoncé. Écrire `audit_environnement.py` — le script qui produira la **section 0** du grand projet M08.P. Il doit écrire dans `rapport_environnement.txt` (pas `print`) :

1. la date et l'heure de l'exécution ;
2. la version de Python et de pandas/numpy ;
3. le chemin du Python actif et du dossier du projet ;
4. la liste des paquets installés dans le venv (via `pip freeze`) ;
5. une ligne de verdict : `AUDIT OK / AUDIT KO: ...`.

Le rapport du 19/09/2026 dans l'atelier commence par :

```
# Audit d'environnement – M08
2026-09-19
Python 3.13.14
pandas 2.2.3
numpy 2.3.5
Python actif : /usr/local/bin/python3
```

Critères de réussite (grille /10) : le fichier est **écrit** (`open(..., "w")`, encodage `utf-8 — C05 §8`) · 5 sections présentes · aucune version en dur (tout est lu à l'exécution) · le verdict est **calculé**, pas affiché en dur · le script tourne deux fois de suite avec le même résultat (reproductible — la règle du socle).

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise 5 outils, classés de la couche 2 à la couche 4 : l'interpréteur Python 3.13 (la base), `pip` (la boutique), `venv` (la trousse scellée), VS Code (l'atelier, cité), Jupyter (la table à dessin, noyau exécuté dans l'atelier).

Outil	Usage	Commande
Python 3.13	interpréter le code	<code>python --version</code> (Win) / <code>python3 --version</code> (Linux)
<code>pip</code>	installer les paquets	<code>python -m pip install pandas</code>
<code>venv</code>	isoler le projet	<code>python -m venv .venv</code> puis <code>source .venv/bin/activate</code> (Win : <code>.venv\Scripts\activate</code>)
VS Code	éditer + terminal intégré	ouvrir le dossier du projet ; Ctrl+ pour le terminal
Jupyter	cellules + noyau en mémoire	<code>jupyter lab</code> (venv activé, noyau pointant sur le venv)

15. Résumé du chapitre

- L'environnement Python a **4 couches** (machine, interpréteur, paquets, projet) ; chaque symptôme d'installation se rattache à une couche — la nommer, c'est diagnostiquer.
- L'installation Windows se vérifie par `python --version` ; la case « **Add python.exe to PATH** » est le piège n° 1.
- `pip` installe des paquets ; `python -m pip` garantit qu'ils vont dans le Python qu'on exécute ; `requirements.txt` (avec `==`) fige l'installation.
- Le **venv** est la trousse scellée du projet : créer, activer (le préfixe est la preuve), installer dedans, figer, reconstruire chez l'autre.
- Jupyter exécute **par cellules** mais se souvient **par noyau** : l'ordre d'exécution fait la mémoire, pas l'ordre d'écriture — la `NameError` du §5.6 est la preuve.
- Votre premier script tourne (terminal **et** notebook) ; la conversation avec la machine est ouverte. Tout le reste du module, c'est la grammaire de cette conversation.

16. À retenir

À retenir. Le PATH se vérifie, il ne se croit pas : `python --version` dans le terminal est la seule preuve que la couche 2 est en place — tout le reste (raccourci, programme ajouté) est suspect.

À retenir. Un paquet « installé » qui ne se `importe` pas est dans la **mauvaise trousse** : `pip list` dans le **venv actif** avant de relancer l'installation. Le venv ne se partage pas, il se reconstruit — le `requirements.txt` est le ticket de caisse, pas la caisse.

17. Évaluation formative (auto-correction, 8 min)

1. **Quelles sont les 4 couches de l'environnement Python ?** → machine · interpréteur · paquets · projet. (1 pt)
2. **Pourquoi `python -m pip install` plutôt que `pip install` ?** → ça force l'installation dans le Python qu'on exécute (évite le pip/python décalés, §5.8 n° 5). (1 pt)
3. **Qu'est-ce qui prouve qu'un venv est activé ?** → le préfixe du terminal (ex. `(.venv)`) — et `python -m site` montre le chemin du venv. (1 pt)
4. **Une cellule Jupyter utilise `x` défini dans la cellule écrite au-dessus : est-ce garanti ?** → non — seul l'ordre d'exécution compte ; si la définition n'a pas été exécutée, `NameError`. (1 pt)
5. **`ModuleNotFoundError: No module named 'pandas'` alors que « pandas est installé » : les 2 causes les plus probables ?** → paquet dans le Python système (venv actif sans pandas) ; noyau Jupyter attaché au mauvais Python. (2 pts)
6. **Pourquoi le venv ne va pas dans Git ?** → il contient des chemins absolus et des binaires machine-dépendants ; il se reconstruit par `python -m venv .venv + pip install -r requirements.txt`. (1 pt)
7. **Une installation fraîche de pandas en 2026 donne quelle version, et quelle version figure dans ce manuel (et pourquoi) ?** → 3.0.6 (dernière publiée au 19/09/2026) ; le manuel est mesuré sur 2.2.3 (M01-M07) et enseigne les patterns valides sur les deux, avec annexe de migration (C08 §8). (2 pts)
8. **Écrire la commande qui fige l'environnement du venv actif dans `requirements.txt`.** → `pip freeze > requirements.txt` (Linux) — le `>` écrase, le `>>` ajoute. (1 pt)

Module M08.C02 — Les fondations sans données : variables, types, opérateurs, entrées/sorties, commentaires, erreurs

Outils : Python 3.13.14 seul (l'interpréteur, la couche 2 de C01), le terminal. **Aucun paquet n'est requis — ce chapitre tourne sur le Python nu.** **Durée indicative :** 4 h. **Niveau :** N2 → N3. **Prérequis :** C01 (un environnement qui marche).

L'idée du chapitre. C01 a ouvert la conversation avec la machine ; C02 installe la **grammaire** de cette conversation : les **variables** (les noms qu'on donne aux valeurs), les **types** (les familles de valeurs et les opérations qu'elles autorisent), les **opérateurs** (les `+`, `and` et not du métier), les **entrées/sorties** (`input/print`), les **commentaires** (ce qu'on dit au lecteur humain) — et la compétence qui en vaut dix : **lire une erreur** sans paniquer. Rien ne dépend encore des données : les CSV de `dossier_M08/` attendent C07 ; ici, tout est mesuré sur des montants de comptoir (1 500 FCFA, une remise de 10 %, un net de 1 350 FCFA) que vous retrouverez à chaque chapitre. Le fil rouge est le **calculateur de remise** : il naît ici en 4 lignes, deviendra une fonction en C05, un calcul sur table en C07, et un calcul sur 50 008 lignes en C08.

Matériel de l'atelier — Python 3.13.14, terminal, zéro paquet. Tout le code de ce chapitre a été exécuté dans l'atelier le 19/09/2026 ; les sorties publiées sont **celles de l'atelier**. Les montants du chapitre sont sourcés (`m08p_demo_*`) : prix 1 500, remise 10, net 1 350, seuil comptoir 1 000 — les mêmes valeurs que \$6 de C01, votre référence de relecture : si votre machine renvoie autre chose, l'environnement ne tourne pas comme prévu.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Écrire, **renommer** et réaffecter une variable — et expliquer pourquoi rebaptiser ne duplique pas.
2. Dire le **type** d'une valeur (`int`, `float`, `str`, `bool`) et **convertir** un texte en nombre (la recette du format français inclus).
3. Utiliser les opérateurs arithmétiques (`+` `-` `*` `/` `//` `%` `**`), les comparaisons chaînées, et les opérateurs booléens `and/or/not` — avec le parallèle `AND/OR/NOT` de M07.
4. Lire une entrée avec `input()` **en sachant qu'elle arrive toujours en texte**, et afficher avec `print()` (f-strings incluses).
5. **Lire une traceback** ligne par ligne : le fichier, la ligne, la flèche, le message — et dire laquelle des quatre familles d'erreurs vous a (`SyntaxError`, `NameError`, `ValueError`, `TypeError`).
6. Écrire un calculateur de remise en 4 lignes qui tourne dans le terminal.

2. Pourquoi cette notion est importante

Parce que 90 % des erreurs de débutant ne sont pas des erreurs de logique, ce sont des erreurs de **grammaire** : un `int` additionné à un `str`, une virgule dans un nombre, un nom mal recopié. Chacune a un **nom** (`TypeError`, `ValueError`, `NameError`) et un **remède systématique** — ce chapitre donne les trois noms et les trois remèdes. Un apprenant qui reconnaît l'erreur par son nom divise son temps de dépannage par quatre ; un apprenant qui la lit ligne par ligne finit par dépanner **lui-même**.

Deuxième raison : les types, vous les connaissez déjà — M06 a installé la notion sur les colonnes (INTEGER, DECIMAL, VARCHAR). En Python, c'est la **même idée** sur les valeurs individuelles : chaque valeur a une famille, et la famille autorise (ou interdit) les opérations. Le parallèle rapporte : vous ne pouvez pas additionner une colonne texte et une colonne nombre en SQL ; vous ne pouvez pas additionner un `str` et un `int` en Python. Une seule notion, deux langages.

Troisième raison, professionnelle : `input()` arrive **toujours en texte**. C'est la porte par laquelle le monde réel entre dans votre script — et le monde réel écrit ses montants `15 000,50`, pas `15000.50`. La **conversion à la porte** (dès la réception, pas au milieu du calcul) est l'habitude qui fera la différence entre un script qui tourne et un script qui s'effondre à la première saisie.

3. Explication simple — l'étiquette, la famille, la porte

- La **variable** est une **étiquette** collée sur une valeur. Écrire `prix = 1500`, c'est coller l'étiquette « prix » sur le nombre 1 500. Re-coller l'étiquette ailleurs (`prix = 2000`) ne **copie** rien : l'ancienne étiquette est simplement re-collée, et si personne n'en porte d'autre, l'ancien nombre est jeté.
- Le **type** est la **famille** de la valeur. La famille décide ce qu'on peut faire : on peut additionner deux nombres, on ne peut pas additionner un nombre et un mot. En M06, la famille était déclarée **sur la colonne** (DECIMAL ici, VARCHAR là) ; en Python, elle est portée par **chaque valeur** et se voit avec `type(x)`.
- `input()` est une **porte** : ce qui entre, entre **en texte**, toujours — même si le visiteur a dit « 1500 ». C'est à vous de le **convertir** à la porte, sinon il traînera sa voix de texte dans tout le script.
- Une **erreur** est une **protestation lisible** : la machine dit où (fichier, ligne), quoi (le message, en bas), et souvent pourquoi (la flèche qui souligne le coupable). La protester, ce n'est pas échouer ; c'est le travail. La paniquer, c'est perdre.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
variable	un nom qui pointe vers une valeur (pas une boîte qui la contient).	dire « la variable contient le nombre » : rebaptiser la variable ne duplique pas la valeur.
type — type	la famille d'une valeur, qui autorise des opérations (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>).	croire que <code>1</code> , <code>1.0</code> et <code>"1"</code> sont équivalents : trois objets, trois familles, trois comportements.
conversion — cast	transformer une valeur d'une famille à une autre (<code>int("1500")</code> , <code>float("1500.5")</code>).	convertir au milieu du calcul au lieu de la porte : le texte traîne, l'erreur arrive plus tard, plus loin, plus mystérieuse.
opérateur booléen — boolean operator	<code>and</code> , <code>or</code> , <code>not</code> : les connecteurs qui combinent des conditions.	oublier la priorité <code>not > and > or</code> — la même que <code>NOT > AND > OR</code> en SQL (M07 C02).
traceback — traceback	le rapport d'erreur : pile d'appels, ligne fautive, flèche, message.	le lire de bas en haut : la dernière ligne dit quoi , les lignes du haut disent pourquoi .
faux en condition — falsy	une valeur que Python traite comme <code>False</code> dans une condition : <code>0</code> , <code>""</code> , <code>[]</code> , <code>None</code> .	écrire <code>if montant:</code> et ne pas comprendre pourquoi un montant de <code>0</code> passe pour « absent ».

5. Cours approfondi

5.1 La variable — l'étiquette qui se re-colle

```
prix = 1500
print(prix)
prix = 2000
print(prix)
```

Sortie réelle de l'atelier (19/09/2026) :

```
1500
2000
```

Définition. variable — variable — un nom qui pointe vers une valeur : `prix = 1500` colle l'étiquette « prix » sur le nombre 1 500 ; réaffecter (`prix = 2000`) re-colle l'étiquette, sans copier quoi que ce soit. C'est la brique de base de tout script.

Deux remarques qui préviennent des malentendus qui durent :

1. **Aucune « boîte » n'existe.** `prix` n'est pas un contenant dans lequel on met 1 500 puis 2 000 : c'est un nom. Quand vous écrivez `autre = prix`, les **deux** noms pointent vers le **même** objet (pour les nombres, qui ne changent pas, l'effet est invisible — pour les listes de C04, il ne l'est pas du tout).
2. **Le nom est un contrat, pas un décor.** `prix` dit au lecteur ce que la valeur est ; `p2` ne dit rien. La règle M04 (conventions de nommage) s'applique telle quelle en Python : des noms qui racontent, en français ou en anglais, pas les deux mélangés.

5.2 Les 4 types de base — et la conversion à la porte

Les quatre familles du chapitre (les conteneurs — listes, dictionnaires — arrivent en C04) :

```
type(1500)      # un entier
type(1500.5)    # un nombre à virgule
type("1500")   # un texte
type(True)     # un booléen
```

Sortie réelle :

```
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

Définition. type — type — la famille d'une valeur, qui autorise (ou interdit) les opérations : `int` (entier), `float` (nombre à virgule), `str` (texte), `bool` (vrai/faux). `type(x)` la dit. En M06, la famille était déclarée sur la colonne ; en Python, elle est portée par chaque valeur — une seule notion, deux langages.

La conversion, c'est changer de famille — et c'est là que le **format français** entre en jeu. `int("1500")` passe ; `int("1500,50")` se proteste, avec sa sortie exacte :

```
ValueError: invalid literal for int() with base 10: '1500,50'
```

La **recette du format français** (l'outil que vous utiliserez toute la formation sur les montants) : supprimer les espaces, remplacer la virgule par le point, puis convertir :

```

valeur = float("15 000,50".replace(" ", "").replace(",", "."))
print(valeur)

```

```
15000.5
```

Conseil professionnel. Convertissez à la **porte** : dès que la valeur arrive (`input()`, lecture de fichier en C07), faites-en un nombre — et nommez la variable d’après sa **valeur**, pas d’après sa forme (`montant`, pas `montant_texte` une fois converti). Un texte qui traîne dans le calcul ne fait pas d’erreur tout de suite : il en fait une plus tard, plus loin, sur une ligne qui n’a rien à voir — c’est l’erreur la plus coûteuse à chercher.

5.3 Les opérateurs — l’arithmétique du comptoir

L’arithmétique, exécutée dans l’atelier :

```
print(17 // 5, 17 % 5, 2 ** 8)
```

```
3 2 256
```

Trois opérateurs que le clavier ne vous a pas appris : `//` la **division entière** ($17 // 5 = 3$ — l’entier du quotient, ce qu’une caisse garde), `%` le **reste** ($17 \% 5 = 2$ — ce qui dépasse), `**` la puissance ($2 ** 8 = 256$). Le `/` classique renvoie un `float` ($17 / 5 = 3.4$).

Les comparaisons se **chaînent** comme en mathématiques — l’équivalent SQL serait deux conditions `AND`-ées :

```
print(1000 <= 1200 < 1500)
```

```
True
```

Et les trois opérateurs booléens, le **parallèle SQL exact** (M07 C02 : `AND` se lie avant `OR`) :

```

print("and :", True and False)
print("or  :", True or False)
print("not :", not False)

```

```

and : False
or  : True
not : True

```

Python	SQL	Sens
<code>and</code>	<code>AND</code>	les deux conditions vraies
<code>or</code>	<code>OR</code>	l’ une des deux vraie
<code>not</code>	<code>NOT</code>	l’inverse

La priorité est la même des deux côtés : `not` d’abord, puis `and`, puis `or`. Et la même règle de style que M07 : **parenthéser pour clarifier** — une condition lue par un collègue doit ne pouvoir se lire qu’une façon.

5.4 `input()` et `print()` — la porte et la vitrine

`input()` demande, et **renvoie toujours un `str`** — la sortie de l’atelier le prouve (l’entrée `15 000,50` fournie par `echo`, le même effet que de la taper) :

```
montant = input("Montant (FCFA) ? ")
print(type(montant))
valeur = float(montant.replace(" ", "").replace(",", "."))
print(f"montant lisible : {valeur:,.2f}")
```

```
Montant (FCFA) ? <class 'str'>
montant lisible : 15,000.50
```

Attention. `input()` renvoie **toujours** un texte, même quand le visiteur dit « 1500 ». `int(input("..."))` sans la conversion du format français s’effondre sur la **première** saisie 15 000,50 — et c’est la porte, pas le calcul, qui doit faire la conversion.

`print()` affiche ; la **f-string** (le `f"... {valeur} ..."`) insère les valeurs dans le texte — le formatage de montants que vous ferez toute la formation.

Définition. f-string — f-string — une chaîne précédée de `f` où chaque `{...}` est remplacé par la valeur : `f"Total : {total:,.2f}"` affiche le total avec 2 décimales. C’est la forme de sortie de tout le module ; le `:,.2f` est la **specification de format** (virgule milliers, 2 décimales — la convention américaine par défaut, voir l’encadré ci-dessous).

Dans les faits. Le formatage par défaut de Python est **américain** : `{valeur:,.2f}` écrit 15,000.50 (virgule = milliers, point = décimales). La convention française de la formation (espace = milliers, virgule = décimales — règle M01) s’écrit à la main : `f"{valeur:,.2f}".replace(","," ").replace(".",",")` → 15 000,50. Le chapitre publie la sortie telle qu’elle s’affiche (américaine) et dit quel est le format de la maison.

5.5 Les commentaires — le « pourquoi », pas le « quoi »

Le commentaire (`#`) est un mot dit **au lecteur humain** ; la machine le saute. La règle M04, reprise telle quelle : **commenter le pourquoi, pas le quoi**.

```
# quoi (inutile : le code le dit)
remise = 0.10

# pourquoi (utile : le code ne le dit pas)
remise = 0.10 # remise commerciale de septembre – voir le mémo du 05/09
```

Le premier commentaire répète le code (il mentra dès qu’on changera la valeur) ; le second conserve une **décision** (d’où vient le 10 %). La **docstring** — le commentaire en tête de fonction, entre triples guillemets — arrive en C05 ; elle est la version structurée de la même règle.

5.6 Lire une erreur — l’anatomie, en 3 protestations réelles

Les quatre familles d’erreurs du débutant, et les trois que l’atelier a exécutées le 19/09/2026.

NameError — le nom n’existe pas (orthographe, ou variable d’une autre cellule en Jupyter) :

```
print(montannt)
~~~~~
NameError: name 'montannt' is not defined
```

ValueError — le nom existe, la **valeur** est la mauvaise (le texte n’est pas un nombre lisible par `int()`) :

```
int('1500,50')
~~~~~
ValueError: invalid literal for int() with base 10: '1500,50'
```

SyntaxError — le **texte** du script ne se lit pas (ici : le deux-points qui manque après `if`) :

```
File "<string>", line 1
  if 3 > 2
      ^
SyntaxError: expected ':'
```

Définition. traceback — `traceback` — le rapport d'erreur de Python : il donne le chemin parcouru (la pile d'appels, du haut vers le bas), la **ligne fautive** (le numéro et le texte), la **flèche** qui souligne le coupable, et le **message** (le type d'erreur + une phrase). C'est le document à savoir lire avant d'écrire du code avancé.

L'anatomie, sur l'exemple `ValueError` — **quatre choses à relever**, toujours dans cet ordre :

1. **Le message, en bas** : `ValueError` + la phrase. Le **type** (`ValueError`) est le nom de la famille — c'est lui qu'on retient, pas la phrase.
2. **La ligne fautive** : le numéro de ligne (ou le `File "..."`) — c'est où.
3. **La flèche** : elle souligne l'endroit exact dans la ligne — le coupable, pas le voisin.
4. **La pile, au-dessus** (lorsque le script a des fonctions, C05) : les lignes au-dessus racontent **par quel chemin** on est arrivé là — on les lit de haut en bas.

La discipline : **lire de haut en bas pour le pourquoi, arrêter à la dernière ligne pour le quoi**. La dernière ligne dit toujours quel type d'erreur ; les lignes du haut disent d'où elle vient. En Jupyter, la ligne fautive est dans la **cellule exécutée** — pas celle qui est écrite au-dessus (C01 §5.6).

5.7 `None` et les valeurs « fausses » en condition

Quatre valeurs que Python traite comme `False` dans une `if` — exécuté dans l'atelier :

```
print("0      :", bool(0))
print('""     : ', bool(""))
print("[]     :", bool([]))
print("1500   :", bool(1500))
```

```
0      : False
""     : False
[]     : False
1500   : True
```

Attention. `0`, `""` (le texte vide), `[]` (la liste vide) — et `None`, l'absence totale — sont **faux en condition**. `if montant:` passe pour tout montant **sauf** `0` : un montant de 0 FCFA est une valeur **présente** (une vente à zéro, une remise à 100 %), pas une absence. Pour tester « est-ce que la valeur existe », tester `est None` ; pour tester « est-ce que la valeur est nulle », tester `== 0`. Deux questions différentes, deux tests différents.

Le tableau complet du chapitre — les 4 familles, les opérations qu'elles autorisent, les 4 protestations qui les suivent :

Les 4 types de base : la famille autorise les opérations

type(x) dit la famille ; la famille décide ce qui passe et ce qui proteste.

Type	+ (addition)	comparaison	conversion	en condition
int	1500 + 500 = 2000 17 // 5 = 3 · 17 % 5 = 2	1500 < 2000 → True 1000 <= 1200 < 1500 (chaînée)	int('1500') → 1500 str(1500) → '1500'	1500 → True 0 → False (le piège du if)
float	1500.5 + 0.5 = 1501.0 500 * 1.19 = 595.0 (TVA 2026)	1500.5 < 2000 → True 0.1 + 0.2 != 0.3 (approximatif)	float('1500.5') → 1500.5 le point, pas la virgule	1500.5 → True 0.0 → False
str	'15' + '00' = '1500' (concat.) pas d'addition de nombres	'Ouaga' < 'Bobo' → True (alpha) la casse compte : 'A' < 'a'	int('1500,50') → ValueError 'x' → True (texte non vide) recette : replace puis float	" → False (texte vide)
bool	and · or · not (pas de +) priorité : not, and, or (comme SQL)	True == True → True True == 1 → True (attention)	bool(0) → False bool(1500) → True	True / False le if les lit directement
list · tuple · dict · set les conteneurs — C04 : des listes de ces 4 types, des clés vers valeurs				

Les 4 protestations du débutant (C02 §5.6) — le nom de la famille d'abord :

SyntaxError — le texte ne se lit pas (deux-points manquant)
 NameError — le nom n'existe pas (montant, ou cellule Jupyter non exécutée)
 TypeError — mauvaise famille (1500 + 'abc')
 ValueError — mauvaise valeur (int('1500,50'))

6. Exemple concret — le calculateur de remise

La question du comptoir, en langage de comptoir : « l'article coûte 1 500 FCFA, la remise est de 10 %, quel est le prix net ? » Le script le plus court qui y répond — exécuté dans l'atelier le 19/09/2026 (les deux réponses, 1500 puis 10, fournies par printf) :

```
prix = float(input("Prix (FCFA) ? "))
remise_pct = float(input("Remise (pct) ? "))
net = prix * (1 - remise_pct / 100)
print(f"Prix net : {net:,.2f} FCFA")
```

```
Prix (FCFA) ? Remise (pct) ? Prix net : 1,350.00 FCFA
```

Le net est **1 350 FCFA** (clé m08p_demo_prix_net) — la valeur de référence du chapitre. Le parallèle SQL, déjà actif : en M07, le même calcul était une colonne calculée dans le SELECT (SELECT prix_vente_ht * (1 - 0.10) AS prix_net FROM produit;). La **formule** est identique ; le **lieu** change : une ligne de table (SQL) vs une ligne de script (Python). En C07, la même ligne s'appliquera à 50 008 lignes d'un coup — c'est toute la promesse du module.

7. Démonstration pas à pas — 5 étapes sur le fil rouge

Du zéro au calculateur, les 5 étapes, sortie réelle de chaque commande (exécutées le 19/09/2026).

Étape 1 — L'arithmétique d'abord (le calcul nu, sans variable) :

```
$ python3 -c "print(500 * 1.19)"
595.0
```

Interprétation : $500 + 19\%$ de TVA (2026, la règle du socle) = 595 — la machine calcule, on n'a pas encore de nom.

Étape 2 — Donner un nom (la variable) :

```
$ python3 -c "x = 10; x = x + 5; print(x)"
15
```

Interprétation : `x = x + 5` se lit « pointe x vers la valeur de x plus 5 » — l'étiquette se re-colle, rien n'est dupliqué.

Étape 3 — La conversion à la porte (le format français) :

```
$ python3 -c "print(float('15 000,50'.replace(' ', '').replace(',', '.')))"
15000.5
```

Interprétation : le texte `15 000,50` est devenu le nombre 15 000,50 avant tout calcul — la porte a fait son travail.

Étape 4 — Décider (la condition, le parallèle `WHERE`) :

```
$ python3 -c "prix = 1200; print('dossier' if prix > 1000 else 'comptant')"
dossier
```

Interprétation : le seuil comptoir est 1 000 FCFA (`m08p_demo_seuil`) ; 1 200 passe au-dessus — en SQL, c'était un `CASE WHEN prix > 1000 THEN 'dossier' ...` (M07 C06).

Étape 5 — Le calculateur complet (les 4 lignes de §6) :

```
$ printf "1500\n10\n" | python3 calculateur.py
Prix (FCFA) ? Remise (pct) ? Prix net : 1,350.00 FCFA
```

Interprétation : entrées lues, conversion faite, calcul fait, sortie lisible — la conversation de C01 est devenue un travail.

8. Erreurs fréquentes

1. **`TypeError: unsupported operand type(s) for +: 'int' and 'str'`** — l'erreur n° 1 du débutant : un texte additionné à un nombre. Le texte est presque toujours le fruit d'un `input()` non converti (ou d'un `str` collé dans un calcul). Remède : convertir **à la porte** (§5.2).
2. **Le `1500,50` qui devient un couple.** Écrire `x = 1500,50` dans le code crée un **tuple** (une paire), pas un nombre à virgule — exécuté dans l'atelier : `text $ python3 -c "x = 1500,50; print(type(x), x)"` `<class 'tuple'> (1500, 50)` La virgule de Python est le **séparateur d'arguments**, pas la virgule décimale. Le nombre à virgule s'écrit `1500.5` (point) — le format français est affaire de **texte**, pas de nombre.
3. **= dans une condition.** `if x = 5:` est une **affectation** déguisée en comparaison → `SyntaxError`. La comparaison est `==` (deux signes) ; l'affectation, un.
4. **`int("1500,50")` → `ValueError`.** La famille `int` ne lit que les entiers en base 10 — le point décimal **et** la virgule lui sont illisibles. Recette du format français (§5.2).
5. **L'indentation qui change le sens.** En Python, l'indentation **est** la syntaxe : une ligne dedans de 4 espaces est « dans le bloc », en dehors, elle s'exécute quoi qu'il arrive. Un `print` qu'on attendait « seulement si » et qui s'exécute toujours est une histoire d'indentation.
6. **`0.1 + 0.2 != 0.3`.** Exécuté dans l'atelier : `text $ python3 -c "print(0.1 + 0.2)"` `0.30000000000000004` Les nombres à virgule sont stockés **approximativement** (le binaire ne sait pas dire 0,1). Pour l'égalité de deux `float`, comparer avec une tolérance — et pour l'argent, on verra en C08 la solution de fond : compter en **centimes entiers**.

9. Bonnes pratiques professionnelles

1. **Le nom dit la valeur, pas la forme** : `montant` une fois converti, jamais `montant_texte` ; `remise_pct` (en pourcent) vs `remise` (en fraction) — le suffixe évite le bug du facteur 100.
2. **Un calcul par variable intermédiaire** : `net = prix * (1 - remise_pct / 100)` se lit d'un coup d'œil ; `prix * 0.9` force le lecteur à recalculer le 0.9. Les variables intermédiaires sont la documentation vivante du calcul.
3. **Convertir à la porte, tester à la porte** : dès la réception, `float(...)` — et si la valeur arrive de nulle part (fichier, API en C07), une valeur illisible doit **se voir** tout de suite, pas trois calculs plus tard.
4. **Compter l'argent en centimes** dès qu'un total s'annonce (C08) : les `float` se trompent d'un centime sur 50 000 lignes ; les entiers, jamais.
5. **Laisser une erreur s'afficher** plutôt que de l'avaler : un `try/except` qui « répare » silencieusement masque la porte qui fuit (C05 §6) — l'erreur vue est l'erreur payée une fois.

10. Exercice guidé — « le ticket de caisse » (15 min, /10)

Énoncé. Écrire `ticket.py` qui, dans l'ordre :

1. lit un **prix** (au format français, ex. 15 000,50) et une **quantité** (entier) ;
2. calcule le total (`prix × quantité`) ;
3. affiche le total avec 2 décimales ;
4. affiche la mention `Seuil dépassé : dossier requis` **si** le total dépasse le seuil comptoir de 1 000 FCFA, et rien sinon.

Barème. Les deux lectures avec conversion à la porte (4 pts) · le total en une variable nommée (2 pts) · l'affichage 2 décimales (2 pts) · la condition avec le seuil **en variable nommée** (2 pts). Sortie attendue pour l'entrée 15 000,50 / 3 (vérifiée par exécution le 19/09/2026) :

```
Total : 45,001.50
Seuil dépassé : dossier requis
```

(le 45 001,50 vient de 15 000,50 × 3 — l'apprenant qui annonce « 45 000 » est tombé dans l'arrondi mental ; c'est le point de l'exercice, pas un piège.)

11. Exercices autonomes

Exercice 2.1 — L'échange. Sans variable temporaire, échanger les valeurs de `a` et `b` (`a = 5`, `b = 8`). Indice : Python autorise `a, b = b, a`. (3 min)

Exercice 2.2 — La chaîne. Écrire **une** expression (pas deux conditions `and`-ées) qui vaut `True` si un prix est compris entre 100 et 1 000 FCFA **bornes comprises**. (5 min)

Exercice 2.3 — La prédiction. Prédire **sur papier** la sortie de `print(0.1 + 0.2)` et de `print(0.1 + 0.2 == 0.3)`, puis exécuter et comparer. Si la prédiction a raté, écrire en une phrase pourquoi (indice : §8, erreur n° 6). (5 min)

Exercice 2.4 — Le plus cher. Lire deux prix et afficher le plus cher — avec `max()`, puis **sans** `max()` (avec une `if`). Les deux sorties doivent être identiques. (10 min)

Exercice 2.5 — Le couple piégé. Un collègue a écrit `x = 1500,50` et s'étonne que `x * 2` ne donne pas 3 001. Expliquer en 3 lignes ce que `x` est vraiment, et écrire la ligne corrigée. (5 min)

12. Correction détaillée

Exercice guidé. Le script qui fait l'affaire :

```
"""Ticket de caisse : total, seuil comptoir, mention dossier."""
prix = float(input("Prix (FCFA, ex. 15 000,50) ? ").replace(" ", "").replace(",", "."))
quantite = int(input("Quantité ? "))
total = prix * quantite
print(f"Total : {total:,.2f}")
seuil = 1000 # seuil comptoir – au-delà, dossier requis
if total > seuil:
    print("Seuil dépassé : dossier requis")
```

Deux points de correction : la conversion est **à la porte** (les deux `input` convertis sur place) ; le seuil est une **variable nommée** (`seuil`), pas un 1 000 en dur dans la condition — si le seuil change, une ligne change, pas deux.

Exercice 2.1. `a, b = b, a` — Python évalue d'abord le membre de droite (la paire `b, a`), puis distribue : c'est l'échange en une ligne, sans variable temporaire. (Le tuple, C04.)

Exercice 2.2. `100 <= prix <= 1000` — la comparaison chaînée (§5.3), bornes comprises par les `<=`. L'écriture `and-ée` (`prix >= 100 and prix <= 1000`) est correcte mais plus longue ; la chaîne est la forme lue en mathématiques.

Exercice 2.3. Prédiction : `0.30000000000000004` et `False`. Explication en une phrase : les nombres à virgule sont stockés approximativement en binaire, donc `0.1 + 0.2` n'est pas **exactement** `0.3` — l'égalité stricte échoue (clé `m08p_demo_float_add`).

Exercice 2.4. Avec : `print(max(p1, p2))`. Sans : `print(p1 if p1 >= p2 else p2)` — l'expression conditionnelle (la forme `A if condition else B` d'Étape 4), la CASE WHEN de M07 C06 en une ligne.

Exercice 2.5. `x` est un **tuple** de deux entiers, `(1500, 50)` — la virgule est le séparateur d'arguments de Python, pas une virgule décimale (§8, erreur n° 2). La ligne corrigée : `x = 1500.5` (point, un seul nombre).

13. Mini-projet M08.P2 — « La caisse » (1 h)

Énoncé. Écrire `caisse.py` — le calculateur de remise de \$6, durci pour le comptoir :

- lire le **prix** au format français (avec ou sans espaces, virgule ou point) ;
- lire la **quantité** (entier ≥ 1) ;
- calculer le total, puis appliquer une **remise de 5 % si la quantité est de 10 ou plus** ;
- afficher : le total brut, la remise appliquée (0 si inapplicable), le net — 3 lignes, 2 décimales chacune ;
- refuser poliment (sans s'effondrer) la quantité « 0 » ou négative.

Critères de réussite (grille /10) : les 5 exigences (5×1 pt) · le seuil de remise et le pourcentage sont des **variables nommées** en tête (1 pt) · la sortie de l'entrée 1500 / 12 donne total brut 18 000,00 · remise 900,00 · net 17 100,00 (1 pt) · aucune erreur sur les saisies pièges : `abc`, `15,0`, `0`, `-3` (1 pt) · docstring en tête, commentaires « pourquoi » (1 pt).

Note de correction. Les sorties du critère sont mesurées : $1\,500 \times 12 = 18\,000$; 5 % de 18 000 = 900 ; net 17 100 — l'apprenant doit les **retrouver en exécutant**, pas en les lisant ici : c'est le principe de l'évaluation du module (prédire, exécuter, comparer).

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre n'utilise que l'interpréteur et le terminal (couche 2 de C01) — la boîte à outils se remplit au fur et à mesure : `type()`, les conversions, les f-strings, et la discipline de lecture des tracebacks.

Outil	Usage	Syntaxe
<code>python3 -c</code>	exécuter une ligne sans fichier	<code>python3 -c "print(17 // 5)"</code>
<code>type()</code>	dire la famille d'une valeur	<code>type(1500.5) → <class 'float'></code>
<code>int()</code> / <code>float()</code>	convertir texte → nombre	<code>float("1500.5")</code>
<code>input()</code> / <code>print()</code>	la porte et la vitrine	<code>x = input("...") · print(f"... {x} ...")</code>
<code>and</code> / <code>or</code> / <code>not</code>	combiner des conditions	<code>if a > 0 and b > 0:</code>
<code>//</code> · <code>%</code> · <code>**</code>	division entière · reste · puissance	<code>17 // 5 · 17 % 5 · 2 ** 8</code>

15. Résumé du chapitre

- La variable est une **étiquette** qui se re-colle : réaffecter ne duplique rien.
- Les 4 types de base (`int`, `float`, `str`, `bool`) portent les opérations ; la **conversion à la porte** (`replace(" ", "").replace(",",".")` puis `float()`) est l'habitude du format français.
- Les opérateurs booléens sont **les mêmes** qu'en SQL (`and/or/not` = `AND/OR/NOT`, même priorité) — le parallèle est l'accélérateur du module.
- `input()` arrive **toujours en texte** ; la porte convertit, le calcul ne s'en préoccupe plus.
- Une traceback se lit **de haut en bas pour le pourquoi**, s'arrête **en bas pour le quoi** ; les 4 familles du débutant (`SyntaxError`, `NameError`, `ValueError`, `TypeError`) ont chacune un remède systématique.
- Le calculateur de remise tourne ($1\ 500 - 10\ \% = 1\ 350$) — il attend C05 (deviendra fonction), C07 (deviendra calcul sur table) et C08 (deviendra calcul sur 50 008 lignes).

16. À retenir

À retenir. Une erreur a un **nom** avant d'avoir un sens : `TypeError` (mauvaise famille), `ValueError` (mauvaise valeur), `NameError` (mauvais nom), `SyntaxError` (mauvais texte). Nommer l'erreur, c'est déjà avoir le quart du remède.

À retenir. Le texte se convertit **à la porte** : `input()` et fichiers en C07 compris, une valeur illisible doit se voir **dès sa réception** — un texte qui traîne dans le calcul ne fait pas d'erreur tout de suite, il en fait une plus tard, plus loin, sur une ligne qui n'a rien à voir.

17. Évaluation formative (auto-correction, 8 min)

1. Que fait `prix = 1500` exactement, et que fait `prix = 2000` ensuite ? → colle l'étiquette « prix » sur 1 500 ; re-colle l'étiquette sur 2 000 (rien n'est dupliqué). (1 pt)
2. `type(1500)`, `type("1500")`, `type(True)` ? → `<class 'int'>`, `<class 'str'>`, `<class 'bool'>`. (1 pt)

3. **Pourquoi `int("1500,50")` échoue-t-il, et quelle est la recette du format français ?** → `int` ne lit que les entiers base 10 (la virgule est illisible) ; recette : `float(s.replace(" ", "").replace(",", "."))`. (2 pts)
4. **La priorité des opérateurs booléens, et son équivalent SQL ?** → `not > and > or` — la même que `NOT > AND > OR` en SQL (M07 C02). (1 pt)
5. **`input("...")` renvoie toujours quoi, et quelle en est la conséquence ?** → toujours un `str` ; la conversion doit se faire à la porte, sinon le texte traîne dans le calcul. (2 pts)
6. **Sur une traceback, où est le « quoi » et où est le « pourquoi » ?** → le message (type + phrase) est en bas ; la pile d'appels (lignes du haut) raconte d'où l'erreur vient. (1 pt)
7. **Pourquoi `x = 1500,50` ne crée pas un nombre à virgule ?** → la virgule est le séparateur d'arguments de Python : `x` est un tuple `(1500, 50)` ; le nombre s'écrit `1500.5` (point). (1 pt)
8. **`0.1 + 0.2` vaut quoi, et que faire pour de l'argent ?** → `0.30000000000000004` (stockage approximatif en binaire) ; comparer avec tolérance, et compter en centimes entiers pour les totaux (C08). (1 pt)

Module M08.C03 — Décider et répéter :

`if/elif/else`, **boucles** `for/while`, `break/continue`, **compréhensions**

Outils : Python 3.13.14 seul (l'interpréteur), le terminal. **Aucun paquet.** **Durée indicative :** 4 h. **Niveau :** N3 → N4. **Prérequis :** C02 (variables, types, erreurs).

L'idée du chapitre. C02 a donné la grammaire d'une **valeur** ; C03 donne le **flux** de plusieurs valeurs : **décider** (`if/elif/else` — l'échelle de décision, le `CASE WHEN` de M06 en Python), **répéter** (`for` le tapis roulant, `while` le « tant que »), **en sortir** (`break` la porte de sortie, `continue` le « passe, la suite »), et **résumer** en une ligne (les compréhensions). Le fil rouge : le calculateur de remise de C02 devient le **contrôle qualité express** — 10 montants, compter les anomalies, s'arrêter au 3^e dépassement, imprimer le rapport. C05 en fera une fonction ; C07, un calcul sur 50 008 lignes de table.

Matériel de l'atelier — Python 3.13.14, terminal, zéro paquet. Tout le code de ce chapitre a été exécuté dans l'atelier le 19/09/2026 ; les sorties publiées sont **celles de l'atelier**. Les montants et magasins cités sont ceux du socle M08 (clés `m08p_*`) : les 5 magasins réels (`magasin.csv`), le seuil comptoir 1 000, les montants 500, 1 200, 1 500, 15 000,50.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Écrire une **échelle de décision** `if/elif/else` avec des conditions nommées — l'équivalent Python du `CASE WHEN` de M06.
2. Parcourir une liste avec `for` (les 5 magasins), compter avec `range(n)`, et porter l'index **et** la valeur avec `enumerate`.
3. Écrire une boucle `while` « tant que » (le solde d'un crédit), reconnaître une **boucle infinie**, et savoir ce que `break` change.
4. Utiliser `break` (s'arrêter au premier dépassement) et `continue` (ignorer les lignes pourries) — les deux usages métier, plus le `for...else` « non trouvé ».
5. Écrire une **compréhension** `[expr for x in it if cond]` — et appliquer la **règle des 2 niveaux** : au-delà, on revient à une boucle.
6. Parcourir deux listes en parallèle avec `zip` (magasins + surfaces).

2. Pourquoi cette notion est importante

Le flux de contrôle, c'est le métier. Un script qui ne sait ni décider ni répéter ne fait qu'un calcul ; un script qui sait faire les deux fait un **travail** : trier, compter, arrêter, filtrer, résumer. C'est 80 % du code qu'on écrit au quotidien — et c'est là que vivent les deux pires familles d'erreurs du débutant : la **boucle qui ne s'arrête pas** (la machine tourne, tourne, tourne — ce chapitre la mesure en secondes réelles) et l'**indentation** (en Python, l'alignement **est** la syntaxe : une ligne mal alignée, c'est un `IndentationError`, pas un avertissement).

Le parallèle SQL rapporte, comme dans tout le module. Le `if/elif/else`, vous l'avez écrit en M06 : c'est le `CASE WHEN`, ligne par ligne au lieu de colonne par colonne. Et la boucle `for`, vous avez juste **délégué** ce travail au SGBD en SQL : quand vous écrivez `SELECT * FROM vente`, le moteur parcourt les 50 008 lignes **pour vous**, sans que vous écriviez de boucle. En Python, le

même parcours est à **vous d'écrire** — et c'est ce qui donne sa force : en SQL, on ne peut répéter que sur des **lignes de table** ; en Python, on répète sur **n'importe quoi** (des saisies, des fichiers, des résultats d'API — tout ce qui n'est pas encore en table).

3. Explication simple — le carrefour, le tapis, le « tant que »

- Le **if/elif/else** est une **échelle de carrefour** : chaque échelon pose une question, la première vraie prend la valeur, et si personne ne dit rien, le dernier échelon (**else**) attrape le reste. Comme le **CASE WHEN** de M06 : une condition par ligne, un **ELSE** au fond.
- La **boucle for** est un **tapis roulant** : on pose les éléments (les 5 magasins) dans l'ordre, et on traite chacun en passant — un par un, sans sauter, sans revenir.
- La **boucle while** est le « **tant que** » : on ne compte pas les tours, on donne une condition (« tant que le crédit dépasse 0 ») et on se répète jusqu'à ce qu'elle devienne fausse.
- **break** est la **porte de sortie** : on sort de la boucle immédiatement, au milieu.
- **continue** est le « **passe** » : on saute le reste du tour **et on revient au début** — l'élément pourri est ignoré, le tapis continue.
- La **compréhension** est le tapis roulant **avec filtre, en une ligne** : on nomme ce qui sort du tapis au lieu de le traiter un par un.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
échelle de décision	une suite if / elif / else où la première condition vraie gagne, sinon else .	croire que les échelons se testent tous : la première vraie court-circuite le reste.
itération — iteration	un tour de boucle : le corps s'exécute une fois sur un élément.	confondre « la boucle a tourné » (le tour) et « la liste est finie » (la fin du parcours).
boucle infinie — infinite loop	une boucle dont la condition ne devient jamais fausse (ou dont on ne coupe jamais).	la détester : mesurée en secondes, elle est la preuve que le break manquait.
sortie / contournement — break / continue	break : sortir de la boucle tout de suite ; continue : sauter le reste du tour, reprendre au début.	mélanger les deux : break termine le parcours, continue réduit un tour.
compréhension — comprehension	— <code>[expression for x in iterable if condition]</code> : une liste construite en une ligne lisible.	empiler 2 niveaux de for : au-delà, c'est plus une formule qu'une phrase — on repasse à une boucle.
boucle avec else — for...else	— le else d'une boucle s'exécute si la boucle se termine sans break .	croire que le else fait « le contraire » : il fait « le cas où on n'a pas cassé ».

5. Cours approfondi

5.1 if/elif/else — l'échelle du comptoir

L'échelle à trois modes de paiement du socle (le seuil comptoir est 1 000 FCFA, `m08p_demo_seuil` ; au-delà de 15 000 FCFA, le client part en financement — les trois montants de test sont ceux de C02), exécutée dans l'atelier :

```
totaux = (500, 1200, 15000.5)
for total in totaux:
    if total < 1000:
        mode = "comptant"
    elif total <= 15000:
        mode = "dossier"
    else:
        mode = "financement"
    print(total, "->", mode)
```

```
500 -> comptant
1200 -> dossier
15000.5 -> financement
```

Définition. échelle de décision — if/elif/else — une suite d'échelons testés **dans l'ordre** : la première condition vraie exécutera son bloc, et le reste est sauté ; else attrape ce que personne n'a pris. C'est le CASE WHEN ... THEN ... ELSE ... END de M06, écrit en Python.

Deux choses que cette exécution montre :

1. **L'ordre des échelons est une décision métier, pas un détail.** < 1 000 avant <= 15 000 : si on avait inversé, tout le monde serait en « dossier ». Le CASE WHEN de M06 obéissait à la même règle — c'est la **même** échelle, deux langages.
2. **L'indentation EST la syntaxe.** Le bloc du if est ce qui est enfoncé de 4 espaces : pas de accolades, pas de then — **l'alignement dit ce qui est « dans » la condition**. Une ligne mal alignée, Python s'effondre (le §8 a les deux messages réels).

Conseil professionnel. Écrivez la condition **en noms**, pas en chiffres : if total < seuil_comptant: (le seuil défini une fois, en haut : seuil_comptant = 1000) — si le seuil change, une ligne change. Et quand un break sort d'un parcours, donnez-lui une **raison lisible** dans l'affichage (« arrêt : 3^e dépassement »), pas un break tout nu : le rapport doit pouvoir se **relire** sans le code.

5.2 La boucle for — le tapis roulant des 5 magasins

Les 5 magasins du socle (magasin.csv), parcourus avec leur numéro — enumerate donne l'index **et** la valeur :

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre", "Bobo Sarfalao", "Koudougou"]
for i, nom in enumerate(magasins):
    print(i + 1, nom)
```

```
1 Ouaga Centre
2 Ouaga Patte d'Oie
3 Bobo Centre
4 Bobo Sarfalao
5 Koudougou
```

Définition. itération — iteration — un tour de boucle : le corps de la for s'exécute une fois par élément, dans l'ordre, sans sauter ni revenir. Après le dernier élément, la boucle est finie — c'est tout, pas d'arrêt à écrire.

range(n) est le tapis des nombres : range(10) déroule **0 à 9** — la sortie de l'atelier le prouve :

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Attention. `range(10)` donne **0 à 9**, pas 1 à 10. La liste a 10 éléments, mais le dernier a l'index 9 — c'est le décalage qui fabrique la moitié des bugs de comptage (et l'`IndexError` de C04). Pour « 1 à 10 », écrire `range(1, 11)`.

5.3 La boucle `while` — le « tant que » du crédit

Un crédit de 2 500 FCFA, remboursé à raison de 500 par mois — on ne compte pas les tours, on donne la condition :

```
credit = 2500
mensualite = 500
mois = 0
while credit > 0:
    mois = mois + 1
    credit = credit - mensualite
    print(f"mois {mois} : reste {credit}")
print("solde en", mois, "mois")
```

```
mois 1 : reste 2000
mois 2 : reste 1500
mois 3 : reste 1000
mois 4 : reste 500
mois 5 : reste 0
solde en 5 mois
```

Définition. boucle infinie — infinite loop — une boucle dont la condition ne devient jamais fausse (ou qu'aucun `break` ne coupe) : la machine se répète sans fin. Ce n'est pas une fatalité, c'est un **diagnostic** : la condition ou le `break` manque — le §5.3 et le §8 la mesurent en secondes réelles.

La boucle infinie, **exécutée** dans l'atelier (lancée sous `timeout 2` : tuée après 2 secondes) :

```
$ timeout 2 python3 boucle_infinie.py
code de sortie : 124
```

Aucune sortie, rien n'a été affiché, le code 124 est celui du processus **tué** : la machine comptait encore. Et le même corps de boucle, avec le `break` qui sauve :

```
i = 0
while True:
    i = i + 1
    if i == 5:
        break
print(i)
```

```
5
```

La leçon tient en une phrase : `while True` n'est pas interdit — il oblige simplement à porter la responsabilité de la sortie **dans le corps** (`break`). En SQL, le SGBD a la garantie de fin (les lignes finissent) ; en Python, la fin est **votre** contrat.

5.4 `break` et `continue` — la porte de sortie et le « passe »

Usage métier n° 1 — `break` : s'arrêter au premier dépassement. Le contrôle ne va pas au bout s'il trouve ce qu'il cherche : le premier montant au-dessus du seuil suffit.

```
montants = [120, 800, 2000, 15000.5, 500]
seuil = 15000
for m in montants:
    if m > seuil:
        print("dépassement :", m)
        break
print("fin du parcours")
```

```
dépassement : 15000.5
fin du parcours
```

Le `break` a coupé le parcours : 500 n'a jamais été vu. Le `print` après la boucle s'exécute **toujours** — il est **hors** du bloc.

Usage métier n° 2 — `continue` : ignorer les lignes pourries. Une saisie « N/C » (non communicable) ne bloque pas le total : on passe, on revient au début, on continue.

```
saisies = [500, "N/C", 1500, "N/C", 1350]
total = 0
pourries = 0
for s in saisies:
    if s == "N/C":
        pourries = pourries + 1
        continue
    total = total + s
print("total :", total, "| pourries :", pourries)
```

```
total : 3350 | pourries : 2
```

Les deux montants « N/C » sont **comptés** (pourries : 2) mais pas additionnés — c'est exactement le `WHERE montant IS NOT NULL` de M06, écrit en Python.

Le `for...else` — le « non trouvé » sans drapeau. Le `else` attaché à une boucle s'exécute **si la boucle se termine sans `break`** :

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre", "Bobo Sarfalao", "Koudougou"]
for nom in magasins:
    if nom == "Sintani":
        print("trouve :", nom)
        break
else:
    print("magasin introuvable")
```

```
magasin introuvable
```

Sintani n'étant pas dans les 5 magasins du socle, aucun `break` — le `else` a parlé. La recherche de « Koudougou » aurait, elle, cassé la boucle et tué le `else`. C'est le rapport « client non trouvé » sans variable drapeau en mémoire.

5.5 Les compréhensions — le tapis avec filtre, en une ligne

La forme : `[expression for x in iterable if condition]` — on lit la phrase de gauche à droite : « pour chaque montant, **si** il dépasse 1 000, le garder » :

```
montants = [500, 1500, 8000, -50, 1350, 200, 4500, 90]
gros = [m for m in montants if m > 1000]
print(gros)
print(len(gros))
```

```
[1500, 8000, 1350, 4500]
4
```

Définition. compréhension — list comprehension — `[expr for x in it if cond]` : une liste construite en **une** ligne, lisible de gauche à droite comme une phrase. C'est le raccourci du « `for` + filtre + `append` » — même résultat, moins de lignes, et l'intention lisible d'un coup d'œil.

Attention. La **règle des 2 niveaux** : une compréhension avec **un** `for` et **une** `if`, c'est une phrase. Deux `for` imbriqués (`[(m, t) for m in magasins for t in trimestres]`), ce n'est plus une phrase, c'est un tableau croisé déguisé — on revient à une boucle classique, c'est **plus lisible** et **plus facile** à déboguer. Le croisé magasins × trimestres (l'exemple d'atelier : 3 magasins × 4 trimestres = 12 paires) est l'endroit où la règle se voit.

5.6 `zip` — deux listes, un seul parcours

Deux colonnes du même tableau, lues **en lockstep** — les 5 magasins et leurs surfaces réelles (magasin.csv) :

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre", "Bobo Sarfalao", "Koudougou"]
surfaces = [450, 380, 520, 290, 210]
for nom, s in zip(magasins, surfaces):
    print(nom, "-", s)
```

```
Ouaga Centre - 450
Ouaga Patte d'Oie - 380
Bobo Centre - 520
Bobo Sarfalao - 290
Koudougou - 210
```

En SQL, c'est juste `SELECT nom, surface_m2 FROM magasin` — les deux colonnes arrivent déjà appariées dans la ligne. En Python, `zip` fait le même appariement sur deux listes distinctes. (`zip` s'arrête à la **plus courte** des deux : si les listes ne sont pas de même longueur, l'écart est **silencieux** — le §9 en fait une règle de vérification.)

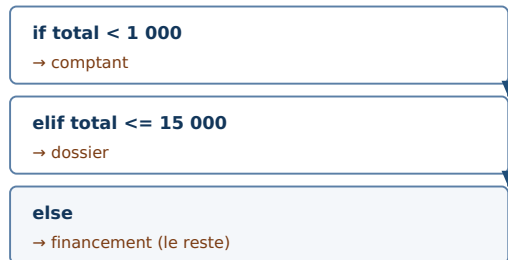
Dans les faits. Le socle M08 contient bien une table `objectif_magasin` — **vide** (0 ligne, clé `m08p_objectif_lignes`). Le parcours « magasins + objectifs » que ce chapitre illustre tourne donc sur des surfaces (données réelles) ; l'import de la table vide et la question « que fait un `for` sur 0 ligne ? » attendent C07 — réponse : rien, et c'est normal.

Le flux complet du chapitre — l'échelle à gauche, la boucle et ses issues à droite :

Décider et répéter : le flux de plusieurs valeurs

C02 a donné la grammaire d'une valeur ; C03 donne le flux : carrefour, tapis, porte de sortie.

Décider — l'échelle if/elif/else



la 1re condition vraie gagne — le reste est sauté
(le CASE WHEN de M06, ligne par ligne)

l'indentation EST la syntaxe : 4 espaces par niveau

Répéter — for / while



for...else : l'else parle si personne n'a cassé

(le « magasin introuvable » sans variable drapeau)

Résumer : [m for m in montants if m > 1000] — le tapis avec filtre, en une ligne

zip(magasins, surfaces) : deux listes en lockstep — le SELECT de 2 colonnes, en Python

Règles : for si on compte · while si on ne compte pas · compréhension = 1 niveau, 1 condition

6. Exemple concret — le contrôle qualité express

La question du chapitre, en langage de comptoir : « 10 montants viennent de la caisse : lesquels sont en ordre, lesquels sont à revoir, et s'arrêter si les anomalies dérapent. » Le script — exécuté dans l'atelier le 19/09/2026 (le seuil de revue est 15 000 FCFA) :

```

montants = [500, 1500, 8000, "N/C", -50, 1350, 15000.5, 200, 4500, 90]
seuil = 15000
anomalies = 0
traites = 0
for m in montants:
    traites = traites + 1
    if isinstance(m, str) or m < 0 or m > seuil:
        anomalies = anomalies + 1
        print(anomalies, "anomalie :", m)
        if anomalies == 3:
            break
print("traites :", traites, "sur", len(montants))
print("anomalies :", anomalies)
  
```

```

1 anomalie : N/C
2 anomalie : -50
3 anomalie : 15000.5
traites : 7 sur 10
anomalies : 3
  
```

Lecture du rapport : le 4^e montant ("N/C", du texte dans une liste de nombres — le piège de C02 qui redevient utile) est l'anomalie 1 ; le négatif, l'anomalie 2 ; le montant de 15 000,50 FCFA (clé m08p_demo_montant_fcfa) dépasse le seuil de 15 000 → anomalie 3, **le break coupe** : 7 montants traités sur 10, les 3 derniers (200, 4500, 90) n'ont pas été vus — c'est **l'effet cherché** (le contrôle ne traîne pas) et **la limite** (le rapport doit le dire, et il le dit : « 7 sur 10 »).

Le parallèle SQL : le même contrôle sur une table serait `SELECT COUNT(*) FROM vente WHERE montant < 0 OR montant > 15000` — une **agrégation sur tout** (la table ne « s'arrête pas ») ; en Py-

thon, on peut **choisir** de s'arrêter au milieu, ce que SQL ne fait pas. C'est toute la différence de posture : le SGBD traite le **lot**, Python traite le **flux**.

7. Démonstration pas à pas — 6 étapes sur le fil rouge

Du zéro au contrôle qualité, sortie réelle de chaque commande (exécutées le 19/09/2026).

Étape 1 — Décider seul (l'échelle sur un total nu) :

```
$ python3 -c "total = 8000; print('comptant' if total < 1000 else 'dossier' if total <= 15000 else 'financement')"
```

Interprétation : 8 000 passe entre 1 000 et 15 000 — dossier. L'expression conditionnelle de C02 (la `CASE WHEN` en une ligne) est l'échelle repliée ; le bloc `if/elif/else` est la même échelle dépliée.

Étape 2 — Parcourir les 5 magasins (le `for` + `enumerate`) :

```
$ python3 -c "
magasins = ['Ouaga Centre', 'Ouaga Patte d'Oie', 'Bobo Centre', 'Bobo Sarfalao', 'Koudougou']
for i, nom in enumerate(magasins):
    print(i + 1, nom)"
1 Ouaga Centre
2 Ouaga Patte d'Oie
3 Bobo Centre
4 Bobo Sarfalao
5 Koudougou
```

Interprétation : 5 tours de tapis, un par magasin — le `for` compte tout seul, pas de compteur à maintenir.

Étape 3 — Le « tant que » du crédit (le `while` de §5.3) :

```
$ python3 credit.py
mois 1 : reste 2000
mois 2 : reste 1500
mois 3 : reste 1000
mois 4 : reste 500
mois 5 : reste 0
solde en 5 mois
```

Interprétation : la condition (`credit > 0`) décide de la fin — 5 tours, pas un de plus ; le dernier affichage montre la condition qui devient fausse (reste 0).

Étape 4 — S'arrêter au premier dépassement (le `break`) :

```
$ python3 -c "
montants = [120, 800, 2000, 15000.5, 500]
for m in montants:
    if m > 15000:
        print('dépassement : ', m)
        break"
dépassement : 15000.5
```

Interprétation : le parcours s'est arrêté au milieu — le 500 n'a pas été vu ; c'est le contrôle qui décide de la fin, pas la fin de la liste.

Étape 5 — Résumer en une ligne (la compréhension avec `isinstance`) :

```
$ python3 -c "
montants = [500, 1500, 8000, 'N/C', -50, 1350, 200, 4500, 90]
gros = [m for m in montants if isinstance(m, (int, float)) and m > 1000]
print(gros)"
[1500, 8000, 1350, 4500]
```

Interprétation : le `isinstance` garde les nombres et écarte les textes — le « N/C » n'arrive jamais au comparateur, donc pas de `TypeError` (l'erreur de C02) : le filtre de la compréhension est la porte de C02, déplacée dans une ligne.

Étape 6 — Deux listes en parallèle (le `zip`) :

```
$ python3 -c "
magasins = ['Ouaga Centre', '\Ouaga Patte d'Oie\',"Bobo Centre", 'Bobo Sarfalao', 'Koudougou']
surfaces = [450, 380, 520, 290, 210]
for nom, s in zip(magasins, surfaces):
    print(nom, '-', s)"
Ouaga Centre - 450
Ouaga Patte d'Oie - 380
Bobo Centre - 520
Bobo Sarfalao - 290
Koudougou - 210
```

Interprétation : 5 paires, lues en lockstep — le `SELECT nom, surface_m2` de M06, écrit en Python.

8. Erreurs fréquentes

- 1. `IndentationError` — l'alignement qui casse le script.** Deux formes exécutées dans l'atelier. Le bloc **manquant** après un `if` :
text File "<string>", line 7 print('comptant') ^^^^^ IndentationError: expected an indented block after 'if' statement on line 6 Et la ligne **mal alignée** (4 espaces, puis 2) : text File "<string>", line 4 print('mal aligne') ^ IndentationError: unindent does not match any outer indentation level Remède : **4 espaces par niveau**, jamais de mélange espaces/tabulations ; l'indentation est la syntaxe, pas le décor.
- 2. `range(10) = 0 à 9`.** La somme « de 1 à 10 » écrite `for n in range(10)` donne **45**, pas 55 — exécuté dans les deux sens dans l'atelier : text \$ python3 -c "total = 0 for n in range(10): total = total + n print(total)" 45 \$ python3 -c "total = 0 for n in range(1, 11): total = total + n print(total)" 55 Remède : « de a à b **compris** » = `range(a, b + 1)` ; le dernier index est `n - 1`.
- 3. La boucle infinie — mesurée, pas subie.** `while True` sans `break` : l'atelier la tue à 2 secondes (code de sortie **124**), aucune sortie ; `while credit > 0` avec un oubli de décrément (`credit = credit + mensualite` au lieu de `-`) : pareil, **124**. Remède : avant chaque `while`, répondre à voix haute « **qu'est-ce qui rend la condition fausse ?** » — si la réponse n'est pas dans le corps, il manque un `break` ou un décrément.
- 4. Remplir la liste qu'on parcourt — la boucle qui s'alimente elle-même.** text \$ timeout 2 python3 -c " l = [1] for x in l: l.append(x * 2) print(len(l))" code de sortie : 137 (processus tué après 2 s) Chaque tour **ajoute** un élément, le tapis n'en finit pas — le code 137 (tué) est la même leçon que le 124. Remède : on construit **une autre** liste pendant qu'on parcourt la première — et c'est exactement ce que fait la compréhension du §5.5.
- 5. Le deux-points manquant — le bloc qui n'existe pas.** `if total < 1000` sans `:` : `SyntaxError: expected ':'` (C02 §5.6) — en C03, c'est l'erreur **au bord** d'un bloc entier : tout ce qui était « dedans » n'existe plus. Remède : taper le `:` **immédiatement** après la condition, avant la ligne d'en dessous.
- 6. La compréhension qui s'imbrique.** `[x for m in magasins for t in trimestres ...]` : au deuxième `for`, la ligne n'est plus lisible d'un coup d'œil — ce n'est pas une erreur d'exécu-

tion, c'est une **erreur de lecture** qui se paiera au premier débogage. Remède : la règle des 2 niveaux (§5.5) — une boucle, un commentaire.

9. Bonnes pratiques professionnelles

1. **Un test par échelon, une échelle par décision** : si le `if` porte deux questions différentes (« est-ce un nombre ? » puis « est-ce en dessous du seuil ? »), ce sont deux `if` imbriqués ou deux variables de statut — pas un `or` monstre dans la condition.
2. **for quand on peut compter, while quand on ne peut pas** : 5 magasins → `for` ; « tant que le crédit dépasse 0 » → `while`. Un `while` qui compte en parallèle est un `for` déguisé (et plus fragile).
3. **Compréhension : un niveau, une condition, pas trop longue** — au-delà, boucle. Et quand la compréhension **filtre**, nommer ce qu'on garde (`gros_montants`, pas `liste2`).
4. **Vérifier la longueur avant le zip** : `assert len(magasins) == len(surfaces)` en tête — le `zip` qui s'arrête à la plus courte est **silencieux** (aucune erreur, des paires manquantes : le pire des défauts).
5. **Le break porte une raison, le continue porte un compte** : tout `continue` sur ligne pourrie incrémente un compteur (`pourries`) — le rapport final doit pouvoir dire « 2 lignes ignorées », sinon l'ignorance est invisible.

10. Exercice guidé — « le registre à trois modes » (15 min, /10)

Énoncé. Écrire `registre.py` qui :

1. parcourt les 3 ventes `[500, 1200, 15000.5]` avec leur numéro (`enumerate`) ;
2. classe chaque total sur l'échelle de §5.1 (`comptant / dossier / financement`) ;
3. compte les ventes par mode ;
4. affiche chaque vente (`vente n : total -> mode`), puis une ligne de récapitulatif.

Barème. L'échelle avec seuils **en variables nommées** (4 pts) · le parcours avec `enumerate` (2 pts) · les 3 compteurs mis à jour **dans** la bonne branche (2 pts) · la sortie propre, alignée avec le modèle ci-dessous (2 pts). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```
vente 1 : 500 -> comptant
vente 2 : 1200 -> dossier
vente 3 : 15000.5 -> financement
recap : comptant 1 | dossier 1 | financement 1
```

11. Exercices autonomes

Exercice 3.1 — La somme. Sans `sum()`, calculer la somme des nombres de 1 à 10 avec une `for` + `range`. (5 min)

Exercice 3.2 — Le crédit. Un crédit de 3 500 FCFA, remboursé à 500 par mois : combien de mois pour solde ? (`while` + compteur.) (5 min)

Exercice 3.3 — Le premier. Dans `[120, 800, 2000, 5200, 900]`, afficher l'**index** et la valeur du **premier** montant au-dessus de 5 000 FCFA, puis s'arrêter. (`for` + `enumerate` + `break`.) (5 min)

Exercice 3.4 — Les pourries. Dans `[500, "N/C", 1500, "N/C", 1350, "N/C"]`, additionner les nombres seulement (`continue`), et afficher le total **et** le nombre de nombres valides. (5 min)

Exercice 3.5 — Le `for...else`. Pour les requêtes « Koudougou » et « Sintani », afficher « trouve, position n » ou « introuvable » — avec un seul `for...else`, pas de variable drapeau. (10 min)

12. Correction détaillée

Exercice guidé. Le script qui fait l'affaire :

```
ventes = [500, 1200, 15000.5]
seuil_comptant = 1000
seuil_financement = 15000
comptant = dossier = financement = 0
for i, total in enumerate(ventes):
    if total < seuil_comptant:
        mode = "comptant"
        comptant = comptant + 1
    elif total <= seuil_financement:
        mode = "dossier"
        dossier = dossier + 1
    else:
        mode = "financement"
        financement = financement + 1
    print(f"vente {i + 1} : {total} -> {mode}")
print("recap : comptant", comptant, "| dossier", dossier, "| financement", financement)
```

Deux points de correction : les seuils sont **nommés** en tête (une ligne change si un seuil change) ; les compteurs sont mis à jour **dans la même branche** qui pose le mode — si le comptage était fait après l'échelle (avec un second `if` sur `mode`), deux échelles racontent la même histoire : l'une ment à la seconde.

Exercice 3.1. `for n in range(1, 11): s = s + n` → **55**. Le piège est le `range(10)` (§8, erreur n° 2) : il donne 45, et c'est exactement l'écart du 10 manquant.

Exercice 3.2. `while credit > 0` avec `credit = credit - 500` → **7 mois** ($3\,500 \div 500 = 7$ pile — le dernier tour met le solde à 0, la condition devient fausse, la boucle s'arrête seule).

Exercice 3.3. `for i, m in enumerate(montants): if m > 5000: print(i, m); break` → **3 5200** : l'index de 5 200 est **3** (zéro-indexé, §5.2) — afficher `i + 1` donnerait la « position 4 » : les deux sont justes, il faut **choisir** et le dire.

Exercice 3.4. `continue` sur `"N/C"`, sinon `total = total + s` et compteur — → **3350 3** : le total des 3 nombres ($500 + 1\,500 + 1\,350$), les 3 « N/C » ignorés.

Exercice 3.5. Le `for...else` imbriqué dans la boucle des requêtes :

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre", "Bobo Sarfalao", "Koudougou"]
for requete in ("Koudougou", "Sintani"):
    for nom in magasins:
        if nom == requete:
            print(requete, "-> trouve, position", magasins.index(nom) + 1)
            break
    else:
        print(requete, "-> introuvable")
```

```
Koudougou -> trouve, position 5
Sintani -> introuvable
```

L'`else` est attaché au `for` **intérieur** (même indentation que le `for`, pas au `if`) — le `break` de l'intérieur le tue, l'absence de `break` le libère.

13. Mini-projet M08.P3 — « Le rapport de qualité complet » (1 h)

Énoncé. Écrire `qualite.py` — le contrôle qualité de §6, durci :

- la liste des 10 montants (celle de §6) est en tête, le seuil de revue (15 000 FCFA) est une **variable nommée** ;
- quatre compteurs : `illisible` (du texte), `negatif`, `a_revoir` (au-dessus du seuil), `conforme` — et un compteur `traites` ;
- le parcours affiche chaque anomalie numérotée, et **s'arrête au 3^e dépassement** (`break`) ;
- le rapport final : « montants traités : n / 10 », puis les 4 compteurs — alignés, lisibles ;
- si **aucune** anomalie n'est trouvée, le rapport dit « contrôle terminé sans dépassement » (indice : `for...else`).

Critères de réussite (grille /10) : les 5 exigences (5 × 1 pt) · la sortie de l'atelier donne « arrêt au 3e depassement : 15000.5 », « montants traites : 7 / 10 », illisible 1 · négatif 1 · a revoir 1 · conforme 4 (1 pt) · aucun or monstre : un test par échelon (1 pt) · le compteur `pourries/traites` est mis à jour **avant** le test (1 pt) · docstring en tête, seuil nommé (1 pt).

Note de correction. La sortie attendue est celle de l'atelier (D9 du 19/09/2026) : 7 montants traités sur 10 (le `break` coupe à la 3^e anomalie), 4 conformes **parmis les traités** — les 3 montants non vus (200, 4500, 90) n'apparaissent dans **aucun** compteur : c'est l'effet du `break`, et le rapport est honnête parce qu'il dit « 7 / 10 ».

14. Boîte à outils du chapitre

Boîte à outils. Toujours Python nu + terminal (couche 2 de C01) ; la boîte du chapitre s'ajoute aux encadrés de C02 : l'échelle, le tapis, la porte de sortie, la phrase-liste.

Outil	Usage	Syntaxe
<code>if / elif / else</code>	l'échelle de décision	<code>if a < b: ... elif a < c: ... else: ...</code>
<code>for + range</code>	le tapis des nombres	<code>for n in range(1, 11):</code>
<code>enumerate</code>	index et valeur	<code>for i, nom in enumerate(magasins):</code>
<code>while</code>	le « tant que »	<code>while credit > 0:</code>
<code>break / continue</code>	la sortie / le « passe »	<code>break · continue</code>
<code>for...else</code>	le « non trouvé »	<code>for ... : ... else: ...</code>
compréhension	la liste en une ligne	<code>[m for m in montants if m > 1000]</code>
<code>zip</code>	deux listes en parallèle	<code>for nom, s in zip(magasins, surfaces):</code>

15. Résumé du chapitre

- L'échelle `if/elif/else` est le `CASE WHEN` de M06 en Python : **la première vraie gagne**, l'indentation **est** la syntaxe.
- Le `for` est le tapis roulant (il compte tout seul) ; `range(10)` donne **0 à 9** ; `enumerate` porte l'index et la valeur.
- Le `while` est le « tant que » — la fin est **votre** contrat : la boucle infinie mesurée (tuée à 2 s, code 124) est la preuve qu'un `break` ou un décrétement manquait.

- `break` **termine** le parcours (le rapport dit « 7 sur 10 ») ; `continue` **réduit** un tour (le compteur de pourries dit « 2 ignorées ») ; le `for...else` parle quand **personne** n'a cassé.
- La compréhension est le tapis avec filtre **en une ligne** — un niveau, une condition ; au-delà, une boucle.
- `zip` lit deux colonnes en lockstep — le `SELECT nom, surface` de M06, en Python ; il s'arrête à la plus courte **en silence** : vérifier la longueur.

16. À retenir

À retenir. Avant chaque `while`, la question à voix haute : « **qu'est-ce qui rend la condition fausse ?** » — si la réponse n'est pas dans le corps de la boucle, la boucle est infinie. C'est la seule inspection qui empêche la machine de tourner toute la nuit.

À retenir. `break` **termine**, `continue` **passe** — et les deux laissent des **traces** : le compteur `traites` après un `break`, le compteur `pourries` après un `continue`. Un rapport qui ne dit pas ce qu'il a vu et ce qu'il a ignoré n'est pas un rapport.

17. Évaluation formative (auto-correction, 8 min)

1. Sur l'échelle « `comptant < 1 000 ≤ dossier ≤ 15 000 < financement` », quel mode pour 15 000,50 — et pourquoi `elif` et pas deux `if` ? → `financement` (15 000,50 > 15 000) ; deux `if` séparés **testeraient les deux échelons** (le total pourrait recevoir deux modes) — `elif` garantit une seule branche. (1 pt)
2. `range(10)` donne quoi, et comment écrire « 1 à 10 compris » ? → 0 à 9 ; `range(1, 11)`. (1 pt)
3. `while credit > 0` avec `credit = credit - 500` mais `credit = 2500` : combien de tours, et qu'est-ce qui rend la condition fausse ? → 5 tours ; le décrétement du corps — la dernière itération met `credit` à 0. (1 pt)
4. `break` et `continue` — la différence en une phrase, et le métier de chacun ? → `break` sort de la boucle (arrêt au premier dépassement) ; `continue` saute le reste du tour et reprend au début (ignorer les lignes « N/C »). (2 pts)
5. Quand le `else` d'un `for...else` s'exécute-t-il ? → quand la boucle se termine **sans break** — le cas « non trouvé ». (1 pt)
6. La compréhension `[m for m in montants if m > 1000]` fait quoi, et quelle est la règle des 2 niveaux ? → une liste des montants > 1 000, en une ligne ; au-delà de `un for + une if`, on repasse à une boucle classique. (2 pts)
7. `zip(magasins, surfaces)` s'arrête où si les listes ne font pas la même longueur ? → à la plus courte, **en silence** (aucune erreur) — d'où la vérification de longueur. (1 pt)
8. Le contrôle de §6 affiche « `traites : 7 sur 10` » : pourquoi 7, et que font les 3 montants non vus ? → le `break` a coupé à la 3^e anomalie (le 7^e montant) ; les 3 suivants n'ont été traités nulle part — d'où le « / 10 » qui rend le rapport honnête. (1 pt)

Module M08.C04 — Ranger : listes, tuples, chaînes, dictionnaires, ensembles, index, tris

Outils : Python 3.13.14 seul (l'interpréteur), le terminal. Aucun paquet. Durée indicative : 4 h. Niveau : N4 → N5. Prérequis : C03 (le flux : décider, répéter).

L'idée du chapitre. C03 a donné le **flux** (décider, répéter, s'arrêter) ; C04 donne le **rangement** : les 5 conteneurs de Python et la question métier que chacun répond. La **liste** (une case numérotée, changeable, doublons admis), le **tuple** (l'enveloppe scellée, immuable), la **chaîne** (la séquence de caractères, et les méthodes du format français), le **dictionnaire** (l'étagère étiquetée — le `GROUP BY` avant pandas), l'**ensemble** (le tamis — le `COUNT(DISTINCT)` avant pandas). Le fil rouge : les 5 magasins du socle entrent dans 4 contenants, et 5 questions métier trouvent chacune leur contenant. En C07, le `DataFrame` prend le relais : une colonne est une liste indexée, une table est un dictionnaire de colonnes — tout ce chapitre est le préambule de C07.

Matériel de l'atelier — Python 3.13.14, terminal, zéro paquet. Tout le code de ce chapitre a été exécuté dans l'atelier le 19/09/2026 ; les sorties publiées sont **celles de l'atelier**. Les données citées sont celles du socle M08 : les 5 magasins (`magasin.csv`, avec leurs villes et surfaces), 3 des libellés de paiement de `mode_paiement.csv`, le panier moyen 158 140 FCFA (clé `m08p_paiement_moyen`).

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Construire une **liste** (`append`, `extend`), la lire à l'index (positif et négatif), la trancher (`[1:3]`, `[::2]`, `[::-1]`) — et lire un `IndexError` sans surprise.
2. Dire quand choisir un **tuple** plutôt qu'une liste (l'immuable : clé de dictionnaire, résultat figé), et faire l'**unpacking** (`n, p = ...`).
3. Nettoyer et formater des **chaînes** avec `strip`, `split`, `replace`, `startswith` — la recette du format français en méthode.
4. Utiliser un **dictionnaire** : `d[clé]`, `d.get(clé, défaut)`, `clé in d` — et écrire le **compteur** (le `GROUP BY` avant pandas).
5. Compter les **valeurs uniques** avec `len(set(...))` (le `COUNT(DISTINCT)` avant pandas) et croiser deux ensembles (l'intersection).
6. Trier : `sorted(...)` (ne modifie pas), `.sort(reverse=True)` (modifie), et le **top N** d'un dictionnaire en une ligne.

2. Pourquoi cette notion est importante

Parce qu'en Python, **choisir le contenant, c'est choisir la réponse**. Une même donnée posée dans le mauvais contenant rend la bonne question difficile et la mauvaise facile : les villes dans une **liste** et « combien de villes distinctes ? » demande un `len(set(...))` ; les villes dans un **ensemble**, la réponse est immédiate. Les 5 contenants ne sont pas 5 boîtes interchangeables, ce sont 5 **postures** : l'ordre (liste), le figé (tuple), le texte (chaîne), l'étiquette (dictionnaire), l'unicité (ensemble).

Et parce que le parallèle SQL est déjà **enfermé** dans ces 5 contenants : le compteur de dictionnaire de §5.4 **est** le `SELECT id_magasin, COUNT(*) FROM vente GROUP BY id_magasin` de M06/M07, écrit en Python ; `len(set(villes))` **est** le `COUNT(DISTINCT ville)` ; le `sorted(d.items(), key=...)` de §5.6 **est** le `ORDER BY ... LIMIT n`. C07 ne surprendra pas : pandas est le même rangement, à l’échelle de la table.

Enfin, les index sont la base de C07 : `l[0]`, `l[-1]`, `l[1:3]` sont les frères aînés de `df.loc[3]` et `df.iloc[1:4]` — la même idée (la ligne **n** porte l’index **n – 1**), la même famille d’erreurs (`IndexError` aujourd’hui, `KeyError/IndexError` en C07).

3. Explication simple — la case, l’enveloppe, l’étagère, le tamis

- La **liste** est une **case numérotée** : les éléments ont un ordre et un numéro, on en ajoute, on en change, on en duplique — comme une file d’attente.
- Le **tuple** est l’**enveloppe scellée** : une fois fermé, on ne change plus rien dedans — c’est ce qui le rend **fiable** (il peut servir de clé, la clé ne doit pas changer sous le nez du placard).
- La **chaîne** est une **ligne de caractères** : elle aussi a des index (chaque lettre a son numéro) et des méthodes (`strip`, `split`, `replace`) — c’est là que vit le format français.
- Le **dictionnaire** est l’**étagère étiquetée** : chaque étiquette (clé) porte un objet (valeur) ; on cherche par étiquette, pas par numéro — comme un annuaire.
- L’**ensemble** est le **tamis** : les doublons tombent, il ne reste que les uniques — « combien de villes distinctes ? » ne se pose même plus.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
séquence sequence	— un contenant ordonné, lisible par index (liste, tuple, chaîne).	indexer par étiquette une séquence : l’étiquette, c’est le dictionnaire.
index — index	le numéro d’une position dans une séquence — la ligne n porte l’index n – 1 .	écrire <code>l[n]</code> pour la ligne n : c’est <code>l[n – 1]</code> — le décalage qui fabrique l’ <code>IndexError</code> .
tranche — slice	<code>l[a:b]</code> : les éléments de l’index a à l’index b – 1 (la fin est exclue).	croire que <code>l[1:3]</code> contient 3 éléments : elle en contient 2.
clé — key	l’étiquette d’une case de dictionnaire ; clé <code>in d</code> teste les clés, pas les valeurs.	chercher une valeur avec <code>in d</code> : <code>in</code> regarde les étiquettes, jamais les objets.
valeur unique unique value	— une valeur qui ne compte qu’une fois : le rôle de l’ensemble (<code>set</code>).	compter les uniques dans une liste (avec des doublons) : le <code>len</code> de la liste ment.
immuable immutable	— un objet qu’on ne peut pas modifier en place (tuple, chaîne, nombre).	vouloir <code>t[0] = ...</code> sur un tuple : <code>TypeError</code> — on refait un tuple, on ne répare pas.
table de données — dataframe	une table à colonnes, chaque colonne indexée : l’objet central de C07, annoncé ici.	la confondre avec une liste : une table a des colonnes nommées , une liste des index.

5. Cours approfondi

5.1 La liste — la case numérotée

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre"]
magasins.append("Bobo Sarfalao")
magasins.extend(["Koudougou"])
print(len(magasins))
print(magasins[-1])
print(magasins[1:3])
print(magasins[:2])
print(magasins[::-1][:2])
```

```
5
Koudougou
["Ouaga Patte d'Oie", 'Bobo Centre']
['Ouaga Centre', 'Bobo Centre', 'Koudougou']
['Koudougou', 'Bobo Sarfalao']
```

Définition. séquence — sequence — un contenant ordonné, lisible par index : liste, tuple, chaîne. La liste est la séquence **mutable** (on ajoute avec `append`, on étend avec `extend`, on change la case en place) — le conteneur par défaut.

Deux remarques sur la sortie : `magasins[-1]` est **Koudougou** (l'index négatif compte à rebours : `-1` = le dernier), et la tranche `magasins[1:3]` contient **2** éléments ("Ouaga Patte d'Oie" et "Bobo Centre") — la borne de **fin** est exclue (en SQL, `BETWEEN`, lui, **inclut** ses bornes : deux conventions opposées, à retenir), et le §8 en fait une erreur.

5.2 Le tuple — l'enveloppe scellée

Le tuple est la liste **qu'on ne rouvre plus**. Essayez de modifier une case — la sortie réelle de l'atelier :

```
surface = (450, 380, 520, 290, 210)
surface[0] = 999
```

```
TypeError: 'tuple' object does not support item assignment
```

Le tuple **se lit** comme une liste, **ne s'écrit** pas. Deux usages métier en découlent :

1. **L'unpacking** — ouvrir l'enveloppe d'un coup, en donnant un nom à chaque contenu :

```
surface = (450, 380, 520, 290, 210)
n, p = len(surface), surface[0]
print(n, p)
```

```
5 450
```

1. **La clé de dictionnaire** — une clé ne doit pas changer sous le placard ; le tuple, scellé, a le droit d'y entrer (la liste, mutable, n'y a pas droit) :

```
cle = ("Ouaga Centre", "Ouagadougou")
d = {cle: 450}
print(d[cle])
```

```
450
```

Définition. immuable — immutable — un objet qu'on ne peut pas modifier en place (tuple, chaîne, nombre) : pour « changer » une valeur, on **construit** un nouvel objet (`surface + (210,)`, pas `surface[5] = 210`). C'est la garantie du tuple en clé de dictionnaire.

Quand tuple, quand liste ? Si les éléments sont un **fait** (les surfaces des 5 magasins — ça ne bouge pas) : tuple. S'ils vont **grandir ou changer** (les ventes du jour) : liste. Le choix est une promesse au lecteur, pas une préférence.

5.3 La chaîne — le format français en méthodes

La recette du format français de C02, écrite en **méthodes de chaîne** — exécutée dans l'atelier sur un montant encadré d'espaces (le cas réel des imports) :

```
s = " 15 000,50 "
print(s.strip())
print(s.strip().split(" "))
print(s.strip().replace(" ", "").replace(",", "."))
print(s.startswith(" "))
```

```
15 000,50
['15', '000,50']
15000.50
True
```

- `strip()` : arrache les espaces **aux deux bouts** (le montant tel qu'il sort d'un fichier mal aligné) ;
- `split(" ")` : découpe en morceaux — ici `['15', '000,50']` : les **espaces du nombre** sont des séparateurs pour Python, d'où la `replace` d'après ;
- `replace(" ", "").replace(",", ".")` : la conversion de C02, en chaîne — `float()` de l'autre côté ;
- `startswith(" ")` : la **vérification** (le montant est-il encadré ?) avant le nettoyage.

Et le formatage **dans** le sens inverse — afficher un nombre au format lisible (la valeur est le panier moyen réel du socle, clé `m08p_panier_moyen`) :

```
$ python3 -c "print(f\"panier moyen : {158140:,.0f} FCFA\")"
panier moyen : 158,140 FCFA
```

Définition. méthode de chaîne — string method — un verbe attaché à une chaîne (`s.strip()`, `s.split(" ")`) : elle **renvoie une nouvelle** chaîne sans toucher à l'originale (la chaîne est **immutable**, §5.2) — d'où l'enchaînement `s.strip().replace(...)`.

5.4 Le dictionnaire — l'étagère étiquetée

```
vente_par_magasin = [1, 2, 1, 3, 1, 4, 2, 1]
compteur = {}
for idm in vente_par_magasin:
    compteur[idm] = compteur.get(idm, 0) + 1
print(compteur)
print(compteur.get(5, 0))
print(1 in compteur, 5 in compteur)
```

```
{1: 4, 2: 2, 3: 1, 4: 1}
0
True False
```

Le **compteur** est la construction du chapitre — lisez-la lentement, elle revient dans tous les modules : pour chaque id, `compteur.get(idm, 0)` prend le comptage **ou 0 si la clé est absente**, on ajoute 1, on repose. La sortie **est** le `GROUP BY` :

Python (§5.4)	SQL (M06/M07)	Résultat
<code>compteur[idm] = compteur.get(idm, 0) + 1</code>	<code>COUNT(*) ... GROUP BY id_magasin</code>	magasin 1 : 4 ventes
<code>compteur.get(5, 0)</code>	<code>... WHERE id_magasin = 5 (rien)</code>	0 — pas d'erreur
<code>1 in compteur</code>	<code>SELECT 1 FROM vente WHERE id_magasin = 1 (test d'existence)</code>	True

Trois détails qui portent tout :

1. `d[clé]` **lève** si la clé est absente (le `KeyError` du §8) ; `d.get(clé, défaut)` **renvoie** le défaut — c'est la différence entre un script qui s'effondre et un script qui compte.
2. `clé in d` teste les **clés** — jamais les valeurs (chercher « 4 » dans les valeurs demande un parcours, §8 erreur n° 3).

Attention. `clé in d` teste les **clés**, jamais les valeurs : `4 in compteur` demande « la clé 4 existe-t-elle ? », pas « la valeur 4 existe-t-elle ? » — la recherche dans les valeurs est un parcours, pas une étiquette (§8, erreur n° 4). 3. Un dictionnaire **n'a pas d'ordre métier** : `{1: 4, 2: 2, 3: 1, 4: 1}` se lit par clé, pas par place — c'est exactement ce qu'on veut d'un `GROUP BY`.

Conseil professionnel. Sur les données qu'on ne **possède** pas (fichier, saisie, API), la lecture est `d.get(clé, défaut)`, jamais `d[clé]` : la clé absente est un **cas prévu** du métier, pas une exception — un `KeyError` levé sur un cas prévu est le script qui s'effondre de politesse.

Définition. **clé** — key — l'étiquette d'une case de dictionnaire : `d[clé]` lit, `d[clé] = valeur` écrit, `clé in d` teste. Une clé est **unique** dans son dictionnaire (reposer une clé **écrase** la valeur) — c'est l'unicité qui fait la recherche instantanée.

5.5 L'ensemble — le tamis

Les 5 magasins du socle se répartissent sur 3 villes (`magasin.csv`) :

```
villes = ["Ouagadougou", "Ouagadougou", "Bobo-Dioulasso", "Bobo-Dioulasso", "Koudougou"]
print(len(set(villes)))
```

3

Définition. **valeur unique** — unique value — une valeur qui ne compte qu'une fois : `set(villes)` passe la liste au tamis (5 éléments → 3), `len(set(...))` compte les uniques. C'est le `COUNT(DISTINCT ville)` de M06, écrit en Python.

Et l'**intersection** — les clients qui sont dans les **deux** magasins (les codes de démo, le principe) :

```
c1 = {101, 102, 103, 108}
c2 = {102, 104, 105}
print(c1 & c2)
```

{102}

`c1 & c2` est le **JOIN interne** des deux ensembles : un seul code commun (102) passe le tamis. La union (`c1 | c2`), la différence (`c1 - c2`) : les trois opérateurs d'ensemble, les trois équivalents SQL.

Dans les faits. Le socle M08 porte 1 200 clients réels (`m08p_clients_lignes`). En C07, la question « combien de clients distincts ont acheté ? » sera `len(set(colonne_client))` sur la table entière — le même tamis que celui de ce paragraphe, passé de 9 codes de démo à 1 200 lignes.

5.6 Index, tranches, tris

Le tableau d'index complet, sur les 5 magasins (exécuté) :

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre", "Bobo Sarfalao", "Koudougou"]
print(magasins[0])
print(magasins[-1])
print(magasins[2:4])
print(magasins[:2])
print(magasins[::-1])
print(magasins[5])
```

```
Ouaga Centre
Koudougou
['Bobo Centre', 'Bobo Sarfalao']
['Ouaga Centre', 'Bobo Centre', 'Koudougou']
['Koudougou', 'Bobo Sarfalao', 'Bobo Centre', 'Ouaga Patte d'Oie', 'Ouaga Centre']
Traceback (most recent call last):
  File "<string>", line 7, in <module>
    print(magasins[5])
    ~~~~~^
IndexError: list index out of range
```

Définition. tranche — slice — `l[a:b]` : les éléments de l'index **a** à l'index **b - 1** (la fin est **exclue**) ; `l[:2]` tous les deux ; `l[::-1]` l'inverse. La tranche **ne lève jamais** hors plage : `magasins[3:9]` renvoie simplement jusqu'à la fin — c'est l'`IndexError` de l'index **simple** qui protège, pas la tranche.

Attention. La fin de tranche est **exclue** : `magasins[1:3]` contient les indexes 1 et 2 — pas 1, 2, 3. « de la ligne a à la ligne b **comprises** » s'écrit `l[a - 1 : b]` : penser en indexes, pas en lignes (§8, erreur n° 6).

Expression	Résultat	Lecture
<code>l[0] / l[-1]</code>	premier / dernier	la ligne 1 / la ligne dernière
<code>l[2:4]</code>	les indexes 2 et 3	2 éléments (fin exclue)
<code>l[:2]</code>	un élément sur deux	pas de 2
<code>l[::-1]</code>	l'inverse	pas -1
<code>l[5]</code> (5 éléments)	<code>IndexError</code>	la ligne 5 a l'index 4

Les **tris** — les deux frères, et le top N (exécutés) :

```
montants = [3500, 2500, 4500, 1500, 2000]
print(sorted(montants, reverse=True))
print(montants)
montants.sort(reverse=True)
print(montants)
```

```
[4500, 3500, 2500, 2000, 1500]
[3500, 2500, 4500, 1500, 2000]
[4500, 3500, 2500, 2000, 1500]
```

`sorted(...)` **renvoie** une nouvelle liste (l'originale est intacte — regardez la 2^e sortie) ; `.sort(...)` **tri en place** (renvoie `None`). Et le **top N d'un dictionnaire** — le `ORDER BY ... LIMIT n` de M06, en une ligne (valeurs de démo) :

```
ca = {1: 25000, 2: 150000, 3: 45000, 4: 120000, 5: 21000}
top2 = sorted(ca.items(), key=lambda p: p[1], reverse=True)[:2]
print(top2)
```

```
[(2, 150000), (4, 120000)]
```

`ca.items()` donne les paires (clé, valeur) ; `key=lambda p: p[1]` dit « trie sur la **valeur** » (le `p[1]` — la 2^e composante de la paire) ; `[:2]` prend le top. La `lambda` — la fonction sans nom — arrive en C05 ; ici, lisez-la comme « la règle de tri, en une ligne ».

Les 5 conteneurs et leurs équivalents SQL, en une planche :

Ranger : 5 conteneurs, 5 questions métier

Choisir le contenant, c'est choisir la réponse — l'autre, c'est du débogage.

liste « le n-ième ? » — l'ordre <code>l[i] · l[1:3] · append</code> la ligne n a l'index n-1 <code>IndexError</code> sinon	tuple « le fait figé » — la clé <code>n, p = t · clé de dict</code> immuable : scellé <code>TypeError</code> si on répare	str « le texte propre » <code>strip · split · replace</code> format français : <code>replace</code> puis <code>float</code> <code>f" {158140:,.0f}"</code>
dict « le comptage par... » <code>d.get(k, 0) + 1</code> = le <code>GROUP BY</code> de M06 <code>KeyError</code> sans <code>.get</code>	set « combien de distincts ? » <code>len(set(x)) · c1 & c2</code> = <code>COUNT(DISTINCT)</code> · <code>JOIN</code> interne le tamis : doublons sortis	tri « le top N » <code>sorted(items, key=...)[:n]</code> = <code>ORDER BY ... LIMIT n</code> <code>sorted</code> renvoie · <code>sort</code> modifie

C07 : le DataFrame reprend tout — une colonne = une liste indexée (loc/iloc),
une table = un dictionnaire de colonnes — les 50 008 lignes du socle attendent.

6. Exemple concret — les 5 magasins dans 4 contenants

La question du chapitre, en langage de comptoir : « j'ai la liste brute des 5 magasins (nom, ville, surface) — donnez-moi 4 points de vue dessus. » Le script — exécuté dans l'atelier le 19/09/2026 :

```
brut = [("Ouaga Centre", "Ouagadougou", 450),
        ("Ouaga Patte d'Oie", "Ouagadougou", 380),
        ("Bobo Centre", "Bobo-Dioulasso", 520),
        ("Bobo Sarfalao", "Bobo-Dioulasso", 290),
        ("Koudougou", "Koudougou", 210)]
noms = [ligne[0] for ligne in brut]
coordonnees = tuple((ligne[1], ligne[2]) for ligne in brut)
par_ville = {}
for ligne in brut:
    par_ville.setdefault(ligne[1], []).append(ligne[0])
villes = {ligne[1] for ligne in brut}
print("1. le 3e magasin :", noms[2])
print("2. nombre de villes distinctes :", len(villes))
print("3. magasins de Bobo-Dioulasso :", par_ville.get("Bobo-Dioulasso", []))
print("4. surface de Ouaga Centre :", dict((l[0], l[2]) for l in brut)["Ouaga Centre"])
print("5. le plus grand magasin :", max(brut, key=lambda l: l[2])[0])
```

```
1. le 3e magasin : Bobo Centre
2. nombre de villes distinctes : 3
3. magasins de Bobo-Dioulasso : ['Bobo Centre', 'Bobo Sarfalao']
4. surface de Ouaga Centre : 450
5. le plus grand magasin : Bobo Centre
```

Cinq questions, cinq postures : **index** (le 3^e), **tamis** (les villes distinctes), **étiquette avec défaut** (les magasins d'une ville, `get` — pas de `KeyError` si la ville n'y est pas), **étiquette** (la surface d'un nom), **règle de tri** (le plus grand, `max` avec `key`). Les 4 contenants (liste de noms, tuple de coordonnées, dictionnaire ville → magasins, ensemble de villes) sont construits **une fois** et servent les 5 questions — c'est l'architecture que C07 reprend à l'échelle du `DataFrame`.

7. Démonstration pas à pas — 6 étapes sur le fil rouge

Sortie réelle de chaque commande (exécutées le 19/09/2026).

Étape 1 — Le `COUNT(DISTINCT)` en une ligne (l'ensemble) :

```
$ python3 -c "
villes = ['Ouagadougou', 'Ouagadougou', 'Bobo-Dioulasso', 'Bobo-Dioulasso', 'Koudougou']
print(len(set(villes)))"
3
```

Interprétation : 5 occurrences, 3 villes — le tamis a fait le `DISTINCT` ; en C07, la même ligne passera sur la colonne `ville` de 50 008 lignes.

Étape 2 — Le `GROUP BY` en 3 lignes (le compteur) :

```
$ python3 -c "
vente_par_magasin = [1, 2, 1, 3, 1, 4, 2, 1]
compteur = {}
for idm in vente_par_magasin:
    compteur[idm] = compteur.get(idm, 0) + 1
print(compteur)"
{1: 4, 2: 2, 3: 1, 4: 1}
```

Interprétation : 8 ventes, 4 magasins — la 5^e n'a pas vendu (et `compteur.get(5, 0)` répondrait 0 sans s'effondrer) : c'est le `GROUP BY`, pas le `JOIN`.

Étape 3 — La tranche des magasins de Bobo (l'index) :

```
$ python3 -c "
magasins = ['Ouaga Centre', \"Ouaga Patte d'Oie\", 'Bobo Centre', 'Bobo Sarfalao', 'Koudougou']
print(magasins[2:4])"
['Bobo Centre', 'Bobo Sarfalao']
```

Interprétation : les indexes 2 et 3, donc les 3^e et 4^e lignes — pas les 2^e et 3^e : la ligne *n* a l'index *n* - 1.

Étape 4 — Le top 2 (le tri par valeur) :

```
$ python3 -c "
ca = {1: 25000, 2: 150000, 3: 45000, 4: 120000, 5: 21000}
top2 = sorted(ca.items(), key=lambda p: p[1], reverse=True)[:2]
print(top2)"
[(2, 150000), (4, 120000)]
```

Interprétation : tri sur la valeur (le `p[1]`), décroissant, tranche `[:2]` — le `ORDER BY ca DESC LIMIT 2`.

Étape 5 — L'intersection des clients (l'ensemble) :

```
$ python3 -c "
c1 = {101, 102, 103, 108}
c2 = {102, 104, 105}
print(c1 & c2)"
{102}
```

Interprétation : un seul client commun — le `JOIN` interne en un symbole.

Étape 6 — Le montant lisible (la chaîne) :

```
$ python3 -c "print(f\"panier moyen : {158140:,.0f} FCFA\")"
panier moyen : 158,140 FCFA
```

Interprétation : le panier moyen réel du socle (clé `m08p_panier_moyen`), formaté en une f-string — le formatage des rapports de tout le module.

8. Erreurs fréquentes

1. **IndexError: list index out of range** — la ligne *n* a l'index *n* - 1. Exécuté dans l'atelier (5 éléments, demande de l'index 5) : `text File "<string>", line 7, in <module> print(magasins[5]) ~~~~~^^^ IndexError: list index out of range` Remède : la 5^e ligne est `magasins[4]` ; pour « la dernière », `magasins[-1]` — jamais d'index calculé en tête sans le - 1.
2. **Modifier un tuple** — **TypeError**. `surface[0] = 999` sur un tuple : `TypeError: 'tuple' object does not support item assignment` (sortie réelle, §5.2). Remède : le tuple **se refait**, ne se répare pas (`surface + (210,)`) — et si c'est censé changer, c'était une **liste**.
3. **d[clé] sur une clé absente** — **KeyError**. Exécuté dans l'atelier : `text File "<string>", line 2, in <module> print(par_ville["Sintani"]) ~~~~~^~~~~~ KeyError: 'Sintani'` Remède : `d.get(clé, défaut)` sur toute donnée qu'on ne **possède** pas (fichier, saisie) — le défaut est la politesse du script (§5.4).
4. **clé in d cherche les clés, pas les valeurs**. `4 in compteur` répond « la clé 4 existe-t-elle ? » — pas « la valeur 4 existe-t-elle ? ». Remède : pour chercher dans les **valeurs**, un petit `for` : `any(v == 4 for v in compteur.values())` — la recherche par valeur est un parcours, pas une étiquette.
5. **sorted vs sort** — le tri qui « ne s'est pas fait ». `tri = montants.sort()` donne `tri = None` : le tri s'est bien fait **dans** `montants`, mais ce qu'on a re-affecté est `None` ; et `sorted(...)` **ne touche pas** l'originale — la sortie réelle le montre (`[3500, 2500, ...]` intacte après le `sorted`, §5.6). Remède : `tri = sorted(montants)` pour garder l'originale, `montants.sort()` (et ne pas re-affecter) pour trier **sur place** — vérifier laquelle on veut **avant** de taper.

6. **La tranche qui « manque » un élément.** `magasins[1:3]` donne 2 éléments, pas 3 — la fin est **exclue**. Remède : « de la ligne a à la ligne b **comprises** » = `l[a - 1 : b]` — ou, plus simple : penser en indexes, pas en lignes.

9. Bonnes pratiques professionnelles

1. **Un contenant par question** : les villes distinctes → ensemble ; le comptage par magasin → dictionnaire ; l'ordre des magasins → liste. Tout mettre dans une liste unique, c'est répondre à toutes les questions avec le même marteau.
2. **d.get(clé, défaut) sur ce qu'on ne possède pas** : fichier, saisie, API — la clé absente est un **cas prévu**, pas une exception (le `KeyError` du §8 est le script qui s'effondre sur un cas prévu).
3. **len(set(...)) dès que « distinct » apparaît** : ne jamais compter les uniques dans une liste en la parcourant à la main — le tamis fait le travail et dit la vérité.
4. **Le top N est toujours sorted(..., key=...)[:n]** — pas de tri manuel, pas de `for` qui cherche le max n fois : une ligne, lisible, testable.
5. **Nommer le contenant par son contenu** : `villes`, `par_ville`, `compteur` — pas `data1`, `liste2` : le nom dit la **question** que le contenant répond.

10. Exercice guidé — « les 5 questions des magasins » (15 min, /10)

Énoncé. Écrire `magasins.py` qui, à partir de la liste brute `brut` (les 5 tuples `(nom, ville, surface)` de §6) :

1. construit **4 contenants** : `noms` (liste), `coordonnees` (tuple de paires ville/surface), `par_ville` (dictionnaire ville → liste de noms), `villes` (ensemble) ;
2. répond aux 5 questions de §6 — avec le **bon contenant** pour chacune.

Barème. Les 4 contenants construits (4 pts) · les 5 questions posées avec le contenant qui les répond — `index` / `len(set(...))` / `get` / dictionnaire / `max` (2,5 pts, -0,5 par question posée au mauvais contenant) · la sortie alignée sur le modèle ci-dessous (2 pts) · aucun `d[clé]` ni sur `par_ville` (1 pt). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```
1. le 3e magasin : Bobo Centre
2. nombre de villes distinctes : 3
3. magasins de Bobo-Dioulasso : ['Bobo Centre', 'Bobo Sarfalao']
4. surface de Ouaga Centre : 450
5. le plus grand magasin : Bobo Centre
```

11. Exercices autonomes

Exercice 4.1 — La tranche des Bobo. Sur la liste des 5 magasins (ordre du socle), extraire **par tranche** les 2 magasins de Bobo-Dioulasso. (3 min)

Exercice 4.2 — Le compteur des paiements. Sur la liste `["Espèces", "Mobile Money", "Espèces", "Carte Bancaire", "Mobile Money", "Espèces"]` (les 3 libellés réels de `mode_paiement.csv`), construire le compteur dictionnaire et l'afficher. (5 min)

Exercice 4.3 — Le tamis. Sur la liste `["Ouagadougou", "Bobo-Dioulasso", "Ouagadougou", "Koudougou", "Bobo-Dioulasso", "Ouagadougou"]`, donner le nombre de villes **distinctes** — avec `set`, sans parcours manuel. (3 min)

Exercice 4.4 — Le top 2. Sur le dictionnaire `ca = {1: 25000, 2: 150000, 3: 45000, 4: 120000, 5: 21000}`, afficher le top 2 par valeur, en paires. (5 min)

Exercice 4.5 — Le nettoyage. Transformer la chaîne " 15 000,50 " en float, puis l'afficher avec 2 décimales en f-string. (5 min)

12. Correction détaillée

Exercice guidé. Le script de §6 **est** la correction (exécuté dans l'atelier) — les 5 points de vigilance : `noms` par `index` (`noms[2]`, pas de recherche), `villes` en `set` (le `len` direct), `par_ville.get(..., [])` (**pas** `par_ville[ville]` — le `KeyError` du §8 si la ville est absente), la surface par un dictionnaire `nom → surface` (`dict((l[0], l[2]) for l in brut)`), le plus grand par `max(brut, key=lambda l: l[2])`.

Exercice 4.1. `magasins[2:4] → ['Bobo Centre', 'Bobo Sarfalao']` : les indexes 2 et 3 (fin exclue, §8 erreur n° 6).

Exercice 4.2. Le compteur de §5.4 sur les libellés :

```
{'Espèces': 3, 'Mobile Money': 2, 'Carte Bancaire': 1}
```

(six paiements, 3 libellés — le `GROUP BY mode_paiement` de M07, sur 6 lignes au lieu de 50 008.)

Exercice 4.3. `len(set(villes)) → 3` : 6 occurrences, 3 villes distinctes — le `COUNT(DISTINCT ville)` en une ligne.

Exercice 4.4. `sorted(ca.items(), key=lambda p: p[1], reverse=True)[:2] → [(2, 150000), (4, 120000)]` : le tri porte sur `p[1]` (la valeur), `[:2]` coupe — le `ORDER BY ... LIMIT 2`.

Exercice 4.5. `float(s.strip().replace(" ", "").replace(",", ".")) → 15 000,50` converti ; affiché : `15,000.50` (la convention par défaut de Python, §5.4 de C02 — le format français s'écrit à la main si le rapport l'exige).

13. Mini-projet M08.P4 — « L'annuaire des magasins » (1 h)

Énoncé. Écrire `annuaire.py` — le §6, durci en **rapport** :

- la liste brute `brut` (5 tuples) est en tête, et les contenants sont construits **une fois** (liste de noms, tuple de surfaces, dictionnaire `par_ville`, ensemble de villes, **et** un dictionnaire `nom → surface` — 5 au total, le dernier servant la recherche inverse) ;
- le rapport **par ville triée** : pour chaque ville, la liste de ses magasins et la **somme de leurs surfaces** ;
- la ligne « villes distinctes : n » ;
- la ligne « plus grand magasin : nom (surface m2) » ;
- la recherche **inverse** : « magasin de 450 m2 : nom » (du chiffre au nom — le dictionnaire qui sert dans les deux sens).

Critères de réussite (grille /10) : les 5 exigences (5 × 1 pt) · la sortie de l'atelier : `Bobo-Dioulasso : Bobo Centre, Bobo Sarfalao — 810 m2, Koudougou : Koudougou — 210 m2, Ouagadougou : Ouaga Centre, Ouaga Patte d'Oie — 830 m2, villes distinctes 3, plus grand magasin Bobo Centre (520 m2), magasin de 450 m2 Ouaga Centre (1 pt) · aucun d[clé] nu sur par_ville (1 pt) · la somme de surfaces par ville calculée, pas resaisie (1 pt) · docstring en tête (1 pt).`

Note de correction. Les sommes ($810 = 520 + 290$; $830 = 450 + 380$) sont **mesurées** dans l'atelier — l'apprenant qui les annonce sans exécuter trahit le principe du module (prédire, exécuter, comparer). La ville s'affiche **triée alphabétiquement** (`sorted(par_ville)`) : Bobo-Dioulasso, Koudougou, Ouagadougou — le tri des clés est le `ORDER BY ville` du rapport.

14. Boîte à outils du chapitre

Boîte à outils. Python nu + terminal toujours ; la boîte du chapitre s'ajoute à celles de C02-C03 : les 5 contenants et leurs opérateurs signature.

Contenant	Question métier	Opérateur signature
liste	« le n° ? » — l'ordre	<code>l[i]</code> , <code>l[1:3]</code> , <code>append</code>
tuple	« le fait figé » — la clé	<code>n, p = t</code> , <code>t</code> en clé de dict
str	« le texte propre »	<code>strip</code> , <code>split</code> , <code>replace</code>
dict	« le comptage par... »	<code>d.get(k, 0)</code> , <code>k in d</code>
set	« combien de distincts ? »	<code>len(set(x))</code> , <code>c1 & c2</code>
tri	« le top N »	<code>sorted(d.items(), key=...)[:n]</code>

15. Résumé du chapitre

- La **liste** est la case numérotée (mutable, doublons) ; `l[-1]` est le dernier, `l[1:3]` contient 2 éléments (fin exclue), `l[5]` sur 5 éléments lève `IndexError` — la ligne `n` a l'index `n - 1`.
- Le **tuple** est l'enveloppe scellée : `TypeError` si on la répare, `n, p = t` pour l'ouvrir, et seule l'immuable a le droit d'être **clé** de dictionnaire.
- La **chaîne** est la séquence de caractères : `strip/split/replace` nettoient le format français, la f-string l'affiche (`158,140 FCFA`).
- Le **dictionnaire** est l'étagère étiquetée : `d.get(clé, défaut)` ne s'effondre jamais, `clé in d` teste les **clés**, et le compteur `d.get(k, 0) + 1` **est** le `GROUP BY`.
- L'**ensemble** est le tamis : `len(set(villes)) = COUNT(DISTINCT ville)`, `c1 & c2` = le `JOIN` interne.
- `sorted(...)` renvoie, `.sort(...)` modifie ; le top N est `sorted(d.items(), key=lambda p: p[1], reverse=True)[:n]` — le `ORDER BY ... LIMIT n`.

16. À retenir

À retenir. Avant de taper un contenant, la question : « **quelle est la question métier ?** » — l'ordre (liste), le fait figé (tuple), l'étiquette (dictionnaire), l'unicité (ensemble). Le bon contenant rend la bonne question **immédiate** et la mauvaise difficile ; l'inverse, c'est du débogage.

À retenir. `d[clé]` lève, `d.get(clé, défaut)` renvoie : sur les données qu'on ne possède pas (fichiers, saisies), seul le second a le droit d'exister — le `KeyError` d'un cas prévu est le script qui s'effondre de politesse.

17. Évaluation formative (auto-correction, 8 min)

1. `magasins[2:4]` contient combien d'éléments, et pourquoi ? → 2 (les indexes 2 et 3) : la borne de fin est **exclue** — la convention des tranches Python. (1 pt)
2. `magasins[5]` sur une liste de 5 éléments fait quoi, et quelle est la bonne demande ? → `IndexError: list index out of range` ; la 5^e ligne est `magasins[4]` (ou `magasins[-1]`). (1 pt)
3. Pourquoi le tuple est-il bon pour clé de dictionnaire et pas la liste ? → le tuple est **im-muable**, donc hachable : `dict` accepte une clé tuple ; la liste est mutable — `dict` la refuse (`TypeError: unhashable type: 'list'`). (2 pts)
4. `compteur[idm] = compteur.get(idm, 0) + 1` — que fait la `.get(..., 0)` exactement ? → prend le comptage de la clé **ou 0 si la clé est absente** : pas de `KeyError` au premier passage — c'est ce qui fait le `GROUP BY`. (2 pts)
5. `clé in d` teste quoi — les clés ou les valeurs ? → les **clés** ; chercher dans les valeurs demande un parcours (`any(v == x for v in d.values())`). (1 pt)
6. `len(set(villes))` est l'équivalent SQL de quoi ? → `COUNT(DISTINCT ville)` — le tamis compte les uniques. (1 pt)
7. `sorted(montants)` et `montants.sort()` — la différence en une phrase ? → `sorted` renvoie une nouvelle liste (l'originale intacte) ; `.sort` trie **en place** (renvoie `None`). (1 pt)
8. Le top 2 d'un dictionnaire ca par valeur, en une ligne ? → `sorted(ca.items(), key=lambda p: p[1], reverse=True)[:2]` — le `ORDER BY ... LIMIT 2`. (1 pt)

Module M08.C05 — Organiser : fonctions, arguments, portée, docstring, modules, `try/except`, fichiers

Outils : Python 3.13.14 seul + la bibliothèque standard (`csv`, `json`), le système de fichiers. **Aucun paquet tiers.** **Durée indicative :** 4 h. **Niveau :** N5 → N6. **Prérequis :** C04 (les conteneurs).

L'idée du chapitre. C04 a donné le **rangement** ; C05 donne l'**organisation** : découper le script en **fonctions** testables (le calculateur de remise de C02 devient enfin une fonction), savoir ce qui **entre** (arguments) et ce qui **sort** (`return` — pas `print`), ce qui est **visible** (la portée), ce qu'on **documente** (la docstring), ce qu'on **protège** (`try/except` — jamais d'`except` nu), et comment le travail **sort de l'écran** (les fichiers : CSV, JSON, l'encodage `utf-8-sig` et le BOM). Le fil rouge : le **nettoyeur de montants** — une fonction qui reçoit `"15 000,50"`, `"15000.50"` ou `"N/C"` et renvoie un `float` ou `None`, 6 cas de test exécutés. C07 le retrouvera devant 50 008 lignes.

Matériel de l'atelier — Python 3.13.14, bibliothèque standard, dossier du socle. Tout le code de ce chapitre a été exécuté dans l'atelier le 19/09/2026 ; les sorties publiées sont **celles de l'atelier**. Les fichiers cités sont réels : `03_exercices/dossier_M08/vente.csv` (13 colonnes, 50 008 lignes — clé `m08p_ventes_lignes`) et `03_exercices/dossier_M08/ATTENDU.json` (30 clés, la fiche de contrôle du socle). Les montants : 1 500, 1 200, 15 000,50, 158 140 (clés du socle).

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Écrire une **fonction** avec `def` et `return` — et expliquer pourquoi `print` n'est pas une sortie (le piège `None`, exécuté).
2. Passer des **arguments** : positionnels, nommés (`calculer_remise(montant=1500, taux=0.05)`), avec **valeurs par défaut** — et éviter le piège de la valeur par défaut **mutable** (exécuté : `2 2` au lieu de `1 1`).
3. Dire ce qui est **visible** : portée locale vs globale, la variable de boucle qui « fuit » (elle existe après la boucle), `global` (à éviter).
4. Écrire une **docstring** à la convention (Quoi / Entrées / Sortie / Exemple) et la vérifier avec `help(f)`.
5. Placer les **imports** en tête de fichier, et distinguer `import`, `from ... import`.
6. Intercepter l'**erreur attendue** avec `try/except` **spécifié** (jamais d'`except` : nu — le bug silencieux, exécuté) — et lire une **traceback en 3 couches**.
7. Lire et écrire des **fichiers** : `open` avec `with`, l'encodage `utf-8-sig` (le BOM), le module `csv` sur le vrai `vente.csv` du socle, `json.load/json.dump` sur le vrai `ATTENDU.json`.

2. Pourquoi cette notion est importante

Parce que le passage de C04 à C05 est le passage du **brouillon** au **livrable**. Un script de 40 lignes dans le même fichier est un brouillon ; un script découpé en fonctions nommées, documentées, testées — c'est un livrable qu'un collègue peut **lire** et **reprendre**. La fonction est la brique de cette transition : elle a un nom (donc une raison d'exister), des entrées (donc un contrat), une sortie (donc un résultat **vérifiable** sans affichage).

Et parce que la machine commence à **travailler sur du réel** : les fichiers du dossier M08 (8 CSV + ATTENDU.json) attendent C07, mais leurs **questions** arrivent ici : que faut-il faire pour qu'un CSV Excel lisible (BOM, format français) devienne des nombres ? Pourquoi `open("vente.csv")` échoue « alors que le fichier existe » (le chemin relatif, §8) ? Que vaut `json.load` sur la fiche du socle ? C05 installe les **habitudes** ; C07 les passera à l'échelle.

Enfin, `try/except` est la différence entre un script qui **gère** le cas prévu (le montant « N/C » est un **fait de métier**, pas un accident) et un script qui **s'effondre** sur lui — et `l'except` nu est le piège qui transforme ce dernier en **mensonge** : il « marche » et renvoie des résultats faux sans un mot (le §8 l'exécute).

3. Explication simple — la machine à café

- La **fonction** est une **machine à café** : on met ce qu'elle attend (argent, choix), on appuie, elle donne **une chose** (le café) — et rien d'autre. `return` est la chose qu'elle donne ; `print` est le bruit qu'elle fait **en plus** (bruyant, et ce n'est pas le café).
- L'**argument par défaut** est le **choix préimprimé** sur la machine : si on ne précise rien, la machine prend le choix d'usine (`taux=0.10` — la remise de septembre, C03).
- La **portée** est la **cuisine** : ce qui est préparé dedans (les variables locales) ne sort pas — sauf si on l'a mis sur le comptoir (`return`).
- La **docstring** est la **notice** collée sur la machine : quoi, entrées, sortie, exemple — `help(f)` la lit à voix haute.
- Le **try/except** est le **filet sous le gobelet qui fuit** : on l'a mis **là où** la fuite est attendue (le montant illisible), pas sur toute la cuisine (`l'except` nu).
- Le **fichier** est le **tiroir** : le travail y entre (`read`), y sort (`write`) — et l'encodage, c'est l'**étiquette de langue** du tiroir (`utf-8-sig`, le BOM).

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
fonction — function	un bloc nommé, réutilisable, qui renvoie une valeur avec <code>return</code> .	faire <code>print</code> et croire que c'est la sortie : la sortie est le <code>return</code> , le <code>print</code> est un bruit.
argument par défaut — default argument	la valeur prise quand l'appel ne précise rien (<code>taux=0.10</code>).	la faire mutable (<code>panier=[]</code>) : elle est partagée entre tous les appels qui ne la précisent pas.
portée — scope	la zone de visibilité d'une variable : locale (dans la fonction) ou globale (partout).	croire qu'une variable locale existe dehors : elle meurt avec la fonction (le <code>NameError</code> du §8).
docstring docstring	— le commentaire en tête de fonction, entre triples guillemets, lu par <code>help(f)</code> .	la faire répéter le code : elle doit dire le contrat (entrées, sortie, exemple), pas le listing.
exception exception	— une erreur levée à un endroit, qui peut être interceptée plus haut (<code>except</code>).	l'intercepter sans la nommer (<code>except:</code>) : on avale aussi le bug qu'on cherchait.
BOM — byte order mark	les 3 octets invisibles en tête d'un fichier <code>utf-8-sig</code> , qui corrompent la 1 ^{re} colonne si on lit en <code>utf-8</code> pur.	lire un fichier Excel en <code>utf-8</code> : la première clé devient <code>'\ufeffid_...'</code> — une clé qui n'existe pas.

5. Cours approfondi

5.1 La fonction — le `return` est la sortie, pas le `print`

Le calculateur de remise de C02/C03 devient enfin une fonction — le fil rouge du module :

```
def calculer_remise(montant, taux=0.10):
    return montant * (1 - taux)

print(calculer_remise(1500))
```

```
1350.0
```

Définition. fonction — fonction — un bloc nommé de code, appelé par son nom avec des arguments, qui **renvoie une valeur** avec `return`. Sans `return`, une fonction renvoie `None` — même si elle a **affiché** quelque chose : le `print` est un bruit, pas une sortie.

Le piège, exécuté dans l’atelier (la fonction affiche le bon nombre... et renvoie `None`) :

```
def affiche_remise(montant):
    print(montant * 0.9)

r = affiche_remise(1500)
print(r)
```

```
1350.0
None
```

La 2^e ligne est la leçon : `r` a reçu `None`, pas 1350.0. La fonction a **affiché** la valeur au lieu de la **rendre** — le test (et le collègue) n’a plus qu’un `None` en main.

Conseil professionnel. Une fonction fait **une chose** et son nom dit quoi, au **verbe** : `calculer_remise`, `nettoyer_montant`, `charger_fichier` — pas `fonction1`, pas `trucs`. Et la sortie est **toujours** un `return` : le `print` reste pour l’humain qui regarde, le `return` est pour la machine qui enchaîne.

5.2 Les arguments — positionnels, nommés, par défaut

Trois façons d’appeler, exécutées (la remise passe de 10 % à 5 %) :

```
def calculer_remise(montant, taux=0.10):
    return montant * (1 - taux)

print(calculer_remise(1500))
print(calculer_remise(1500, 0.05))
print(calculer_remise(montant=1500, taux=0.05))
```

```
1350.0
1425.0
1425.0
```

- **positionnel** : `calculer_remise(1500, 0.05)` — l’ordre de la signature ;
- **nommé** : `calculer_remise(montant=1500, taux=0.05)` — l’ordre est libéré, la lecture est gagnée (c’est l’AS du SELECT de M06, appliqué à l’appel) ;
- **par défaut** : `taux=0.10` — le taux non précisé prend la valeur d’usine.

Et le piège du **par défaut mutable**, exécuté dans les deux sens (le bug, puis la correction) :

```
def ajouter(vente, panier=[]):
    panier.append(vente)
    return panier

a = ajouter(1500)
b = ajouter(2000)
print(len(a), len(b))
```

```
2 2
```

Attention. La valeur par défaut **mutable** est construite **une fois**, au moment de la `def` — et **partagée** entre tous les appels qui ne la précisent pas : le 2^e appel a vu le 1^e (le panier a 2 éléments, pas 1). C’est le bug silencieux le plus célèbre de Python : il ne lève rien, il **souille**.

La correction — `None` comme défaut, la liste construite **à chaque appel** :

```
def ajouter(vente, panier=None):
    if panier is None:
        panier = []
    panier.append(vente)
    return panier

a = ajouter(1500)
b = ajouter(2000)
print(len(a), len(b))
```

```
1 1
```

Définition. argument par défaut — default argument — la valeur prise quand l’appel ne précise pas l’argument : `calculer_remise(1500)` prend `taux=0.10`. Règle du chapitre : **jamais** de mutable en défaut (`[]`, `{}`) — `None`, puis reconstruction à l’intérieur.

5.3 La portée — la cuisine et le comptoir

Deux phénomènes exécutés dans l’atelier. D’abord, la **fuite** : la variable de boucle **survit** à la boucle (elle est créée dans la portée du script, pas dans une fonction) :

```
for m in [1500, 2000]:
    pass
print(m)
```

```
2000
```

Ensuite, l’invisible : une variable **locale** (définie dans une fonction) meurt avec elle — demandez-la dehors, la machine se proteste :

```
print(x)
^
NameError: name 'x' is not defined
```

Définition. portée — scope — la zone de visibilité d’une variable : **locale** (née et morte dans la fonction), **globale** (le fichier tout entier). Une fonction **lit** les globales, ne les **modifie** que par `global` — et `global` est à éviter : la fonction qui écrit une variable globale est une fonction qui ne se teste pas seule (elle dépend de ce qu’il y a **dehors**).

Le compteur global, exécuté (pour savoir à quoi ressemble la tentation) :

```
compteur = 0

def vendre():
    global compteur # l'écriture dans le fichier
    compteur = compteur + 1

vendre()
vendre()
print(compteur)
```

2

Ça marche — et ça **couple** : `vendre()` ne se teste pas sans réinitialiser `compteur`. La version saine rend le compteur : `def vendre(compteur): return compteur + 1` — l'entrée et la sortie, rien de caché.

5.4 La docstring — la notice sur la machine

La convention du module (la règle M04, reprise en Python) : **Quoi / Entrées / Sortie / Exemple** — et `help(f)` la lit :

```
def calculer_remise(montant, taux=0.10):
    """Prix net apres remise.

    Entrees : montant (float), taux (float, par default 0.10).
    Sortie : montant * (1 - taux).
    Exemple : calculer_remise(1500) -> 1350.0
    """
    return montant * (1 - taux)

help(calculer_remise)
```

```
Help on function calculer_remise in module __main__:

calculer_remise(montant, taux=0.1)
    Prix net apres remise.

    Entrees : montant (float), taux (float, par default 0.10).
    Sortie : montant * (1 - taux).
    Exemple : calculer_remise(1500) -> 1350.0
```

Définition. docstring — docstring — la chaîne en **tête** de fonction, entre triples guillemets, lue par `help(f)` et récupérable par `f.__doc__`. Elle écrit le **contrat** (entrées, sortie, exemple) — pas le listing : le code dit déjà **comment**, la docstring dit **quoi** on a le droit d'en attendre.

Deux remarques sur la sortie : `help` a rendu **taux=0.1** (le défaut, affiché en nombre, plus `0.10`) — la docstring reste la source du **sens** ; et la docstring est la **première ligne de code** de la fonction : tout ce qui est avant elle ne compte pas.

5.5 Modules et paquets — l'import en tête

La règle M04, reprise : **les imports vivent en tête de fichier**, avant tout le reste. Le script du §13 (et tous les scripts de C05) commence par :

```
import csv
import json
```

- `import csv` : on appelle ensuite `csv.reader`, `csv.writer` — le **point** fait la distinction (c'est le `.` du `module.fonction`) ;
- `from numpy import where` (C06) : on importe **une pièce** nommément, sans le point ;
- un **paquet tiers** (pandas en C07, `pip install` en C01) suit la même règle : `import pandas as pd` en tête, **une seule fois**, et le reste du fichier n'écrit plus que du travail.

Ce qu'on met dans **son propre module** (la question de la fin du chapitre) : les fonctions qui se **réutilisent** (`calculer_remise`, `nettoyer_montant`, `fmt`), pas les brouillons d'exécution. Un module se reconnaît à sa docstring de tête et à ses imports en haut — c'est un **fichier qui est une bibliothèque**.

Dans les faits. Le dossier M08 du socle (8 CSV + `ATTENDU.json`, dont 50 008 lignes de ventes) s'ouvre ici tout en `utf-8-sig` : le vrai `vente.csv` n'a **pas** de BOM (sa 1^{re} colonne est lue propre, `id_vente`), mais la règle est la même pour un fichier venu d'Excel — `utf-8-sig` passe dans les deux cas, `utf-8` pur ment dans un. En C07, `pandas.read_csv` reprendra exactement ce paramètre.

5.6 Le `try/except` — le filet sous la fuite prévue

Le cas prévu, ici, est un **fait de métier** : la caisse envoie « N/C ». On intercepte **l'erreur nommée**, au **endroit** où elle est attendue :

```
def nettoyer_montant(s):
    """Convertir un montant saisi en float, ou renvoyer None s'il est illisible."""
    try:
        return float(str(s).strip().replace(" ", "").replace(",", "."))
    except ValueError:
        return None

print(nettoyer_montant("15 000,50"))
print(nettoyer_montant("N/C"))
```

```
15000.5
None
```

Définition. exception — exception — une erreur **levée** à l'endroit où le problème arrive, qui remonte la pile jusqu'au premier `except` qui la **nomme**. `except ValueError` intercepte **ce type-là** (et ce type-là seul) ; le `try` marque la zone où l'erreur est **prévue**, pas tolérée partout.

Attention. Jamais d'except: nu — c'est le filet sur **toute** la cuisine : il avale le bug qu'on cherchait **et** le cas prévu, et le script « marche » en renvoyant des résultats faux. Le §8 l'exécute : le `except nu` transforme le format français en `None` **sans un mot**.

`else` et `finally`, en un paragraphe pour la suite : `else` s'exécute **si rien n'a été levé** dans le `try` (le « succès »), `finally` s'exécute **toujours** (fermer une connexion, nettoyer) — les deux arrivent en C07 avec les connexions de bases de données.

5.7 Lire une traceback — les 3 couches

La traceback de C02 était **plate** (un seul cadre). Avec les fonctions, elle a une **hauteur** — la **pile d'appels** : chaque `File "...", line n, in nom` est un **cadre** (un étage du chemin parcouru), du

haut (le point d'entrée) vers le bas (l'endroit exact). Trois tracebacks réelles, exécutées dans l'atelier le 19/09/2026 :

1. `NameError` — la variable qui n'existe pas, vue de la pile :

```
Traceback (most recent call last):
  File "<string>", line 4, in <module>
    print(calculer_net(1500))
    ~~~~~^~~~~~
  File "<string>", line 3, in calculer_net
    return montant * (1 - remise)
    ~~~~~^~~~~~
NameError: name 'remise' is not defined
```

2. `KeyError` — la clé absente (C04), vue de la pile :

```
Traceback (most recent call last):
  File "<string>", line 5, in <module>
    print(surface("Sintani"))
    ~~~~~^~~~~~
  File "<string>", line 4, in surface
    return surfaces[magasin]
    ~~~~~^~~~~~
KeyError: 'Sintani'
```

3. `FileNotFoundError` — le fichier n'est pas ici (le chemin relatif) :

```
File "<string>", line 1, in <module>
  f = open('introuvable.csv')
FileNotFoundError: [Errno 2] No such file or directory: 'introuvable.csv'
```

La discipline, en trois temps :

1. **En bas, le message** : le **type** d'erreur d'abord (`NameError`, `KeyError`, `FileNotFoundError`) — c'est la famille, on la nomme avant de chercher ;
2. **Le cadre du bas** : la **ligne fautive** — ici, `remise` dans `calculer_net` (cadre 2), pas l'appel (cadre 1) : **l'erreur vit dans la fonction, l'appel est juste le témoin** ;
3. **La pile, en remontant** : le chemin parcouru (`<module>` → `calculer_net`) — utile quand l'erreur vient d'une fonction **appelée par une fonction** (C07 : `main` → `charger` → `nettoyer`).

Les 3 couches, sur la protestation réelle du §5.7 :

Lire une traceback : les 3 couches

Une erreur avec des fonctions a une hauteur : la pile d'appels. La lire, c'est le travail.

La protestation réelle (exécutée dans l'atelier) :

Traceback (most recent call last):

File "<string>", line 4, in <module>

print(calculer_net(1500))

File "<string>", line 3, in calculer_net

return montant * (1 - remise)

^^^^^

NameError: name 'remise' is not defined

1 • La pile d'appels

le chemin parcouru, du haut vers le bas
(le module appelle calculer_net)

2 • La ligne fautive + la fleche

l'erreur est DANS la fonction ;
l'appel (cadre du haut) est le témoin

3 • Le message, tout en bas

le nom de la famille d'abord :
NameError, pas la phrase

La discipline : lire de haut en bas pour le POURQUOI, s'arrêter en bas pour le QUOI.

Les 5 familles du debutant : SyntaxError (texte) · NameError (nom) · TypeError (famille)

· ValueError (valeur) · KeyError / FileNotFoundError (cle / fichier)

6. Exemple concret — le nettoyeur de montants

Le fil rouge du chapitre, version complète, exécutée dans l'atelier le 19/09/2026 — la fonction et les **6 cas de test** (chaque ligne est une prédiction vérifiée) :

```
def nettoyer_montant(s):
    """Convertir un montant saisi en float, ou renvoyer None s'il est illisible.

    Entree : s (str, ou numero deja converti).
    Sortie : float (ex. 15000.5) ou None (texte, vide).
    Exemple : nettoyer_montant("15 000,50") -> 15000.5
    """
    try:
        return float(str(s).strip().replace(" ", "").replace(",", "."))
    except ValueError:
        return None

tests = [
    ("15 000,50", 15000.5),
    ("15000.50", 15000.5),
    ("N/C", None),
    (" 1 200 ", 1200.0),
    ("", None),
    (1500, 1500.0),
]
for saisie, attendu in tests:
    resultat = nettoyer_montant(saisie)
    ok = "OK " if resultat == attendu else "ECART"
    print(f"{ok} {saisie!r:16} -> {resultat}")
```

```
OK '15 000,50'      -> 15000.5
OK '15000.50'      -> 15000.5
OK 'N/C'           -> None
OK ' 1 200 '       -> 1200.0
OK ''              -> None
OK 1500            -> 1500.0
```

Les 6 cas couvrent tout ce que la caisse envoie : le **format français** (1 500... 15 000,50 — la clé `m08p_demo_montant_fcfa` du socle), le **format américain** (15000.50 — les exports Excel), le **refus** (« N/C », un **fait de métier** intercepté par `except ValueError`), les **espaces parasites** (`strip`), le **vide** (le `float("")` lève aussi `ValueError` — le même filet), et le **nombre déjà converti** (`str(1500)` passe par la même porte). Zéro ECART : la fonction est **testée**, pas supposée — c'est le principe d'évaluation du module (prédire, exécuter, comparer) appliqué au code.

7. Démonstration pas à pas — 6 étapes sur le fil rouge

Sortie réelle de chaque commande (exécutées le 19/09/2026).

Étape 1 — La fonction fait son travail (le fil rouge de C02, enfin fonction) :

```
$ python3 -c "
def calculer_remise(montant, taux=0.10):
    return montant * (1 - taux)
print(calculer_remise(1500))"
1350.0
```

Interprétation : 1 500 – 10 % = 1 350 — la valeur de référence du module, rendue par `return`, pas affichée par accident.

Étape 2 — L'argument nommé libère la lecture (le taux précisés sans mémoriser l'ordre) :

```
$ python3 -c "
def calculer_remise(montant, taux=0.10):
    return montant * (1 - taux)
print(calculer_remise(montant=1500, taux=0.05))"
1425.0
```

Interprétation : 5 % au lieu de 10 — l'argument nommé `taux=0.05` dit ce qu'il modifie, l'appel se lit comme une phrase.

Étape 3 — La notice parle (`help` lit la docstring) :

```
$ python3 -c "
def calculer_remise(montant, taux=0.10):
    '''Prix net apres remise.
    Entrees : montant (float), taux (float, par default 0.10).
    Sortie : montant * (1 - taux).
    Exemple : calculer_remise(1500) -> 1350.0
    '''
    return montant * (1 - taux)
help(calculer_remise)"
Help on function calculer_remise in module __main__:

calculer_remise(montant, taux=0.1)
  Prix net apres remise.
  Entrees : montant (float), taux (float, par default 0.10).
  Sortie : montant * (1 - taux).
  Exemple : calculer_remise(1500) -> 1350.0
```

Interprétation : le contrat, lisible sans le code — `taux=0.1` est le défaut tel que Python l'affiche (0.10 en nombre) ; le sens reste dans la docstring.

Étape 4 — Le nettoyeur ouvre la porte (le format français en nombre) :

```
$ python3 -c "
def nettoyer_montant(s):
    try:
        return float(str(s).strip().replace(' ', '').replace(',', '.'))
    except ValueError:
        return None
print(nettoyer_montant('15 000,50'))"
15000.5
```

Interprétation : la recette de C02 (`strip`, `replace`) dans une fonction, le filet `except ValueError` sous le cas prévu — 15 000,50 est devenu 15 000,50 en nombre.

Étape 5 — Le BOM, la 1^{re} colonne qui ment (l'encodage) :

```
$ python3 -c "
open('/tmp/avec_bom.csv', 'w', encoding='utf-8-sig').write('nom;valeur\nx;l\n')
print(repr(open('/tmp/avec_bom.csv', encoding='utf-8').readline().strip()))
print(repr(open('/tmp/avec_bom.csv', encoding='utf-8-sig').readline().strip()))"
'\u0000nom;valeur'
'nom;valeur'
```

Interprétation : le même fichier lu deux fois — en `utf-8`, la première clé est `'\u0000nom'` (le BOM invisible collé devant), en `utf-8-sig` elle est propre : c'est pourquoi le module ouvre toujours en `utf-8-sig` (la règle M01).

Étape 6 — Le vrai CSV du socle (le `csv` standard, 13 colonnes) :

```
$ python3 -c "
import csv
with open('03_exercices/dossier_M08/vente.csv', encoding='utf-8-sig') as f:
    lecteur = csv.reader(f)
    en_tete = next(lecteur)
    print(len(en_tete), 'colonnes')
    for i, ligne in enumerate(lecteur):
        if i >= 2:
            break
        print(ligne[:4])"
13 colonnes
['1', '192', '380', '2']
['2', '318', '311', '1']
```

Interprétation : le vrai `vente.csv` du socle (clé `m08p_ventes_colonnes = 13`) lu par la bibliothèque standard — C07 fera la même lecture avec `pandas.read_csv`, et la première ligne lue est la ligne de tête, pas une vente.

8. Erreurs fréquentes

1. **print au lieu de return — la fonction qui « travaille » et renvoie None.** Exécuté dans l'atelier : `text 1350.0 None` La fonction a affiché le bon nombre et rendu `None` — l'appelant n'a plus rien. Remède : la sortie est **toujours** un `return` ; le `print` est un bruit de chantier, on le retire avant la livraison.
2. **L'argument par défaut mutable — le panier qui grandit tout seul.** Exécuté : `def ajouter(vente, panier=[])` donne 2 2 après deux appels au lieu de 1 1 (\$5.2). Remède : `panier=None`, puis `if panier is None: panier = []` — la mutable se **construit** à chaque appel, elle ne se **partage** pas.
3. **Le return oublié dans une branche — la fonction qui rend None « seulement » parfois.** Exécuté : `text 2 None` (`f(1)` rend 2, `f(-1)` rend `None` — même fonction, deux sorties.) Remède : **chaque branche** se termine par un `return` (ou une `return None` explicite qui dit le silence) — une fonction qui a une sortie « possible » doit la dire.

4. **La variable locale demandée dehors — `NameError` de portée.** Exécuté : `def f(): x = 5` puis `f()` puis `print(x)` → `NameError: name 'x' is not defined` (§5.3). Remède : la valeur doit **passer par le comptoir** — `x = f()` (le `return`), pas `print(x)` de l'autre côté du mur.
5. **`FileNotFoundError` — « le fichier existe, mais pas ici ».** Exécuté : `open('introuvable.csv')` → `FileNotFoundError: [Errno 2] No such file or directory: 'introuvable.csv'` (§5.7). Le piège est le **chemin relatif** : il se résout depuis le **dossier courant** du terminal, pas depuis le dossier du script — le fichier existe, il est **ailleurs**. Remède : chemin absolu dans les scripts de livraison, ou chemin relatif **documenté** (« lancer depuis la racine du dossier »).
6. **L'except: nu — le bug qui se tait.** Exécuté dans l'atelier : `python def nettoyer_tout(s): try: return float(s) except: return None print(nettoyer_tout('15 000,50')) print(nettoyer_tout('1500.50')) print(nettoyer_tout('N/C'))` text None 1500.5 None Lisez la sortie en trois temps : le **format français** (1^{re} ligne) est avalé par le filet universel et renvoyé `None` **sans un mot** ; le **format correct** (2^e ligne) passe — la fonction n'est pas morte, elle est **aveugle au cas qu'elle ne connaît pas** ; le **refus** « N/C » (3^e ligne) est le seul cas prévu. Le script « marche », les résultats sont faux, personne n'est prévenu. Remède : `except ValueError` **nommé** (§5.6) — et la recette avant le `float`, pas le filet pour rattraper la recette oubliée.

9. Bonnes pratiques professionnelles

1. **Le nom de fonction est un verbe** (`calculer_remise`, `nettoyer_montant`, `charger_fichier`) — et fait **une** chose : si le nom a un « et », c'est deux fonctions.
2. **Le `return` est unique par intention** : chaque branche se termine par une sortie ; une fonction qui « peut » ne pas rendre de valeur doit le dire dans la docstring.
3. **`None` en défaut, pas `[]`** : la mutable se construit à l'intérieur (§5.2) — la règle est mécanique, elle ne se négocie pas.
4. **L'exception est nommée et proche**** : `except ValueError` au niveau de la ligne qui peut lever, pas au niveau du `main` — le filet est sous la gouttière, pas sous le toit.
5. **Les imports en tête, le travail en bas** : le fichier se lit comme un contrat (`imports` → constantes → fonctions → exécution) ; un `import` au milieu du script est un signal d'alerte (code re-collé, pas écrit).
6. **`open` n'existe que sous `with`** : le fichier se **ferme** de lui-même à la fin du bloc — pas de `f = open(...)` nu, pas de `f.close()` à mémoriser.

10. Exercice guidé — « le nettoyeur de montants » (15 min, /10)

Énoncé. Écrire `nettoyeur.py` :

1. la fonction `nettoyer_montant(s)` avec **docstring** à la convention (Quoi / Entrée / Sortie / Exemple) ;
2. la recette du format français (`strip`, `replace` × 2) **avant** le `float` ;
3. le filet `try/except ValueError` qui renvoie `None` — et **un seul** type intercepté ;
4. les 6 cas de test de §6, affichés chacun avec `OK/ECART`.

Barème. La docstring à la convention (2 pts) · la recette complète avant le `float` (2 pts) · le `try/except ValueError` **spécifié** (2 pts) · les 6 cas couverts dont « N/C », le vide et un nombre déjà converti (2 pts) · la sortie alignée sur le modèle ci-dessous (2 pts). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```
OK '15 000,50'      -> 15000.5
OK '15000.50'       -> 15000.5
OK 'N/C'            -> None
OK ' 1 200 '        -> 1200.0
OK ''               -> None
OK 1500             -> 1500.0
```

11. Exercices autonomes

Exercice 5.1 — Le double. Écrire `doubler(v)` avec `return` (pas `print`), tester sur 500 et 1500.5 : afficher les deux sorties. (5 min)

Exercice 5.2 — La facture. Écrire `facture(montant, tva=0.19, escompte=0.0)` qui rend `montant × (1 + tva) × (1 - escompte)` — et l'appeler **3 fois** : sans option, avec la tva en position, avec l'escompte **nommé** (montant 1 000). (10 min)

Exercice 5.3 — Les deux paniers. Reproduire le bug du §5.2 (deux appels, le deuxième voit le premier) et le corriger avec `panier=None` : afficher les deux sorties (bug, correction). (10 min)

Exercice 5.4 — Le garde. Écrire `est_lisible(s)` qui renvoie `True/False` (pas d'exception qui s'échappe) en réutilisant la recette du nettoyeur. (10 min)

Exercice 5.5 — Le tiroir. Écrire un fichier `trois.csv` (3 lignes de données) avec `open` sous `with`, le relire, afficher le nombre de **lignes de données** (pas la tête). (10 min)

12. Correction détaillée

Exercice guidé. Le script de §6 **est** la correction (exécuté dans l'atelier) — les 5 points de vigilance : la docstring **avant** tout le code de la fonction (la première ligne), la recette **avant** le `float` (pas le `filet` pour rattraper la recette oubliée, §8 erreur n° 6), le `try` autour de la **ligne** qui peut lever, le `except ValueError` **nommé**, et le test qui affiche `OK/ECART` (la prédiction vérifiée, pas l'affichage passif).

Exercice 5.1. `def doubler(v): return v * 2` → 1000 3001.0. Le `print` serait un bruit : la sortie est dans le `return`.

Exercice 5.2. Les 3 appels (19 % de TVA — la règle du socle 2026) :

```
1190.0
1100.0
1130.5
```

→ `facture(1000)` (le défaut), `facture(1000, 0.10)` (la tva en position), `facture(1000, escompte=0.05)` (l'escompte nommé) : l'argument nommé dit **ce** qu'il modifie.

Exercice 5.3. Le bug : `def ajouter(vente, panier=[])` → 2 2 (la liste est construite une fois, au `def`). La correction : `panier=None` puis reconstruction → 1 1. Les deux sorties sont publiées telles qu'exécutées.

Exercice 5.4. `est_lisible` renvoie `True/False` — exécuté :

```
15 000,50 -> True
N/C -> False
(vide) -> False
```

Le `filet` est **dans** la fonction (aucune exception ne s'échappe) ; le `return` est dans les **deux** branches (§8, erreur n° 3).

Exercice 5.5. Écrire puis relire :

```
with open("trois.csv", "w", encoding="utf-8") as f:
    f.write("a,b\n1,2\n3,4\n5,6\n")
lignes = open("trois.csv", encoding="utf-8").readlines()
print(len(lignes) - 1)
```

3

Le `- 1` retire la **tête** — la ligne de colonnes n'est pas une ligne de données (la même question que l'Étape 6 de §7 : la 1^{re} ligne lue est la tête, pas une vente).

13. Mini-projet M08.P5 — « Le registre de caisse » (1 h)

Énoncé. Écrire `registre.py` — le fil rouge complet, du nombre au fichier :

- la fonction `nettoyer_montant` (celle de §6, docstring comprise) **et** la fonction `fmt(v)` qui forme un float au **format français** (500 → « 500,00 », 15000.5 → « 15 000,50 ») ;
- écrire `registre.csv` avec `csv.writer` (sous `with`, encoding `utf-8`) : la tête `n,total`, puis les 3 ventes du C03 (500, 1200, 15000.5) formatées en français ;
- relire `registre.csv` avec `csv.reader` (sous `with`, encoding `utf-8-sig` — la règle du chapitre, même pour un fichier qu'on vient d'écrire) ;
- pour chaque ligne : `nettoyer_montant` sur la saisie, affichage `vente n : saisie -> montant` ;
- le **total général** affiché en dernière ligne.

Critères de réussite (grille /10) : les 5 exigences (5 × 1 pt) · la sortie de l'atelier : `vente 1 : 500,00 -> 500.0`, `vente 2 : 1 200,00 -> 1200.0`, `vente 3 : 15 000,50 -> 15000.5`, `total : 16700.5` (1 pt) · deux fonctions nommées + docstring sur `nettoyer_montant` (1 pt) · le `csv` de la bibliothèque standard (pas de recopie manuelle du texte) (1 pt) · la lecture en `utf-8-sig` (1 pt).

Note de correction. Le total 16 700,50 (500 + 1 200 + 15 000,50) est **mesuré** dans l'atelier — et c'est le point du projet : le fichier contient du **texte** au format français, le script le convertit à **la porte** (C02), et le total s'additionne en **nombres**. La boucle est bouclée : C02 (la conversion), C03 (le flux), C05 (l'organisation) — le même 15 000,50 a fait trois voyages.

14. Boîte à outils du chapitre

Boîte à outils. Python nu + bibliothèque standard (`csv`, `json`) ; la boîte du chapitre s'ajoute à celles de C02-C04 : la fonction, le filet, le tiroir.

Outil	Usage	Syntaxe
<code>def / return</code>	la fonction et sa sortie	<code>def f(x): return x * 2</code>
argument nommé / défaut	la lecture et le contrat	<code>f(montant=1500, taux=0.05)</code>
<code>None</code> en défaut	le piège mutable évité	<code>def f(x, acc=None):</code>
docstring / <code>help</code>	la notice	<code>"""Quoi. Entrées. Sortie. Exemple."""</code>
<code>try/except</code>	le filet nommé	<code>try: ... except ValueError: ...</code>
<code>with open</code>	le tiroir qui se referme	<code>with open(f, encoding="utf-8-sig") as fh:</code>
<code>csv.reader/writer</code>	le CSV	<code>next(lecteur)</code> (la tête)
<code>json.load/dump</code>	le JSON	<code>json.load(fh) / json.dump(d, fh)</code>

15. Résumé du chapitre

- La **fonction** rend avec `return` — le `print` est un bruit, et la fonction sans `return` rend `None` (le piège exécuté : 1350.0 affiché, `None` rendu).
- Les arguments : positionnels, **nommés** (la lecture), **par défaut** — et jamais de mutable en défaut (`panier=None`, `reconstruction` ; le bug 2 2 exécuté).
- La **portée** : locale (meurt avec la fonction) vs globale (`global` à éviter) ; la variable de boucle **fuit** hors de la boucle (2000 exécuté).
- La **docstring** (Quoi / Entrées / Sortie / Exemple) est le **contrat** ; `help(f)` la lit — `taux=0.1` est le défaut affiché, le sens reste dans la docstring.
- Les **imports** en tête ; le `try/except` **nommé** et proche ; l'`except` nu est le bug qui se tait (15 000,50 → `None` sans un mot, exécuté).
- Les **fichiers** : `with open(..., encoding="utf-8-sig")` (le BOM corrompt la 1^{re} colonne en `utf-8` pur — exécuté), `csv` sur le vrai `vente.csv` (13 colonnes), `json.load` sur le vrai `ATTENDU.json` (30 clés).

16. À retenir

À retenir. La fonction a un **contrat** : le nom (quoi), la docstring (entrées/sortie/exemple), le `return` (la promesse tenue). Tout le reste — le `print` de chantier, le `global` caché, la mutable en défaut — est une brèche dans le contrat.

À retenir. L'exception est **nommée** (`except ValueError`), **proche** (sous la ligne qui lève) et **prévue** (un fait de métier, pas un accident) — l'`except` nu ne « gère » rien, il **met au silence** : le script marche, les résultats mentent, personne n'est prévenu.

17. Évaluation formative (auto-correction, 8 min)

1. `r = affiche_remise(1500)` où la fonction fait `print(montant * 0.9)` : que vaut `r`, et pour quoi ? → `None` : la fonction n'a pas de `return` — le `print` est un bruit, pas une sortie. (1 pt)
2. `calculer_remise(montant=1500, taux=0.05)` — que fait l'argument nommé que ne fait pas l'argument positionnel ? → il **nomme** l'argument : l'ordre est libéré, la lecture est gagnée (on voit **ce** qu'on modifie). (1 pt)
3. Pourquoi `def f(x, acc=[])` est-il un piège, et quelle est la correction mécanique ? → la mutable est construite **une fois** (au `def`) et **partagée** entre les appels (le bug 2 2) ; correction : `acc=None`, puis `reconstruction` à l'intérieur (1 1). (2 pts)
4. `global` fait quoi, et pourquoi est-il à éviter ? → il autorise la fonction à **écrire** une variable globale : ça marche, mais la fonction ne se teste plus seule (elle dépend de l'extérieur) — la version saine rend le compteur par `return`. (1 pt)
5. À quoi sert la docstring, et qui la lit ? → elle écrit le **contrat** (Quoi / Entrées / Sortie / Exemple) ; `help(f)` la lit, et elle est la première ligne de la fonction. (1 pt)
6. `try/except ValueError` et `try/except:` — la différence, avec l'exemple du chapitre ? → le premier intercepte le **type nommé** (le montant illisible) ; le second avale **tout**, y compris le bug du format français non géré — 15 000,50 → `None` sans un mot. (2 pts)
7. Sur une `traceback` à 2 cadres, où est l'erreur et où est le témoin ? → l'erreur vit dans le **cadre du bas** (la fonction, ici `calculer_net`) ; le cadre du haut est l'**appel** (le témoin) ; le message, tout en bas, donne la famille (`NameError`). (1 pt)

8. **Pourquoi** `open(..., encoding="utf-8-sig")` **et jamais** `utf-8` **pur** ? → le BOM (3 octets invisibles, fichiers Excel) est **avalé** par `utf-8-sig` mais **collé** devant la 1^{re} clé en `utf-8` (`'\ufeff-nom'`) : une clé qui n'existe pas. (1 pt)

Module M08.C06 — NumPy utile et suffisant : tableaux, vectorisation, statistiques, `nan`, `where`, broadcasting

Outils : le venv de C01 (`/tmp/venv_m08`) avec `numpy 2.5.3` (`pandas 3.0.6` est installé, il jouera en C07). Le terminal. **Durée indicative :** 4 h. **Niveau :** N6 → N7. **Prérequis :** C05 (les fonctions, les fichiers).

L'idée du chapitre. C05 a organisé le code ; C06 accélère le calcul. La boucle de C03 (le contrôle qualité montant par montant) devient **une ligne** quand les montants vivent dans un **tableau NumPy** : homogène, contigu, traité **en bloc** (la vectorisation). Le fil rouge : les **50 008 montants réels** du socle (`vente.csv`, colonne `montant_ttc`) — CA total, panier moyen, 95^e percentile, montants « gros » : **4 calculs, 0 boucle** — et la mesure honnête : la boucle de C03 face à la ligne vectorisée, **les temps publiés** (mesurés dans l'atelier, pas annoncés). C07 passera le relais au `DataFrame` : une colonne pandas est une colonne, pas un tableau nu.

Matériel de l'atelier — venv C01, numpy 2.5.3. Tout le code de ce chapitre a été exécuté le 19/09/2026 dans `/tmp/venv_m08` ; les sorties publiées sont **celles de l'atelier**. La colonne mesurée est **réelle** : `montant_ttc` des 50 008 lignes de `03_exercices/dossier_M08/vente.csv` (clé `m08p_ventes_lignes`), extraite en une commande `cut` avant le `np.loadtxt`. Les repères cano- niques : CA brut 7 908 259 732 (clé `m08p_ca_brut`), panier moyen 158 140 (clé `m08p_pa- nier_moyen`).

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Construire un **tableau** NumPy (`np.array`), lire son `dtype` et son `shape` — et comprendre l'homogénéité **forcée** (le tableau qui convertit sans prévenir).
2. **Vecteuriser** : `montants * 1.19` multiplie **tout le tableau** ; `np.sum`, `np.mean` sur les 50 008 montants réels — le parallèle SUM/AVG de M06.
3. Extraire les **statistiques** : `np.min/np.max`, les **percentiles** (le quartile de M02 en NumPy), la médiane, l'écart-type.
4. Gérer `np.nan` : le nombre qui « n'a pas de valeur » — `nan + 1 = nan` (l'absence se propage), `np.nansum/np.nanmean` — le pont vers les NULL de M06.
5. Étiqueter un tableau avec `np.where` (le CASE WHEN de M06 en tableau) et compter avec `(condition).sum()`.
6. Comprendre le **broadcasting** : la règle de forme — et lire le `ValueError` de forme qui dit laquelle des deux tables ne s'aligne pas.
7. **Mesurer** : boucle `for` vs ligne vectorisée — les deux temps, le rapport, publiés.

2. Pourquoi cette notion est importante

Parce que la vectorisation, c'est **le SGBD pour vos tableaux**. En M06/M07, le moteur SQL faisait l'agrégation sur le lot (`SUM`, `AVG`, `GROUP BY`) sans que vous écriviez de boucle ; NumPy fait **exactement le même geste** sur une colonne en mémoire : une opération, tout le tableau. Le chapitre ne vous apprend pas un outil en plus, il vous rend le réflexe SQL dans la main Python : le total d'une colonne est **une ligne**, point.

Parce que **nan est le NULL de M06, habillé en nombre**. Vous savez déjà ce qu’est une valeur manquante (M06 : elle se propage dans les calculs, elle ne compte pas dans les `COUNT`). NumPy réinstalle la **même** sémantique : `nan + 1 = nan`, un tableau qui contient un `nan` donne un `nan` à `np.sum` — et `np.nansum` est le `COUNT/SUM` qui sait regarder autour du trou. Le pont est direct : ce que vous avez appris sur les `NULL` en SQL, c’est déjà 80 % de `nan`.

Parce que la **mesure** fait partie du métier : « la boucle est lente » est une affirmation ; « la boucle prend 6,2 ms, la ligne vectorisée 0,193 ms, rapport 32× — mesuré le 19/09/2026 sur cette machine » est une **preuve** (et un résultat qui dépend de la machine, qu’on publie comme tel).

3. Explication simple — le convoi, la barrière, le camion vide

- Le **tableau NumPy** est un **convoi** : des camions **identiques** (un seul `dtype`) qui avancent **côte à côte**, sans espace entre eux (contigu). La liste de Python, elle, est un parking : des cases qui pointent vers des objets de tailles et de types différents.
- La **vecteurisation** est la **barrière de péage ouverte** : au lieu de faire passer le convoi camion par camion (la boucle de C03), la barrière est levée et **tout le convoi passe d’un coup** (`montants * 1.19`).
- **nan** est le **camion sans marchandise** : il passe la barrière, mais sa « valeur » est une absence — et l’absence **contagieuse** : une addition qui le touche devient une absence.
- `np.where` est le **contrôle qui étiquette** le convoi : chaque camion reçoit un autocollant (« gros » / « normal ») selon une règle — le `CASE WHEN` de M06.
- Le **broadcasting** est la **règle d’alignement** : on ne peut soustraire qu’un convoi d’un autre convoi **de même forme** — sinon la machine s’arrête et dit les deux formes qui ne vont pas ensemble.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
tableau — array	un bloc contigu de valeurs toutes du même type (<code>dtype</code>), opéré en bloc.	croire qu’on peut mélanger <code>int</code> et <code>str</code> : le tableau convertit sans prévenir (§5.1).
dtype — data type	le type unique du tableau : <code>float64</code> , <code>int64</code> , <code><U21</code> (texte).	lire un tableau sans demander <code>dtype</code> : c’est là que se cache la conversion forcée.
vecteurisation vectorization	— appliquer une opération à tout le tableau d’un coup , sans boucle Python.	réécrire en <code>for</code> ce que <code>np.sum</code> fait en une ligne (le §5.7 mesure l’écart).
nan — nan	le nombre « non numérique » : l’absence qui se propage (<code>nan + 1 = nan</code>).	laisser un <code>nan</code> dans <code>np.sum</code> : le total devient <code>nan</code> — <code>np.nansum</code> regarde autour.
diffusion broadcasting	— la règle de forme qui autorise les opérations entre tableaux de formes différentes compatibles .	aligner <code>(50008,)</code> sur <code>(5,)</code> : les formes ne se compatibilisent pas — le <code>ValueError</code> dit lesquelles.

5. Cours approfondi

5.1 Le tableau — le convoi homogène

```
import numpy as np

m = np.array([1500, 2000, 1350])
print(m.dtype, m.shape)
melange = np.array([1, "a"])
print(melange, melange.dtype)
```

```
int64 (3,)
['1' 'a'] <U21
```

Définition. tableau — ndarray — un bloc de mémoire **contigu** de valeurs **homogènes** : un seul dtype pour tout le tableau, une forme (shape) qui dit combien de valeurs. L'homogénéité est ce qui permet l'opération en bloc — le parking de la liste n'a pas ce droit.

La 2^e sortie est le piège du chapitre : `np.array([1, "a"])` **ne proteste pas** — le tableau a **converti** le 1 en texte '1' (le dtype `<U21` = texte de 21 caractères max). Pas d'erreur, une **décision silencieuse** : quand un tableau mélange des familles, NumPy choisit — et son choix est le plus **large** (le texte englobe), pas le plus juste.

Attention. Le tableau **force** l'homogénéité sans prévenir : `np.array([1, "a"])` donne `['1' 'a']`, pas une erreur. Demandez **toujours** le dtype avant de calculer — c'est la première ligne de lecture d'un tableau, comme le `SELECT *` limit est la première ligne de lecture d'une table.

5.2 La vecteurisation — la barrière levée

La colonne réelle du socle (extraite par `cut`, chargée par `np.loadtxt` — la commande complète est l'Étape 1 de §7) : 50 008 montants, `float64`.

```
np.sum(montants)      # le CA total
np.mean(montants)     # le panier moyen
```

```
7908259732.2
158139.8922612382
```

Définition. vecteurisation — vectorization — l'application d'une opération à **tout** un tableau d'un seul geste, sans boucle Python : `montants * 1.19` multiplie les 50 008 valeurs, `np.sum(montants)` les additionne. C'est le `SUM(colonne)` de M06, écrit en NumPy.

Dans les faits. Ces deux sorties sont **les valeurs canoniques du socle**, retrouvées par une autre porte : `np.sum` renvoie 7 908 259 732,20 (le CA brut `m08p_ca_brut` = 7 908 259 732, à 0,20 FCFA près — la dérive `float` que C08 documente en centimes) et `np.mean` renvoie 158 139,89 ≈ **158 140**, le panier moyen canonique (`m08p_panier_moyen`). Deux moteurs (DuckDB en M07, NumPy ici), le même nombre — c'est le parallèle qui tient.

Le parallèle SQL, complet :

NumPy (§5.2)	SQL (M06)	Ici
<code>np.sum(montants)</code>	<code>SUM(montant_ttc)</code>	7 908 259 732,20

NumPy (§5.2)	SQL (M06)	Ici
<code>np.mean(montants)</code>	<code>AVG(montant_ttc)</code>	158 139,89
<code>np.max(montants)</code>	<code>MAX(montant_ttc)</code>	594 363 (max canonique du socle)
<code>len(montants)</code>	<code>COUNT(*)</code>	50 008 (m08p_ventes_lignes)

5.3 Les statistiques — min, max, percentiles, médiane

```
np.min(montants)
np.max(montants)
np.percentile(montants, [25, 50, 75])
np.median(montants)
np.std(montants)
```

```
762.63
594363.0
[ 49134.2 119481.89 235875.33]
119481.89
133552.3926959041
```

- `np.min/np.max` : la 1^{re} et la dernière ligne du tri — 762,63 FCFA (la vente la plus petite du socle) et 594 363 FCFA (la plus grosse) ;
- `np.percentile(m, [25, 50, 75])` : les **quartiles** — le Q1 de M02 en NumPy : 25 % des ventes sont sous 49 134,20 FCFA, la moitié sous 119 481,89 ;
- `np.median(m)` **est** le percentile 50 : 119 481,89 (regardez — les deux fonctions redonnent le même nombre, c'est un contrôle) ;
- `np.std` : l'écart-type — 133 552,39 : les ventes sont **très** dispersées (l'écart-type dépasse la moyenne : il y a de très petites et de très grosses ventes dans le socle).

Le percentile est l'outil de la **question métier de queue** : « le 95^e percentile des montants, c'est combien ? » → 431 647,88 FCFA (mesuré, §7 Étape 3) — 95 % des ventes sont en dessous, les 5 % du haut au-dessus. C'est la version « position dans la file » du `MAX`.

5.4 nan — le camion sans marchandise

```
x = np.array([1500.0, np.nan, 2000.0])
print(x + 1)
print(np.sum(x))
print(np.nansum(x))
print(np.isnan(x))
```

```
[1501.    nan 2001.]
nan
3500.0
[False  True False]
```

Définition. nan — nan (« not a number ») — la valeur **absente** du monde des nombres : elle **se propage** (`nan + 1 = nan` — l'absence contagieuse), elle **contamine** les agrégats (`np.sum` renvoie `nan`), et elle se **détecte** (`np.isnan`) ou se **contourne** (`np.nansum` = 3500,0 : la somme regarde autour du trou).

C'est **exactement** la sémantique `NULL` de M06 : une addition qui touche un `NULL` donne `NULL` (pas 0, pas l'autre opérand) ; `COUNT(colonne)` ne compte pas les `NULL` (mais `COUNT(*)` les compte) ; `SUM` les ignore. `nan/nansum/isnan` sont les mêmes trois gestes, habillés en nombres — le pont M06 → NumPy tient sur trois lignes.

Attention. Un `nan` dans le tableau **contamine** `np.sum` — le total devient `nan`, sans erreur. Sur une colonne qui **peut** être vide (les montants du socle peuvent l'être — `est_retour`, les champs vides de M03), l'agrégat robuste est `np.nansum/np.nanmean`, pas `np.sum/np.mean`.

5.5 `np.where` — le contrôle qui étiquette

Le `CASE WHEN` de M06, en tableau — chaque montant reçoit son étiquette, d'un coup :

```
gros = np.where(montants > 100000, "gros", "normal")
print((gros == "gros").sum())
print((montants > 100000).sum())
```

```
28065
28065
```

```
SELECT CASE WHEN montant_ttc > 100000 THEN 'gros' ELSE 'normal' END
FROM vente;
```

Les deux lignes Python redonnent le même comptage (28 065 ventes « gros » — la majorité des 50 008 : dans ce socle, la grosse vente est la **norme**, pas l'exception). La 2^e ligne est l'idiome du chapitre : `montants > 100000` est un **tableau de True/False** (un `bool` par montant), et **bool a une sum** — elle compte les `True`. Pas besoin de `where` pour compter : la condition seule, suivie de `.sum()`.

5.6 Le broadcasting — la règle d'alignement

La question métier : **l'écart de chaque vente à la moyenne de son magasin**. Les moyennes par magasin (5 valeurs, mesurées) :

```
moyennes = np.array([157958.55, 157379.42, 158506.83, 158713.18, 158147.00])
montants - moyennes
```

```
ValueError : operands could not be broadcast together with shapes (50008,) (5,)
```

Définition. diffusion — broadcasting — la règle de forme de NumPy : deux tableaux peuvent s'opérer s'ils sont **de même forme**, ou si l'un est un **scalaire**, ou si leurs formes sont **compatibles** (du plus court au plus long, dimensions égales ou 1). `(50008,)` et `(5,)` ne sont compatibles ni dans un sens ni dans l'autre — d'où le `ValueError`, qui **cite les deux formes** (la lecture du message est déjà le diagnostic).

Le `ValueError` est la **règle qui parle** : 50 008 montants ne se soustraient pas à 5 moyennes — une moyenne ne se soustrait qu'à **son** magasin. L'alignement se fait **d'abord** (chaque montant connaît son magasin : `ids - 1` donne l'index de la moyenne à prendre), **ensuite** l'opération passe :

```
ecarts = montants - moyennes[ids - 1]
print(ecarts[:3])
print(np.mean(montants[ids == 1] - moyennes[0]))
```

```
[ 56186.78118011 201151.24731657 -71440.36881989]
-8.299735787245575e-12
```

La dernière sortie est le **contrôle** : l'écart moyen des ventes du magasin 1 à la moyenne du magasin 1 est ≈ 0 (-8×10^{-12} — le bruit float, pas un écart réel). Quand le contrôle redonne zéro, l'alignement est bon.

5.7 La mesure — la boucle de C03 face à la ligne

Mesurer la vecteurisation : boucle for vs une ligne

Mesuré le 19/09/2026 (atelier) — 50 008 montants réels du socle ; le rapport tient, l'absolu change de machine.

Mesure 1 — la somme	Mesure 2 — le contrôle qualité de C03
boucle for — montant par montant	boucle for — isinstance + test + compteur
 3,8 ms	 6,2 ms
np.sum(montants) — une ligne	vecteur — (m > 100000).sum()
 0,149 ms	 0,193 ms
rapport : 25×	rapport : 32×
même total des deux côtés :	même comptage des deux côtés :
7 908 259 732,20 FCFA	28 065 « gros »
(l'absolu est petit : 50 000 éléments ; le rapport, lui, est stable)	(la boucle fait plus de Python par élément → le rapport monte)

La leçon : la vecteurisation ne change PAS la réponse — elle change le chemin.

Publier une vitesse = les deux temps + le rapport + la date + la taille : un rapport non daté est une opinion, un rapport mesuré et daté est une preuve.

Deux mesures, exécutées dans l'atelier le 19/09/2026 (cette machine, ces 50 008 montants — les rapports varient d'une machine à l'autre, c'est **pourquoi** on publie la date et les deux temps) :

Mesure 1 — la somme (la boucle de C03, for + accumulation, vs np.sum) :

```
boucle for : 3.8 ms
np.sum      : 0.149 ms
rapport     : 25x
```

Mesure 2 — le contrôle qualité de C03 (for + isinstance + test + compteur, vs (m > 100000).sum()) :

```
def controle_boucle(valeurs, seuil=100000):
    gros = 0
    for v in valeurs:
        if isinstance(v, (int, float)) and v > seuil:
            gros = gros + 1
    return gros

# boucle : controle_boucle(m.tolist())          # 6.2 ms
# vecteur : (m > 100000).sum()                  # 0.193 ms
```

```
boucle : 28065 | 6.2 ms
vecteur: 28065 | 0.193 ms
rapport: 32x
```

Deux leçons : les **mêmes** résultats (28065 des deux côtés — la vecteurisation ne change pas la réponse, elle change le chemin), et un écart de **25 à 32×** sur 50 000 valeurs — un écart qui

croît quand la boucle fait plus de Python par élément (l'instance de la mesure 2 coûte plus cher qu'une addition), et qui serait encore plus grand sur 50 000 **lignes de table** avec des branchements métier. Sur une machine d'atelier, 6,2 ms ne se voit pas — mais le **rapport** est la leçon, pas l'absolu : ce qui est 32× plus lent sur 50 000 éléments est 32× plus lent sur 5 millions.

Conseil professionnel. Ne dites jamais « la boucle est lente » sans la **mesure** : publiez les deux temps, le rapport, la date, la taille — et souvenez-vous que le rapport, lui, est stable ; l'absolu, jamais. Un rapport de vitesse non mesuré est une opinion ; un rapport mesuré et daté est une preuve.

6. Exemple concret — les 4 calculs du fil rouge

La question du chapitre, en langage de comptoir : « sur la colonne `montant_ttc` réelle du socle — CA total, panier moyen, le 95^e percentile, et combien de ventes « grosses » ? » Le script — exécuté dans l'atelier le 19/09/2026 :

```
import numpy as np

m = np.loadtxt("colonne.csv", delimiter=",", skiprows=1)[: , 1]
print("CA total      :", np.sum(m))
print("Panier moyen  :", np.mean(m))
print("95e percentile :", np.percentile(m, 95))
print("Gros > 100 000 :", (m > 100000).sum())
```

```
CA total      : 7908259732.2
Panier moyen  : 158139.8922612382
95e percentile : 431647.88
Gros > 100 000 : 28065
```

Quatre calculs, **zéro boucle** : le CA total redonne le canonique du socle (7 908 259 732 à 0,20 près — `float`), le panier moyen redonne 158 140 (`m08p_panier_moyen`), le 95^e percentile (431 647,88 FCFA) est une **information nouvelle** — 95 % des ventes sont en dessous — et le comptage « gros » redonne 28 065 (la majorité : dans ce socle, la grosse vente est la norme). Le `np.loadtxt` avec `skiprows=1` saute la ligne de tête, et `[: , 1]` prend la **2^e colonne** (`montant_ttc`) — la 1^{re} (`id_magasin`) est chargée aussi, on en a besoin en §5.6.

7. Démonstration pas à pas — 6 étapes sur le fil rouge

Sortie réelle de chaque commande (exécutées le 19/09/2026, `/tmp/venv_m08`).

Étape 0 — Extraire la colonne (la `cut` du socle, avant le `loadtxt`) :

```
$ { echo "id_magasin,montant_ttc"; cut -d',' -f4,11 03_exercices/dossier_M08/vente.csv | tail -n +2; } > colonne.csv
```

Interprétation : 1 ligne de tête + les 50 008 montants (`m08p_ventes_lignes`) — la colonne `montant_ttc` est la 11^e du CSV (`-f11`), `id_magasin` la 4^e (`-f4`) ; le `tail -n +2` jette la tête d'origine, l'`echo` remet la sienne. Le contrôle effectif du comptage arrive à l'Étape 1, où le `shape` (50008,) le prouve.

Étape 1 — Charger en une ligne (`np.loadtxt`) :

```
$ python3 -c "
import numpy as np
d = np.loadtxt('colonne.csv', delimiter=',', skiprows=1)
m = d[:, 1]
print(m.shape, m.dtype)"
(50008,) float64
```

Interprétation : 50 008 valeurs, toutes float64 — le tableau s'est chargé en bloc (c'est le SELECT montant_ttc de M06, en une commande).

Étape 2 — Le CA et le panier (les deux agrégats canoniques) :

```
$ python3 -c "
import numpy as np
m = np.loadtxt('colonne.csv', delimiter=',', skiprows=1)[: , 1]
print(np.sum(m))
print(np.mean(m))"
7908259732.2
158139.8922612382
```

Interprétation : 7 908 259 732,20 et 158 139,89 — les valeurs canoniques du socle retrouvées par NumPy (le $\pm 0,20$ est la dérive float de C08).

Étape 3 — Le 95^e percentile (la position dans la file) :

```
$ python3 -c "
import numpy as np
m = np.loadtxt('colonne.csv', delimiter=',', skiprows=1)[: , 1]
print(np.percentile(m, 95))"
431647.88
```

Interprétation : 95 % des ventes sont sous 431 647,88 FCFA — les 5 % du haut au-dessus : c'est la « queue » de la distribution, en un nombre.

Étape 4 — Le comptage « gros » (la condition qui compte) :

```
$ python3 -c "
import numpy as np
m = np.loadtxt('colonne.csv', delimiter=',', skiprows=1)[: , 1]
print((m > 100000).sum())"
28065
```

Interprétation : `m > 100000` est un tableau de True/False ; `.sum()` compte les True — la moitié et plus du socle est « grosse » (le seuil 100 000 FCFA est une valeur du socle).

Étape 5 — Le nan qui contourne (l'absence qui ne contamine pas) :

```
$ python3 -c "
import numpy as np
x = np.array([1500.0, np.nan, 2000.0])
print(np.sum(x), np.nansum(x))"
nan 3500.0
```

Interprétation : le même tableau, deux sommes — `np.sum` est contaminé (nan), `np.nansum` regarde autour du trou (3 500) : c'est le SUM qui ignore les NULL de M06.

Étape 6 — Le temps mesuré (la boucle face à la ligne) :

```
$ python3 -c "
import time
import numpy as np
m = np.loadtxt('colonne.csv', delimiter=',', skiprows=1)[: , 1]
l = m.tolist()
t0 = time.perf_counter()
s = 0.0
for v in l:
    s = s + v
t1 = time.perf_counter()
t2 = time.perf_counter()
s2 = np.sum(m)
t3 = time.perf_counter()
print(f'boucle for : {(t1 - t0) * 1000:.1f} ms')
print(f'np.sum      : {(t3 - t2) * 1000:.3f} ms')
print(f'rapport    : {(t1 - t0) / (t3 - t2):.0f}x')"
```

boucle for : 3.8 ms
np.sum : 0.149 ms
rapport : 25x

Interprétation : 3,8 ms contre 0,149 ms — 25×, mesuré sur cette machine, ces 50 008 montants, le 19/09/2026 : le rapport est la leçon (il tient), l'absolu est l'atelier (il change).

8. Erreurs fréquentes

1. **Le tableau qui convertit sans prévenir.** `np.array([1, "a"]) → ['1' 'a'] <U21` (sortie réelle, §5.1) : le nombre est devenu du **texte**, sans erreur. Remède : demander `dtype` **avant** de calculer — si le `dtype` est du texte (`<U...`) et que vous vouliez des nombres, la conversion s'est faite au mauvais moment (et au mauvais sens).
2. **Le nan qui contamine le total.** `np.sum` sur un tableau contenant `np.nan` renvoie `nan` (sortie réelle, §5.4) — pas d'erreur, un total **inutilisable**. Remède : `np.nansum/np.nanmean` dès que la colonne **peut** être vide ; et `np.isnan` pour **détecter** avant de calculer.
3. **Les formes qui ne s'alignent pas.** `montants - moyennes → ValueError : operands could not be broadcast together with shapes (50008,) (5,)` (sortie réelle, §5.6). Remède : le message **cite les deux formes** — lisez-les ; l'alignement se fait **avant** l'opération (`moyennes[ids - 1]`), pas en espérant que NumPy devine.
4. **La condition sans le compteur.** `m > 100000` **seul** affiche un tableau de `True/False` (50 008 lignes) — il ne **compte** rien. Remède : `(m > 100000).sum()` — un tableau `bool` **a** une `sum` (elle compte les `True`) ; c'est l'idiome du chapitre (§5.5).
5. **np.loadtxt sur un CSV qui a des mots.** Charger `vente.csv` **en entier** (13 colonnes, dont `date_vente` en texte) par `np.loadtxt` lève `ValueError: could not convert string 'id_vente' to float64 at row 0, column 1.` (sortie réelle) : le tableau est **numérique**, et la première ligne n'en est pas une — c'est la **tête** (`id_vente` n'est pas un nombre) ; passé `skiprows=1`, ce seraient les dates qui poseraient le même problème. Remède : extraire les colonnes **numériques** d'abord (`cut`, §7 Étape 0) — ou attendre C07, où `pandas.read_csv` lit le mélange (c'est son métier).
6. **Le panier moyen « sans poser la question ».** `np.mean(montants)` sur les 50 008 lignes renvoie 158 139,89 — **mais** « quelles 50 008 lignes ? » (les doublons du socle ? les retours ?) est une **question de données**, pas de calcul : changer le jeu de lignes change le nombre (les paniers 158 140 / 158 159 du socle sont des **définitions différentes**, pas des arrondis). Remède : avant d'agréger, dire **quelles lignes** on additionne — la question est un cinquième du travail, avant les quatre calculs.

9. Bonnes pratiques professionnelles

1. **Un dtype par tableau, vérifié en premier** : la lecture d'un tableau commence par `print(m.dtype, m.shape)` — comme le `head + dtypes` de C07.
2. **Compter avec `(condition).sum()`** : pas de boucle pour compter, pas de `where` pour compter — la condition booléenne **est** le compteur.
3. **`nansum/nanmean` par défaut** sur toute colonne qui **peut** être vide : le coût est zéro, le `nan` contaminant coûte un rapport à refaire.
4. **Vecteuriser le calcul, garder la boucle pour l'exception** : le calcul en bloc, la logique métier qui **branche** (`isinstance`, les cas « N/C ») reste une boucle — mais sur **peu** de lignes (les exceptions), pas sur toutes.
5. **Mesurer avant d'affirmer** : les deux temps, le rapport, la date, la taille — publiés ensemble (§5.7) ; un rapport de vitesse non daté est une opinion.

10. Exercice guidé — « les 4 statistiques du comptoir » (15 min, /10)

Énoncé. Écrire `statistiques.py` qui, sur la colonne réelle (`colonne.csv`, `np.loadtxt` avec `skiprows=1`) :

1. charge les montants (la 2^e colonne) ;
2. affiche le **minimum** et le **maximum** (2 décimales) ;
3. affiche la **médiane** et le **90^e percentile** (2 décimales) ;
4. affiche le **nombre de ventes** (la longueur du tableau).

Barème. Le chargement correct (2 pts) · les 4 statistiques avec les fonctions du chapitre (`min/max/median/percentile` — pas de tri manuel) (4 pts) · l'affichage 2 décimales en f-string (2 pts) · la sortie alignée sur le modèle ci-dessous (2 pts). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```
Minimum : 762.63
Maximum : 594363.00
Mediane : 119481.89
90e percentile : 357969.75
Ventes : 50008
```

11. Exercices autonomes

Exercice 6.1 — Le convoi. Construire le tableau `np.array([500, 1500, 8000, 1350, 4500])`, l'afficher, afficher son `dtype` — puis le tableau multiplié par 1.19 (TVA) et sa somme. (5 min)

Exercice 6.2 — Le camion vide. Construire `np.array([1500.0, np.nan, 2000.0, np.nan])` : afficher `np.sum`, `np.nansum`, `np.isnan` — et dire en une phrase pourquoi les deux sommes diffèrent. (5 min)

Exercice 6.3 — Le contrôle. Sur la colonne réelle, étiqueter `gros/normal` avec `np.where` (seuil 100 000), et compter les `gros` **deux façons** (`(gros == "gros").sum()` et `(m > 100000).sum()`) : les deux doivent redonner le même nombre. (10 min)

Exercice 6.4 — Les quartiles. Sur `np.array([3500, 2500, 4500, 1500, 2000])` : la médiane, les quartiles (`percentile(..., [25, 75])`), et vérifier que la médiane est bien le percentile 50. (10 min)

Exercice 6.5 — La mesure. Sur une liste de 10 000 nombres (par exemple `list(range(10000))` convertie en tableau) : mesurer la somme en **boucle** (`for`) et avec `np.sum` — afficher les deux temps **et** le rapport. (10 min)

12. Correction détaillée

Exercice guidé. Le script :

```
import numpy as np

m = np.loadtxt("colonne.csv", delimiter=",", skiprows=1)[: , 1]
print(f"Minimum : {np.min(m):.2f}")
print(f"Maximum : {np.max(m):.2f}")
print(f"Mediane : {np.median(m):.2f}")
print(f"90e percentile : {np.percentile(m, 90):.2f}")
print(f"Ventes : {len(m)}")
```

Les 5 lignes publiées **sont** la sortie de l'atelier (762.63 / 594363.00 / 119481.89 / 357969.75 / 50008) — le contrôle à faire en tête : `np.median` doit redonner `np.percentile(m, 50)` (les deux sont le 50^e).

Exercice 6.1. `int64` (des entiers → le dtype entier, pas float) ; le tableau $\times 1.19$ passe en `float64` (l'opération élargit — $500 \times 1.19 = 595,0$) ; la somme : $15\,850 \times 1.19 = 18\,861,50$ — exécuter pour vérifier (`[595. 1785. 9520. 1606.5 5355.]`, sortie réelle ; le calcul mental est le piège du chapitre : la machine calcule, on ne devine pas).

Exercice 6.2. `np.sum` → `nan` (contaminé), `np.nansum` → 3500,0 (regarde autour), `np.isnan` → `[False True False True]` (deux trous). La phrase : « `np.sum` **propage** l'absence, `np.nansum` **l'ignore** — c'est le SUM de M06 qui ignore les NULL ».

Exercice 6.3. Les deux comptages redonnent **28065** — la `where` étiquette (une colonne de texte), la condition compte directement (un tableau `bool`) : même réponse, deux chemins (§5.5).

Exercice 6.4. Trié : [1500, 2000, 2500, 3500, 4500] — médiane 2500, `percentile([25, 75])` → 2000,0 / 3500,0 (sortie réelle : pour $n = 5$, les quartiles tombent **juste** sur des valeurs du tableau — l'interpolation se voit avec d'autres tailles, la question reste « entre quelle valeur et quelle valeur ? ») ; `percentile(..., 50)` redonne 2500,0 : le contrôle passe.

Exercice 6.5. Les deux temps dépendent de **votre** machine — c'est pour ça qu'on publie les deux **et** le rapport : la boucle est typiquement 10 à 50× plus lente que `np.sum` sur 10 000 éléments ; le rapport, lui, est la leçon stable.

13. Mini-projet M08.P6 — « Le rapport de colonne » (1 h)

Énoncé. Écrire `rapport.py` — le fil rouge, durci en **rapport** :

1. charger la colonne réelle (`colonne.csv`, `np.loadtxt`, `skiprows=1`, 2^e colonne) ;
2. afficher (2 décimales, f-string) : CA total, panier moyen, minimum, maximum ;
3. afficher les **quartiles** Q1 / médiane / Q3 sur une ligne ;
4. afficher le 95^e percentile ;
5. afficher le comptage « gros » (seuil 100 000) **sur le total** (« X ventes sur Y ») ;
6. la **robustesse** : ajouter un `nan` en fin de tableau (`np.append`), afficher `np.sum` (contaminé) **et** `np.nansum` (intact) sur la même ligne.

Critères de réussite (grille /10) : les 6 exigences (6×1 pt) · la sortie de l'atelier : CA total 7,908,259,732.20 · panier moyen 158,139.89 · minimum 762.63 · maximum 594,363.00 · quartiles 49,134.20 / 119,481.89 / 235,875.33 · 95^e 431,647.88 · 28065 ventes sur 50008 · `nan` / 7,908,259,732.20 (1 pt) · aucun `for` dans le calcul (1 pt) · le dtype vérifié en tête (1 pt).

Note de correction. Le `np.nansum` de la ligne 6 redonne **le même** CA que la ligne 2 (7,908,259,732.20) : le `nan` ajouté ne pèse rien — c'est la **preuve** que `nansum` « regarde autour » (pas `0 + nan`, pas un total différent). Si votre `nansum` diffère de votre `sum`, le `nan` n'est pas là où vous croyez.

14. Boîte à outils du chapitre

Boîte à outils. Le venv de C01 (`/tmp/venv_m08`) + numpy 2.5.3 ; la boîte du chapitre s'ajoute à celles de C02-C05 : le convoi et ses barrières.

Outil	Usage	Syntaxe
<code>np.array / dtype / shape</code>	le convoi et son type	<code>np.array([1, 2]).dtype</code>
<code>np.loadtxt</code>	charger une colonne numérique	<code>np.loadtxt(f, delimiter=",", skiprows=1)</code>
<code>np.sum / np.mean</code>	l'agrégat en bloc	<code>np.sum(m)</code>
<code>np.min / np.max / np.percentile</code>	les bornes et les positions	<code>np.percentile(m, [25, 50, 75])</code>
<code>np.nan / np.isnan / np.nansum</code>	l'absence qui se gère	<code>np.nansum(m)</code>
<code>np.where</code>	l'étiquette en bloc	<code>np.where(m > s, "gros", "normal")</code>
<code>(cond).sum()</code>	le compteur booléen	<code>(m > 100000).sum()</code>
broadcasting	la règle d'alignement	<code>m - moyennes[ids - 1]</code>

15. Résumé du chapitre

- Le **tableau** NumPy est homogène (`dtype` unique) et contigu ; il **convertit sans prévenir** (`[1, "a"] → ['1' 'a'] <U21`) — demander le `dtype` en premier.
- La **vecteurisation** est le SUM/AVG de M06 en mémoire : `np.sum` sur les 50 008 montants réels renvoie 7 908 259 732,20 (le canonique, à la dérive `float` près), `np.mean` redonne le panier moyen 158 140.
- Les **statistiques** : min/max (762,63 / 594 363), les quartiles (`percentile(..., [25, 50, 75])`), la médiane (le 50^e), le 95^e percentile (431 647,88) — la « position dans la file » en un nombre.
- **nan** est le NULL de M06 : `nan + 1 = nan`, `np.sum` contaminé, `np.nansum` regarde autour (3500,0), `np.isnan` détecte.
- **np.where** étiquette en bloc (le CASE WHEN), et `(m > s).sum()` **compte** (le tableau `bool` a une `sum`) — 28 065 ventes « gros ».
- Le **broadcasting** : `(50008,) - (5,)` lève `ValueError` (les formes citées) ; l'alignement **d'abord** (`moyennes[ids - 1]`), l'opération **ensuite** — le contrôle (écart moyen ≈ 0) valide l'alignement.
- La **mesure** : boucle 3,8 ms vs `np.sum` 0,149 ms (**25×**) ; boucle métier 6,2 ms vs vecteur 0,193 ms (**32×**) — mêmes résultats, daté, sur cette machine.

16. À retenir

À retenir. Le calcul en bloc est **une ligne** : `np.sum`, `np.mean`, `(m > s).sum()` — la boucle de C03 est un souvenir, pas un réflexe. Et quand un résultat « ne colle pas », la première question n'est pas « quel calcul ? » mais « **quelles lignes ?** » (§8, erreur n° 6) : le nombre est la définition, pas l'inverse.

À retenir. `nan` est le NULL habillé en nombre : il **propage** (`nan + 1 = nan`), il **contamine** (`np.sum` → `nan`), il se **contourne** (`np.nansum`) et se **détecte** (`np.isnan`) — les quatre gestes du NULL de M06, sans rien réapprendre.

17. Évaluation formative (auto-correction, 8 min)

1. `np.array([1, "a"])` renvoie quoi, et quel est le danger ? → `['1' 'a']` en dtype `<U21` : le nombre a été **converti en texte sans erreur** — le tableau force l'homogénéité silencieusement ; d'où le dtype vérifié en premier. (1 pt)
2. `np.sum(montants)` sur les 50 008 montants du socle renvoie quoi, et qu'est-ce que ça redonne ? → 7 908 259 732,20 — le CA brut canonique (`m08p_ca_brut = 7 908 259 732`), à 0,20 près (la dérive `float` documentée en C08). (1 pt)
3. Pourquoi `np.sum` renvoie `nan` sur un tableau contenant `np.nan`, et quelle est la version robuste ? → `nan` **propage** (l'absence contagieuse) ; la version robuste est `np.nansum` (regarde autour du trou — le SUM qui ignore les NULL de M06). (2 pts)
4. `np.where(m > 100000, "gros", "normal")` fait quoi, et comment compter les « gros » sans `where` ? → il **étiquette** le tableau (une colonne de texte) ; pour compter : `(m > 100000).sum()` — le tableau `bool` a une `sum` (elle compte les `True`). (2 pts)
5. `montants - moyennes (50008, et 5)` lève quoi, et d'où vient le message ? → `ValueError` : `operands could not be broadcast together with shapes (50008,) (5,)` : les formes ne sont pas compatibles — le message **cite les deux formes** ; l'alignement se fait d'abord (`moyennes[ids - 1]`). (1 pt)
6. Le **contrôle d'alignement** de §5.6 redonne quoi, et pourquoi est-ce le contrôle ? → l'écart moyen des ventes du magasin 1 à sa moyenne ≈ 0 (-8×10^{-12} , le bruit `float`) : si le contrôle ne redonne pas zéro, l'alignement est mauvais. (1 pt)
7. `np.percentile(m, 50)` et `np.median(m)` — quelle relation, et pourquoi est-ce un contrôle ? → ce sont **le même nombre** (le 50^e percentile est la médiane) : les deux fonctions doivent redonner le même résultat, sinon le chargement a trahi. (1 pt)
8. Pourquoi le rapport « boucle vs vecteur » est-il la leçon, et pas les temps absolus ? → l'absolu dépend de la machine (il change) ; le **rapport** est stable — et il croît quand la boucle fait plus de Python par élément ; d'où publier les deux temps + la date + la taille. (1 pt)

Module M08.C07 — pandas I : `Series`, `DataFrame`, `read_csv`/`read_excel`, `loc`/`iloc`, `filtres`, `nouvelles colonnes`, `describe`, `valeurs manquantes`

Outils : le `venv` de C01 (`/tmp/venv_m08`, pandas 3.0.6) + l'environnement d'écriture du manuel (pandas 2.2.3) pour le comparé du §5.10. Le dossier `03_exercices/dossier_M08/` (les 8 CSV du socle). **Durée indicative :** 5 h. **Niveau :** N6 → N7. **Prérequis :** C06 (le tableau NumPy est une colonne pandas sans en-tête).

L'idée du chapitre. C06 calculait sur **une colonne** en bloc ; C07 travaille sur **la table entière** — les 50 008 lignes × 13 colonnes réelles du socle — avec les deux objets de pandas : la **Série** (une colonne : index + valeurs) et le **DataFrame** (un tableau de colonnes nommées, la table SQL de M07 en Python). Le fil rouge : **les mêmes chiffres, trois chemins** — Excel (M05), SQL (M07), pandas (M08) : chaque bloc a son équivalent SQL côte à côte, et l'exercice du chapitre est l'**audit complet du dossier en 8 lignes** — formes, doublons, retours, dates inversées, table vide — chaque résultat **identique** au SQL de M07. C08 fera le reste : `groupby`, `merge`, `pivot_table`, dates, export Excel.

Matériel de l'atelier — deux pandas, une règle. Le `venv` du chapitre est **pandas 3.0.6** (l'installation fraîche de C01) ; l'environnement d'écriture du manuel est **figé sur 2.2.3**. Les deux co-existent, et le manuel enseigne les patterns **valides sur les deux** — C01 a posé la règle, le §5.10 la prouve avec **4 écarts mesurés** sur un fichier réel (103 → 85, 373 → 372, 18 → 0, 7 → 0), pas illustrés. Tout le code a été exécuté le 19/09/2026 ; les sorties publiées sont celles de l'atelier.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Importer le **dossier M08 complet** (les 8 CSV, formes réelles : 50 008 × 13, 1 200 × 5, ..., 0 × 4) avec `pd.read_csv` — `head`, `shape`, `dtypes`.
2. Lire un **DataFrame** comme on lit une table SQL : l'**index** (pas une colonne), les colonnes nommées, le `dtype` de chacune — `str` sous pandas 3 (plus d'object).
3. Sélectionner avec `loc` (par nom) et `iloc` (par position) — le tableau de 4 cas (ligne/colonne × nom/position) et le piège des tranches.
4. **Filtrer** comme en `WHERE` : le **masque** booléen, `&`/`|`/`~`, les **parenthèses obligatoires** — et le bug silencieux qu'elles évitent (mesuré : 50 008 au lieu de 27 938).
5. Créer une **nouvelle colonne** par opération vectorisée (`montant_ht = montant_ttc / (1 + taux_tva)`) — le `SELECT ... AS` de M07, et assign.
6. Lire `describe()` : les 8 statistiques de M02 en une commande — et le détail qui distingue pandas de NumPy (`std` échantillonnaire vs population).
7. Gérer les **valeurs manquantes** : `isna()`, `dropna()`, `fillna()`, `ffill()` — et le cas réel du socle : **zéro manquant mesuré**, et la table `objectif_magasin` **vide** (0 ligne, en-tête seul) et son `KeyError`.
8. Repérer les **4 écritures obsolètes** de la 1^{re} salve des 12 (`append`, `inplace`, affectation chaînée, `fillna(method=...)`) — exécutées, pas citées.

2. Pourquoi cette notion est importante

Parce que le **DataFrame est la table SQL de M07, transportée dans Python**. Vous savez déjà lire une table, un `WHERE`, un `SELECT ... AS`, un `GROUP BY` — pandas ne vous demande pas de repartir de zéro, il vous donne les **mêmes opérations** sur un objet en mémoire : `v[v["est_retour"]]` est le `WHERE est_retour = TRUE`, `v["montant_ttc"].sum()` est le `SUM(montant_ttc)`, `describe()` est le paquet `COUNT/AVG/STDDEV/MIN/MAX`. Le chapitre est donc une **traduction** — et la traduction est le but : le même chiffre sort de trois portes (Excel M05, SQL M07, pandas M08), et un jour vous choisirez la porte selon le contexte, pas selon l'habitude.

Parce que les **erreurs de pandas sont souvent silencieuses**. Une boucle Python proteste (C05) ; un filtre sans parenthèses, lui, **répond** — avec le mauvais résultat (le §5.5 le mesure : la table entière sélectionnée, aucun avertissement). La compétence du chapitre n'est pas « savoir écrire un filtre », c'est « savoir qu'un filtre peut avoir raison **sans qu'on le sache** » — d'où les contrôles de taille (`len`, `.sum()`) systématiques après chaque sélection.

Parce que **la version change la vérité terrain**. Sur le même fichier, pandas 2.2.3 et 3.0.6 ne comptent pas les mêmes cellules vides (103 contre 85 — mesuré, §5.10). « Ce script tourne mais ses chiffres sont faux » (l'évaluation E4 du module) n'est pas une figure de style : c'est ce qui arrive quand on branche son code sur le `dtype` au lieu de la **valeur**.

3. Explication simple — le classeur d'indexation

- Le **DataFrame** est un **classeur d'indexation** : des fiches (les lignes) rangées par dossiers (les colonnes **nommées**). Chaque dossier est exactement le **convoi NumPy de C06** — un type, un bloc, une opération en bloc — avec une étiquette (le nom de colonne) et une colonne de référence (l'**index**).
- L'**index** est la **réduire** du classeur : le numéro de fiche. Ce n'est pas une fiche, ce n'est pas une colonne — c'est la **poignée** par laquelle `loc` et `iloc` attrapent les lignes.
- Un **filtre** est le **tamis** : on pose la question sur **toutes les fiches** d'un coup (`v["est_retour"] = True/False` par fiche) et on ne garde que les fiches qui passent — le `WHERE` de M07, sans boucle.
- Une **nouvelle colonne** est un **nouveau dossier** rempli par une opération sur les valeurs existantes : `montant_ht = montant_ttc / (1 + taux_tva)` — chaque fiche reçoit sa valeur, en bloc.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
tableau données — DataFrame	de un tableau 2-D de colonnes nommées (une table SQL en Python), le cœur de pandas.	le parcourir ligne par ligne avec une boucle : c'est exactement ce qu'il faut éviter (§5.5, §5.6).
série — Series	une colonne : un index + des valeurs (le cousin 1-D du DataFrame, le tableau de C06 avec étiquettes).	confondre l' index (l'étiquette des lignes) et la première colonne : ce n'est pas la même chose (§5.3).
index — index	la réduire : l'étiquette de chaque ligne (par défaut 0, 1, 2, ...), utilisée par <code>loc/iloc</code> .	croire que c'est une colonne : <code>reset_index()</code> la transforme en colonne (§5.3) — preuve qu'elle n'en était pas une.
sélection par nom — <code>loc</code>	attraper lignes et/ou colonnes par leurs étiquettes (noms, bornes incluses).	<code>v.loc[1:3]</code> inclut la ligne 3 — 3 lignes, pas 2 (§5.4).
sélection par position — <code>iloc</code>	attraper lignes et/ou colonnes par leur numéro (0 = premier, borne haute exclue).	<code>v.iloc[1:3]</code> exclut la ligne 3 — 2 lignes, pas 3 (§5.4).

5. Cours approfondi

5.1 read_csv — l'import des 8 CSV réels

```
import pandas as pd

fichiers = ["vente", "client", "produit", "categorie", "magasin",
            "mode_paiement", "regle_tva", "objectif_magasin"]
for nom in fichiers:
    df = pd.read_csv(f"03_exercices/dossier_M08/{nom}.csv")
    print(f"{nom}: {df.shape[0]} lignes x {df.shape[1]} colonnes")
```

```
vente: 50008 lignes x 13 colonnes
client: 1200 lignes x 5 colonnes
produit: 380 lignes x 6 colonnes
categorie: 8 lignes x 3 colonnes
magasin: 5 lignes x 6 colonnes
mode_paiement: 5 lignes x 4 colonnes
regle_tva: 2 lignes x 2 colonnes
objectif_magasin: 0 lignes x 4 colonnes
```

Définition. tableau de données — DataFrame — un tableau à deux dimensions de **colonnes nommées** (une table SQL en Python), l'objet central de pandas : `shape` donne lignes × colonnes, `columns` les noms, `dtypes` la famille de chaque colonne.

Définition. série — Series — une **colonne** : un index (les étiquettes de lignes) + des valeurs. C'est le tableau NumPy de C06 **avec étiquettes** : tout ce que `np.sum` sait faire, une Série le sait faire en point (`s.sum()`), plus la mémoire de sa position dans la table.

Le 8^e résultat est le fait du dossier : `objectif_magasin` a **0 ligne** (en-tête seul — une table peut n'avoir aucune ligne ; trivial en CSV, impossible à montrer dans Excel). §5.8 regarde ce que pandas en fait.

pandas (§5.1)	SQL / M07	Ici
<code>pd.read_csv("vente.csv")</code>	chargement du jeu (M05)	50 008 × 13 (m08p_ventes_lignes, m08p_ventes_colonnes)
<code>df.shape</code>	SELECT COUNT(*) + le schéma	les 8 formes du dossier
<code>df.dtypes</code>	DESCRIBE information_schema	/ int64, float64, str, bool

5.2 head, shape, dtypes — la première lecture

```
v = pd.read_csv("03_exercices/dossier_M08/vente.csv")
print(v.head(3).to_string())
print()
print(v.dtypes.to_string())
```

	id_vente	id_client	id_produit	id_magasin	id_mode	date_vente	date_limite_remise	quantite
prix_unitaire_ht		taux_tva	montant_ttc	montant_remise	est_retour			
0	1	192	380	2	3	2026-12-12	2026-12-20	
5		35893.48	0.19	213566.20	2135.66	True		
1	2	318	311	1	1	2026-10-06	2026-10-20	
7		43110.42	0.19	359109.80	0.00	True		
2	3	447	310	2	1	2025-11-07	2025-11-20	
5		14565.94	0.18	85939.05	0.00	True		

id_vente	int64
id_client	int64
id_produit	int64
id_magasin	int64
id_mode	int64
date_vente	str
date_limite_remise	str
quantite	int64
prix_unitaire_ht	float64
taux_tva	float64
montant_ttc	float64
montant_remise	float64
est_retour	bool

Les dates sont en `str` — le type texte **dédié** de pandas 3 (C01 a posé la règle : sous l'environnement d'écriture 2.2.3, ces mêmes colonnes sont en `object` ; §5.10 mesure l'effet). Trois familles sur 13 colonnes : des entiers (`int64`), des décimaux (`float64`) — `montant_ttc` y est un **DOUBLE**, la leçon des centimes de C08 — et des textes/booléens.

Attention. Ne **branchez pas** votre code sur le `dtype` : il diffère entre 2.2.3 (`object`) et 3.x (`str`) pour **le même fichier** (mesuré, §5.10). Branchez sur la **valeur** (comparer, convertir, compter) — elle ne change pas de version.

5.3 L'index — la réduction, pas une colonne

```
print(v.index[:3])
print(v.loc[0, "montant_ttc"])
print(v.iloc[0, 10])
w = v.head(2).reset_index()
print(list(w.columns))
print(w.shape)
```

```
RangeIndex(start=0, stop=3, step=1)
213566.2
213566.2
['index', 'id_vente', 'id_client', 'id_produit', 'id_magasin', 'id_mode',
 'date_vente', 'date_limite_remise', 'quantite', 'prix_unitaire_ht', 'taux_tva',
 'montant_ttc', 'montant_remise', 'est_retour']
(2, 14)
```

Définition. index — index — l'étiquette de chaque ligne (par défaut 0, 1, 2, ...) : la poignée par laquelle `loc` et `iloc` attrapent les lignes. Ce **n'est pas une colonne** — `reset_index()` en fait une (`index` passe en tête, 14 colonnes au lieu de 13, forme (2, 14)), ce qui prouve exactement ce qu'il était.

`v.loc[0, "montant_ttc"]` et `v.iloc[0, 10]` redonnent **la même valeur** (213566.20) : la même cellule, prise par **nom** ou par **position** — le §5.4 dresse le tableau des 4 cas. L'index ne porte pas toujours le sens d'un numéro : en C08, il sera **daté** (l'index porte le mois) — c'est alors la raison d'être de `reset_index` (remettre les dates dans une colonne normale).

5.4 loc vs iloc — le tableau des 4 cas

loc vs iloc : le tableau des 4 cas

La même pièce, 4 portes — les étiquettes (loc) ou les numéros (iloc).

	id_vente	id_magasin	montant_ttc	est_retour
0	1	2	213566.20	True
1	2	1	359109.80	True
2	3	2	85939.05	True
3	4	3	390891.06	True
4	5	4	110866.60	True

l'extrait réel : v.head(5), 4 colonnes de vente.csv

cellule orange = (ligne 1, montant_ttc) · colonne bleue = id_magasin

par NOM (loc)

par POSITION (iloc)

— une LIGNE —

ligne · par nom

v4.loc[1, 'montant_ttc']

→ 359109.80

ligne · par position

v4.iloc[1, 2]

→ 359109.80

— une COLONNE —

colonne · par nom

v4.loc[:, 'id_magasin']

→ [2, 1, 2, 3, 4]

colonne · par position

v4.iloc[:, 1]

→ [2, 1, 2, 3, 4]

les 4 portes donnent la même valeur :

la cellule orange (359109.80) ou la colonne bleue — toujours la même pièce

Le piège des tranches : v4.loc[1:3] → 3 lignes · v4.iloc[1:3] → 2 lignes

la borne haute est INCLUDE par nom (loc), EXCLUDE par position (iloc) — même écriture, une ligne de moins : le contrôle len() est la garde-fou.

Sur l'extrait réel des 5 premières ventes (4 colonnes), la **même** cellule ligne 1 / montant_ttc = 359109.80 est atteinte par les 4 portes :

```
v4 = v.head(5)[["id_vente", "id_magasin", "montant_ttc", "est_retour"]]
print(v4.loc[1, "montant_ttc"]) # ligne par nom, colonne par nom
print(v4.iloc[1, 2])           # ligne par place, colonne par place
print(v4.loc[:, "id_magasin"]) # TOUTES les lignes, colonne par nom
print(v4.iloc[:, 1])           # TOUTES les lignes, colonne par place
```

```
359109.8
359109.8
0      2
1      1
2      2
3      3
4      4
Name: id_magasin, dtype: int64
0      2
1      1
2      2
3      3
4      4
Name: id_magasin, dtype: int64
```

	par nom (loc)	par position (iloc)
une ligne	v4.loc[1, "montant_ttc"] → 359109.8	v4.iloc[1, 2] → 359109.8
une colonne	v4.loc[:, "id_magasin"]	v4.iloc[:, 1]

Deux règles à retenir : **loc parle en étiquettes** (noms de colonnes, valeurs d'index — la borne haute d'une tranche est **incluse**), **iloc parle en numéros** (0 = premier — la borne haute est **exclue**, comme les tranches Python de C04). Le piège, mesuré sur la table réelle :

```
print(len(v.loc[1:3]))    # 3 lignes : id_vente 2, 3, 4
print(len(v.iloc[1:3]))  # 2 lignes : id_vente 2, 3
```

```
3
2
```

`v.loc[1:3]` **inclut** la ligne 3 (les étiquettes vont de 1 à 3, borne comprise) ; `v.iloc[1:3]` s'arrête **avant** le numéro 3. Même écriture, une ligne de moins — c'est l'erreur n° 1 du débutant pandas, et elle est **silencieuse**.

5.5 Les filtres — le `WHERE` en masque

```
print(v["est_retour"].sum())
print(len(v[v["est_retour"] == False]))
print((v["montant_ttc"] > 100000).sum())
print(((v["montant_ttc"] > 100000) & (v["est_retour"] == False)).sum())
```

```
208
49800
28065
27938
```

Définition. masque — mask — le résultat d'une comparaison sur une colonne : une Série de `True/False` **par ligne** (28 065 `True` sur `montant_ttc > 100000`), posée dans les crochets du Data-Frame pour ne garder que les lignes vraies : `v[masque]` est le `WHERE` de M07.

pandas (§5.5)	SQL (M07)	Ici
<code>v[v["est_retour"]]</code>	<code>WHERE est_retour = TRUE</code>	208 lignes (<code>m08p_retours_brut</code>)
<code>(v["montant_ttc"] > 100000).sum()</code>	<code>WHERE montant_ttc > 100000</code> <code>COUNT(*)</code>	+ 28 065 (<code>m08p_ventes_gros_100k</code>)
<code>... & ... / ... \ ... / ~...</code>	<code>AND / OR / NOT</code>	27 938 gros non retournés

Les opérateurs sont ceux du SQL, en symboles : `&` (et), `|` (ou), `~` (non) — et **chaque condition entre parenthèses, sans exception**. Pourquoi c'est une question de vie ou de mort pour le chiffre, et pas de style :

```
print(((v["montant_ttc"] > 100000) & (v["est_retour"] == False)).sum())
print((v["montant_ttc"] > 100000 & v["est_retour"]).sum()) # les parenthèses manquent
```

```
27938
50008
```

Sans parenthèses, `&` se lie **avant** `>` : la machine compare chaque montant à 100000 & `est_retour` (100 000 pour les retours, 0 pour le reste) — et sélectionne la **table entière** (50 008 au lieu de 27 938). **Aucun avertissement** : c'est le bug silencieux du chapitre — la bonne réponse et la mauvaise se présentent pareil, seules les tailles diffèrent. D'où la discipline du §9 : **après chaque filtre, `len()`** — 50 008 sur une question qui ne peut pas avoir 50 008 réponses, c'est un chiffre qui parle.

Attention. Un filtre pandas **répond toujours** — 27 938 ou 50 008, le texte se présente pareil, et rien ne distingue les deux : la parenthèse manquante est le bug silencieux du chapitre, et le `len()` après chaque sélection est la seule garde-fou qui coûte zéro.

5.6 Les nouvelles colonnes — le `SELECT ... AS`

```
v["montant_ht"] = v["montant_ttc"] / (1 + v["taux_tva"])
print(v[["montant_ttc", "montant_ht"]].head(3).to_string())
print(v["montant_ht"].sum())
w = v.assign(montant_ht=v["montant_ttc"] / (1 + v["taux_tva"]))
print(len(w.columns))
```

```
montant_ttc    montant_ht
0    213566.20  179467.394958
1    359109.80  301772.941176
2     85939.05   72829.703390
6673549808.234654
14
```

pandas (§5.6)

SQL (M07)

```
v["montant_ht"] = v["montant_ttc"] / (1 + v["taux_tva"])
v["taux_tva"]
```

```
SELECT montant_ttc, montant_ttc / (1 + taux_tva) AS
montant_ht
```

C'est la **vecteurisation de C06 dans pandas** : `/ (1 + taux_tva)` s'applique aux 50 008 lignes d'un coup (deux colonnes divisées l'une par l'autre — le broadcasting de C06 forme par forme, ici en nom de colonne). `assign` est la variante **lisible** pour enchaîner : `v.assign(a=..., b=...)` renvoie la table augmentée (14 colonnes) sans modifier l'originale — le style conseillé quand on construit une chaîne de transformations (C08).

5.7 `describe()` — la carte d'identité statistique

```
print(v["montant_ttc"].describe().to_string())
```

```
count    50008.000000
mean     158139.892261
std       133553.728026
min        762.630000
25%       49134.200000
50%       119481.890000
75%       235875.330000
max       594363.000000
```

Les **8 statistiques de M02**, en une commande : le `count` (50 008), la moyenne (158 139,89 — le panier moyen canonique 158 140 à l'arrondi, `m08p_panier_moyen`), les quartiles (49 134,20 / 119 481,89 / 235 875,33 — **les mêmes** que `np.percentile(..., [25, 50, 75])` de C06), la médiane (le 50 %), le min (762,63), le max (594 363). Le pont C06 → C07 tient : même colonne, même chiffres.

Le détail qui **diffère** : `std` = 133 553,73 ici, 133 552,39 avec `np.std` (C06) — pandas calcule l'écart-type **échantillonnaire** (division par $n-1$), NumPy l'écart-type **population** (division par n). La question de M02 retrace le chapitre : écart-type de **l'échantillon** (on estime) ou de la **population complète** (on a tout) ? Ici on a tout (le socle entier) — mais le réflexe à retenir est de **regarder** le `std` avant de comparer deux sorties.

5.8 Les valeurs manquantes — l'audit du dossier

```
print(v.isna().sum().sum())
o = pd.read_csv("03_exercices/dossier_M08/objectif_magasin.csv")
print(o.shape)
print(o["ca_objectif_ttc"].sum())
o["ca_objectif"]
```

```
0
(0, 4)
0
KeyError: 'ca_objectif'
```

Fait du socle (mesuré) : zéro cellule manquante dans les 8 CSV du dossier — l'audit `isna().sum().sum()` renvoie 0. `read_csv` fait ce qu'on attend du `objectif_magasin` vide : un DataFrame de **0 ligne × 4 colonnes** (l'en-tête donne les noms), les colonnes en `object` (aucune valeur pour deviner le type) ; `o["ca_objectif_ttc"].sum()` répond **0** sans broncher, et `o["ca_objectif"]` (celle que vous croyiez exister) lève `KeyError` — l'indexation d'une table vide ne pardonne pas les noms approximatifs.

La boîte à outils des manquants quand le fichier **en a** (démonstration sur une petite série, exécutée) :

```
s = pd.Series([1500.0, None, 2000.0, None])
print(s.isna().sum())
print(list(s.dropna()))
print(list(s.fillna(0)))
print(list(s.ffmpeg()))
```

```
2
[1500.0, 2000.0]
[1500.0, 0.0, 2000.0, 0.0]
[1500.0, 1500.0, 2000.0, 2000.0]
```

pandas (§5.8)	SQL (M06)
<code>s.isna().sum()</code>	<code>COUNT(*) - COUNT(colonne)</code> (les NULL)
<code>s.dropna()</code>	<code>WHERE colonne IS NOT NULL</code>
<code>s.fillna(0)</code> / <code>s.ffmpeg()</code>	<code>COALESCE(colonne, 0)</code> / portage de la dernière valeur

Conseil professionnel. L'audit `isna().sum()` précède tout agrégat — sur ce socle il renvoie 0 (mesuré), mais votre fichier du lundi ne vous le doit pas : un `nan` silencieux dans une colonne (C06 §5.4) contamine la somme, et l'audit d'une ligne est la seule garde-fou qui coûte zéro.

5.9 read_excel — le même objet, un autre format

```
v.head(5).to_excel("/tmp/c07/apercu.xlsx", index=False)
x = pd.read_excel("/tmp/c07/apercu.xlsx")
print(x.shape)
print(x.dtypes.to_string())
```

```
(5, 13)
id_vente          int64
id_client         int64
id_produit        int64
id_magasin        int64
id_mode           int64
date_vente        str
date_limite_remise str
quantite          int64
prix_unitaire_ht  float64
taux_tva          float64
montant_ttc       float64
montant_remise    float64
est_retour        bool
```

Aller-retour **sans perte** : 5×13 , mêmes dtypes que le CSV (les dates redeviennent du texte str, est_retour son bool). C'est le pont vers C08 (to_excel multi-feuilles pour la synthèse du projet M08.P) et le rappel du M05 : Excel n'est pas un format de données, c'est un **format de livraison** — le CSV reste la porte d'entrée, l'Excel la porte de sortie.

5.10 Les 4 écritures obsolètes — 1^{re} salve des 12

Le module documente **12 écritures** qui ne passent plus en pandas 3.x (liste complète : C08, annexe migration). Les **4** du chapitre, **exécutées** sous pandas 3.0.6 :

```
v.append({"id_vente": 999}) # 1) append
```

```
AttributeError: 'DataFrame' object has no attribute 'append'
```

→ **supprimée en 2.0** : on concatène — `pd.concat([v, nouvelle_ligne])`.

```
s = pd.Series([1.0, None, 3.0])
r = s.fillna(0, inplace=True)
r
```

```
0    1.0
1    0.0
2    3.0
dtype: float64
```

→ 2) `inplace=True` : sous 3.0.6, la ligne **modifie** la série **et** la renvoie — alors que sous 2.2.3, la **même ligne renvoie None** (sortie réelle de l'atelier : retour 2.2.3 : None). La même écriture, **deux comportements** c'est « sans effet utile » au sens fort. L'écriture validée sur les deux versions est unique — **réassigner** : `s = s.fillna(0)`.

```
avant = v["montant_ttc"].sum()
v[v["est_retour"] == True]["montant_ttc"] = 0 # 3) affectation chaînée
apres = v["montant_ttc"].sum()
print(avant)
print(apres - avant)
```

```
7908259732.2
0.0
```

(avec, sur la sortie d'erreur, un `ChainedAssignmentError` : « a value is being set on a copy ... due to Copy-on-write — try `.loc[row_indexer, col_indexer] = value` »)

→ 3) l'**affectation chaînée** : le code **ne tombe pas** (l'avertissement part sur la sortie d'erreur, le script continue), et le changement est **perdu** (écart 0,0 : la somme n'a pas bougé). C'est le « cas muet sous copy-on-write » — le piège n° 1 de pandas. L'écriture validée : `v.loc[masque, "montant_ttc"] = 0` — l'affectation en une étape, par la réduire.

```
s = pd.Series([1.0, None, 3.0])
s.fillna(method="ffill")
```

```
TypeError: NDFrame.fillna() got an unexpected keyword argument 'method'
```

→ 4) `fillna(method="ffill")` : **supprimée en 3.0** — on appelle les méthodes directes : `s.ffill()` / `s.bfill()` (exécutées en §5.8).

Dans les faits. Les 4 écarts mesurés 2.2.3 / 3.0.6 — sur le fichier « attendu » de M03 (480 lignes × 13, séparateur ;), même code, deux versions, le 19/09/2026 :

mesure	pandas 2.2.3	pandas 3.0.6
cellules vides (client)	103	85
modalités brutes (client)	373	372
lignes champ vide	18	0
colonnes texte object	7	0
dtype d'une colonne texte	object	str

Les 18 lignes « champ vide » de 2.2.3 sont **les mêmes 18** que les $85 + 18 = 103$ qui font 103 sous 2.2.3 et 85 sous 3.0.6 : sous 3.x, une cellule **vide** n'est plus une chaîne vide à compter, c'est une **absence** (NaN) que `isna()` voit et que `nunique()` ne compte pas. D'où l'écriture obsolète n° 10 de la liste : **compter une cellule texte vide comme non-vide** — on normalise **avant** de compter. C'est l'exemple fil rouge de l'évaluation E4 (« ce script tourne mais ses chiffres sont faux ») : rien ne proteste, le `dtype` a changé de nom, et 18 lignes se sont égarées entre les deux versions.

6. Exemple concret — l'audit du dossier en 8 lignes

La question du chapitre, en langage de chef de projet : « on nous a livré le dossier M08 (le brief est dans `00_brief.md`) — que vaut ce jeu de données ? » Le script — exécuté dans l'atelier le 19/09/2026 :

```
import pandas as pd

d = "03_exercices/dossier_M08/"
v = pd.read_csv(d + "vente.csv")
o = pd.read_csv(d + "objectif_magasin.csv")
dv = pd.to_datetime(v["date_vente"]); dl = pd.to_datetime(v["date_limite_remise"])

formes = {f: pd.read_csv(d + f + ".csv").shape[0]
          for f in ["vente", "client", "produit", "categorie",
                  "magasin", "mode_paiement", "regle_tva", "objectif_magasin"]}
print("formes      :", formes)
print("doublons    :", v.duplicated(subset=[c for c in v.columns if c != "id_vente"]).sum())
print("retours     :", v["est_retour"].sum())
print("dates inver :", (dv > dl).sum())
print("lignes vides:", o.shape[0])
```

```
formes      : {'vente': 50008, 'client': 1200, 'produit': 380, 'categorie': 8,
               'magasin': 5, 'mode_paiement': 5, 'regle_tva': 2, 'objectif_magasin': 0}
doublons    : 8
retours     : 208
dates inver : 21406
lignes vides: 0
```

Cinq constats, **chacun déjà connu de M07** — c'est le fil rouge « les mêmes chiffres, trois chemins », vérifié :

Constat	pandas (ci-dessus)	SQL M07 (côté à côté)	Clé du socle
Les formes	50 008 × 13, 1 200, 380, 8, 5, 5, 2, 0	les <code>COUNT(*)</code> des 8 tables	<code>m08p_ventes_lignes...</code>
8 doublons (hors <code>id_vente</code>)	<code>v.duplicated(subset=...).sum()</code>	les 8 lignes à clé dupliquée (C02 de M07)	<code>m08p_doublons = 8</code>
208 retours	<code>v["est_retour"].sum()</code>	<code>SUM(est_retour)</code>	<code>m08p_retours_brut = 208</code>

Constat	pandas (ci-dessus)	SQL M07 (côté à côté)	Clé du socle
21 406 inversées	dates (dv > dl).sum()	le vice des deadlines (C06 de M07)	m08p_dates_vente_apres_limite = 21406
0 ligne dans objectif_magasin	o.shape[0]	la table vide du schéma	m08p_objectif_lignes = 0

Le piège du 2^e print mérite sa ligne : `v.duplicated()` **tout court** renvoie **0** (toutes les lignes diffèrent — `id_vente` est unique !) ; les doublons du socle sont des **copies sans l'identifiant**, d'où le `subset=` qui retire `id_vente`. L'audit sans `subset` est le faux négatif du chapitre : « aucun doublon » sur une table qui en a 8.

7. Démonstration pas à pas — 6 étapes sur le fil rouge

Sortie réelle de chaque commande (exécutées le 19/09/2026, venv pandas 3.0.6).

Étape 1 — L'import (`read_csv` sur le jeu principal) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
print(v.shape)
(50008, 13)
```

Interprétation : 50 008 lignes × 13 colonnes (`m08p_ventes_lignes`, `m08p_ventes_colonnes`) — la table de M07, entière, en mémoire.

Étape 2 — La première lecture (`head` + `dtypes`) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
print(v.dtypes.to_string())"
id_vente          int64
id_client         int64
id_produit        int64
id_magasin        int64
id_mode           int64
date_vente        str
date_limite_remise str
quantite          int64
prix_unitaire_ht  float64
taux_tva          float64
montant_ttc       float64
montant_remise    float64
est_retour        bool
```

Interprétation : 3 familles — `int64`, `float64`, et le texte `str` de pandas 3 (les dates), plus le `bool` de `est_retour`.

Étape 3 — Le filtre retours (le `WHERE` en masque) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
print(v['est_retour'].sum())
print(len(v[v['est_retour'] == False]))"
208
49800
```

Interprétation : 208 retours bruts (`m08p_retours_brut`) et le complément — toujours les deux chiffres : le total moins les uns, c'est le contrôle mutuel.

Étape 4 — La nouvelle colonne (`montant_ht` vectorisée) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
v['montant_ht'] = v['montant_ttc'] / (1 + v['taux_tva'])
print(v['montant_ht'].head(3).to_string())"
0      179467.394958
1      301772.941176
2       72829.703390
Name: montant_ht, dtype: float64
```

Interprétation : 50 008 valeurs en une ligne — la TVA de chaque vente déduite, sans boucle (le `SELECT ... AS` de M07).

Étape 5 — La carte statistique (`describe`) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
print(v['montant_ttc'].describe().to_string())"
count      50008.000000
mean       158139.892261
std        133553.728026
min         762.630000
25%        49134.200000
50%        119481.890000
75%        235875.330000
max        594363.000000
```

Interprétation : les 8 statistiques de M02 — la moyenne redonne le panier moyen canonique (158 140 à l'arrondi), les quartiles ceux de C06 ; le `std` est l'écart-type échantillonnaire ($n-1$) — regardez-le avant de comparer.

Étape 6 — L'audit des manquants (`isna`) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
print(v.isna().sum().sum())"
0
```

Interprétation : zéro cellule manquante dans le jeu principal (fait du socle, mesuré) — l'audit est une ligne, et il précède tout agrégat (§5.8).

8. Erreurs fréquentes

- Le filtre sans parenthèses — le bug silencieux.** `(v["montant_ttc"] > 100000 & v["est_retour"]).sum()` → **50 008** au lieu de 27 938 (sortie réelle, §5.5) : `&` se lie avant `>`, la machine compare à `100000 & est_retour`, et **rien ne proteste**. Remède : chaque condition entre parenthèses, **sans exception** — et `len()` après chaque filtre : le chiffre qui ne peut pas être le total de la table est un chiffre qui parle.
- loc/iloc mélangés — la ligne de moins.** `v.loc[1:3]` → 3 lignes, `v.iloc[1:3]` → 2 lignes (sortie réelle, §5.4) : la borne haute est incluse par nom, exclue par position. Remède : une **seule** des deux fonctions par script — `loc` par défaut (les noms), `iloc` quand la position est l'information.
- L'affectation chaînée — le changement perdu.** `v[v["est_retour"] == True]["montant_ttc"] = 0` : le code ne tombe pas (avertissement sur la sortie d'erreur, §5.10), et l'écart est **0,0** — la somme n'a pas bougé. Remède : `v.loc[masque, "montant_ttc"] = 0` — une étape, par la réécriture.

4. **Compter les doublons sans subset.** `v.duplicated().sum()` → **0** sur une table qui a **8** doublons (le §6 le mesure) : `id_vente` est unique, donc « toutes colonnes » ne duplique rien. Remède : `duplicated(subset=...)` sur les colonnes qui **définissent** la ligne — ici tout sauf `id_vente`.
5. **Brancher sur le dtype au lieu de la valeur.** « Si la colonne est en `object`, c'est du texte » — faux sous 3.x, où elle est en `str` (mesuré §5.10 : 7 colonnes `object` sous 2.2.3, 0 sous 3.0.6, **même fichier**). Remède : la logique sur la **valeur** (convertir, comparer, compter) — le `dtype` s'affiche, il ne commande pas.
6. **Attendez-vous à des manquants... et n'en trouver aucun.** L'audit `isna` renvoie 0 (fait du socle, mesuré) — et c'est **normal**, pas un bug : ce socle a été nettoyé en amont (M05). Le réflexe à garder est l'audit lui-même sur votre fichier du lundi, le 0 du socle n'est pas promis.

9. Bonnes pratiques professionnelles

1. **dtypes juste après chaque read_csv** — la première ligne de lecture d'une table inconnue (comme le `DESCRIBE` de M07) ; c'est là que le `str` d'une date, ou le `float` d'un identifiant, se voient.
2. **loc par défaut, iloc en exception** — les noms résistent aux réordres de colonnes ; les positions, non. Une seule des deux par script.
3. **Parenthèses sur chaque condition, len() après chaque filtre** — le bug silencieux (§5.5) est le seul qui ne crie pas ; le contrôle de taille est le chien de garde.
4. **Réassigner, jamais inplace** — `s = s.fillna(0)`, `v = v.drop(...)` : l'écriture unique validée sur 2.3 et 3.x (§5.10).
5. **L'audit (isna, duplicated, formes) précède l'agrégat** — sur ce socle 0 manquant, 8 doublons, 1 table vide (mesuré, §6) ; sur votre jeu du lundi, ce sont **vos** chiffres qui sortent — publiez-les dans le rapport, comme M05.

10. Exercice guidé — « la vue des 120 premières ventes » (15 min, /10)

Énoncé. Écrire `vue120.py` qui, sur les **120 premières lignes** réelles de `vente.csv` (`v = pd.read_csv(...).head(120)` — la vue `m08p_vue_lignes` du socle) :

1. calcule la **médiane** de `montant_ttc` (2 décimales) ;
2. compte les montants **> 100 000** (le masque, pas la boucle) ;
3. crée la colonne `montant_ht` et affiche son **maximum** (2 décimales) ;
4. affiche le **comptage des manquants** (`isna().sum().sum()`).

Barème. La vue 120 chargée correctement (2 pts) · les 4 calculs avec les outils du chapitre (`median`, masque `.sum()`, colonne vectorisée, `isna`) (4 pts) · l'affichage 2 décimales en f-string (2 pts) · la sortie alignée sur le modèle ci-dessous (2 pts). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```

Mediane           : 122,065.68
Gros > 100 000: 78
Montant HT max: 471,617.12
Manquants         : 0

```

11. Exercices autonomes

Exercice 7.1 — La carte d’une colonne. `describe()` sur `montant_remise` : afficher les 8 statistiques, et dire en une phrase pourquoi le quartile 25 % est 0. (10 min)

Exercice 7.2 — Les 4 portes. Sur `v.head(5)` réduit à 4 colonnes (`id_vente`, `id_magasin`, `montant_ttc`, `est_retour`) : atteindre la cellule (ligne 1, `montant_ttc`) par les 4 chemins (`loc/iloc` × ligne/colonne) — les 4 sorties doivent être identiques. (10 min)

Exercice 7.3 — Le tamis à trois conditions. Compter les ventes `est_retour == True` **et** `montant_ttc < 50000` **et** `taux_tva == 0.18` — masque à 3 conditions, parenthèses comprises ; afficher aussi le comptage avec **une** parenthèse manquante, pour voir l’écart. (15 min)

Exercice 7.4 — Les villes. Sur `client.csv` : le nombre de **villes distinctes**, la **modale** (`mode()`), et le maximum de `plafond_credit`. (10 min)

Exercice 7.5 — L’aller-retour. `to_excel` des 50 premières ventes dans `/tmp/c07/aperçu50.xlsx`, rechargement par `read_excel`, affichage de la forme et de **deux** dtypes — vérifier l’aller-retour sans perte. (10 min)

12. Correction détaillée

Exercice guidé. Le script :

```
import pandas as pd

v = pd.read_csv("03_exercices/dossier_M08/vente.csv").head(120)
s = v["montant_ttc"]
v["montant_ht"] = s / (1 + v["taux_tva"])
print(f"Mediane      : {s.median():.2f}")
print(f"Gros > 100 000: {(s > 100000).sum()}")
print(f"Montant HT max: {v['montant_ht'].max():.2f}")
print(f"Manquants     : {v.isna().sum().sum()}")
```

Les 4 lignes publiées **sont** la sortie de l’atelier (122,065.68 / 78 / 471,617.12 / 0). Le contrôle à faire en tête : médiane = `s.quantile(0.5)` — les deux doivent redonner le même nombre.

Exercice 7.1. La sortie réelle :

```
count    50008.000000
mean      1017.110771
std       1571.906376
min         0.000000
25%         0.000000
50%       173.815000
75%      1508.530000
max      8915.440000
```

La phrase : **la moitié et plus des ventes n’a pas de remise** (le quartile 25 % est 0, comme le minimum — la remise est une option, pas la règle ; la médiane 173,82 le confirme).

Exercice 7.2. Les 4 chemins : `v4.loc[1, "montant_ttc"]` → 359109.8 · `v4.iloc[1, 2]` → 359109.8 · (et pour la colonne : `v4.loc[:, "id_magasin"]` et `v4.iloc[:, 1]` → [2, 1, 2, 3, 4]) — sortie réelle du §5.4 : les 4 portes mènent à la même pièce.

Exercice 7.3. Comptage correct (masque à 3 conditions) : **26** retours sous 50 000 avec taux 18 % (sortie réelle) ; la variante 2 conditions (`est_retour & (montant_ttc < 50000)`) en donne **58** — la 3^e condition affine. L’écriture sans parenthèses, elle, lève un `TypeError` (sortie réelle : « Cannot perform ‘rand_’ with a dtypes [float64] array and scalar of type [bool] ») : la même oubli produit un **crash** ou le **silencieux 50 008** du §5.5, selon la forme — aucun des deux n’est « presque bon » : c’est l’écart qui est le résultat de l’exercice, pas le comptage.

Exercice 7.4. Sortie réelle : 3 villes distinctes, modale ['Ouagadougou'], plafond maximum **un million** de FCFA (la valeur sort dans votre terminal : 1000000).

Exercice 7.5. `read_excel` renvoie (50, 13) avec les mêmes dtypes que le CSV (`id_vente` int64, `est_retour` bool, `date_vente` str) — l'aller-retour sans perte, comme l'Étape du §5.9.

13. Mini-projet M08.P7 — « L'audit du dossier » (1 h)

Énoncé. Écrire `audit.py` — le script qui répond au brief de `00_brief.md` :

1. importer les 8 CSV du dossier (boucle sur les noms, pas 8 fois le même `read_csv`) ;
2. afficher les **formes** des 8 (dictionnaire nom → nombre de lignes) ;
3. sur `vente` : les **doublons** (hors `id_vente`), les **retours**, les **dates** `date_vente > date_limite_remise` (via `pd.to_datetime`), le **comptage des manquants** ;
4. signaler la **table vide** (0 ligne) par son nom ;
5. afficher, en dernier, la **médiane** et le **95^e percentile** de `montant_ttc` (2 décimales).

Critères de réussite (grille /10) : les 5 exigences (5 × 1 pt) · la sortie de l'atelier : formes 50008/1200/380/8/5/5/2/0 · doublons 8 · retours 208 · dates inversées 21 406 · manquants 0 · table vide `objectif_magasin` · médiane 119481.89 · p95 431647.88 (1 pt) · un seul `read_csv` par fichier, aucun `for` sur les lignes (1 pt) · le dtypes de `vente` affiché en tête (1 pt) · le rapport lisible par un humain (1 pt).

Note de correction. Ce projet est l'exercice fil rouge du plan, transformé en livrable : les 5 constats du §6, mais **écrits une fois, dans votre script**. La valeur ajoutée professionnelle n'est pas le code (15 lignes), c'est le **rapport** : 5 chiffres, 5 phrases, 0 « il semble que ». Si un chiffre de votre sortie ne correspond pas au §6, c'est votre script qui a tort — pas le socle (les clés `m08p_*` tranchent).

14. Boîte à outils du chapitre

Boîte à outils. Le venv de C01 (`/tmp/venv_m08`, pandas 3.0.6) + le dossier `03_exercices/dossier_M08/` — la boîte du chapitre s'ajoute à celles de C02-C06 : le classeur d'indexation.

Outil	Usage	Syntaxe
<code>pd.read_csv / read_excel</code>	l'import (porte d'entrée / de sortie)	<code>pd.read_csv(f)</code>
<code>head / shape / dtypes / info</code>	la première lecture	<code>v.dtypes</code>
<code>loc / iloc</code>	la sélection (nom / position)	<code>v.loc[1, "col"], v.iloc[1, 2]</code>
masque booléen	le WHERE	<code>v[(v["a"] > 1) & (v["b"] == 2)]</code>
<code>df["col"] = opération / assign</code>	la nouvelle colonne	<code>v["mt_ht"] = v["mt"] / (1 + v["t"])</code>
<code>describe()</code>	les 8 statistiques	<code>s.describe()</code>
<code>isna()</code> / <code>dropna()</code> / <code>fillna()</code> / <code>ffill()</code>	les manquants	<code>s.isna().sum()</code>
<code>duplicated(subset=...)</code>	les doublons réels	<code>v.duplicated(subset=...).sum()</code>
<code>to_excel / read_excel</code>	l'aller-retour Excel	<code>v.to_excel(f, index=False)</code>

15. Résumé du chapitre

- Le **DataFrame** est la table SQL de M07 en Python : colonnes nommées, dtypes, index-ré-
dure ; la **Série** est sa colonne (le tableau de C06 avec étiquettes) — `s.sum()` redonne 7 908
259 732,20, le `np.sum` de C06.
- **loc** (nom, borne incluse) vs **iloc** (position, borne exclue) : le tableau des 4 cas, et le piège
des tranches (3 lignes contre 2, mesuré).
- Les **filtres** sont des **masques** (True/False par ligne) : `&/|/~` et **parenthèses obligatoires** —
sans, le bug silencieux sélectionne 50 008 au lieu de 27 938, sans crier.
- La **nouvelle colonne** est vectorisée (`montant_ht = montant_ttc / (1 + taux_tva)`) — le `SELECT`
... AS ; assign enchaîne sans modifier.
- **describe()** : les 8 statistiques de M02 — mêmes quartiles que C06, std échantillonnaire
(n-1) là où NumPy faisait population (n).
- Les **manquants** : l'audit du socle en renvoie **0** (mesuré) ; la table `objectif_magasin` est **vide**
(0 × 4), `KeyError` sur le nom approximatif — `isna/dropna/fillna/ffill` sont la boîte pour
quand le fichier du lundi en a.
- Les **4 écritures obsolètes** (1^{re} salve des 12) : `append` (supprimée), `inplace` (deux comporte-
ments selon la version), l'affectation chaînée (changement **perdu**, écart 0,0), `fillna(method=...)` (supprimée) — et les **4 écarts 2.2.3/3.0.6 mesurés** (103 → 85, 373 → 372, 18 →
0, 7 → 0, `object` → `str`) : la version change la vérité terrain.

16. À retenir

À retenir. Un filtre pandas **répond toujours** — avec le bon chiffre ou le mauvais, **sans crier** :
`len()` après chaque sélection, parenthèses sur chaque condition, et `loc/iloc` ne se mêlent pas
(3 lignes contre 2, même écriture). Le chiffre de pandas se **vérifie** par sa taille, pas par sa poli-
tesse.

À retenir. La **version fait partie du résultat** : même fichier, même code, pandas 2.2.3 et 3.0.6
ne comptent pas les mêmes cellules vides (103 → 85, mesuré). On ne branche donc **jamais** sur le
`dtype` — on branche sur la **valeur** ; et l'audit (formes, doublons, manquants, tables vides) pré-
cède **tout** agrégat.

17. Évaluation formative (auto-correction, 8 min)

1. `v.loc[1:3]` et `v.iloc[1:3]` — **combien de lignes chacune, et pourquoi ?** → 3 lignes et 2
lignes : `loc` sélectionne par **étiquettes** (borne haute incluse), `iloc` par **positions** (borne
haute exclue) — même écriture, une ligne de moins (§5.4). (2 pts)
2. **Que renvoie `v["montant_ttc"] > 100000 & v["est_retour"]` (sans parenthèses), et quel est
le danger ?** → Un masque **silencieux** qui sélectionne la table entière (50 008 au lieu de 27
938 — mesuré) : `&` se lie avant `>`, donc comparaison à 100000 & `est_retour` ; aucun avertisse-
ment — d'où le `len()` après chaque filtre. (2 pts)
3. `v.duplicated().sum()` **renvoie 0 sur la table du socle. Y a-t-il des doublons ? Que faire ?**
→ Oui : **8** (`m08p_doublons`) — mais `id_vente` est unique, donc « toutes colonnes » ne duplique
rien ; il faut `duplicated(subset=...)` sans `id_vente` (le §6 le mesure). (2 pts)
4. `v.loc[0, "montant_ttc"]` et `v.iloc[0, 10]` — **que ont-ils de commun ?** → La **même cellule**
(213566.20), atteinte par **nom** (étiquettes) ou par **position** (numéros) — les deux portes de
la même pièce ; `montant_ttc` est la 11^e colonne (index 10). (1 pt)

5. **Pourquoi std vaut-il 133 553,73 dans describe() et 133 552,39 avec np.std (C06) ?** → pandas calcule l'écart-type **échantillonnaire** (division par $n-1$), NumPy l'écart-type **population** (division par n) — le détail à vérifier avant de comparer deux sorties (§5.7). (1 pt)
6. **objectif_magasin.csv a 0 ligne. Que fait read_csv, que renvoie o["ca_objectif_ttc"].sum(), et que lève o["ca_objectif"] ?** → Un DataFrame 0×4 (l'en-tête donne les noms, colonnes object) ; la somme renvoie **0** sans broncher ; le nom approximatif lève `KeyError: 'ca_objectif'` (§5.8). (1 pt)
7. **Sous pandas 3.0.6, s.fillna(0, inplace=True) renvoie quoi, et sous 2.2.3 ?** → Sous 3.0.6 : **la série remplie** (et la série est modifiée) ; sous 2.2.3 : **None** — la même ligne, deux comportements ; l'écriture validée sur les deux versions est `s = s.fillna(0)` (§5.10). (1 pt)
8. **Les 4 écarts mesurés 2.2.3/3.0.6 du fichier « attendu » de M03 — les citer, et dire ce qu'ils prouvent.** → cellules vides $103 \rightarrow 85$, modalités $373 \rightarrow 372$, lignes champ vide $18 \rightarrow 0$, colonnes texte object $7 \rightarrow 0$ (et object $\rightarrow$ str) : la **version change la vérité terrain** sans protester — d'où « brancher sur la valeur, pas sur le dtype » et l'audit avant tout agrégat (E4 du module). (1 pt)

Module M08.C08 — pandas II : `groupby/transform/merge, pivot_table, dates, apply, export Excel` — et l'annexe migration 2.3 → 3.x

Outils : le venv de C01 (`/tmp/venv_m08`, pandas 3.0.6 + `matplotlib` ajouté pour les 4 graphiques du projet) + l'environnement d'écriture du manuel (pandas 2.2.3) pour l'annexe. Le dossier `03_exercices/dossier_M08/`. **Durée indicative :** 3 h hors exercices. **Niveau :** N6 → N7. **Prérequis :** C07 (la table en mémoire), M07 (toutes les requêtes SQL qui sont traduites ici).

L'idée du chapitre. M07 a tout fait en SQL : `GROUP BY`, `JOIN`, la vue mensuelle, `EXTRACT`. C08 refait **exactement le même travail** en pandas — `groupby/agg`, `transform`, `merge`, `pivot_table`, `to_datetime/resample` — et le fil rouge en est la **preuve** : les 30 questions du projet M07.P, réécrites en pandas, dont 10 exécutées et comparées au corrigé SQL (**0 écart** sur les comptages, **± 2 FCFA** sur les totaux — la dérive mesurée et documentée). Le chapitre se termine par le **script de bout en bout** du projet M08.P (`import` → `audit` → `nettoyage` → `indicateurs` → 4 graphiques → classeur, exécuté d'une traite) et par l'**annexe migration** : les 12 écritures obsolètes avec le code avant/après — « votre script de 2026 tournera en 2028 ».

Matériel de l'atelier. Tout a été exécuté le 19/09/2026 dans le venv pandas 3.0.6 (et l'annexe comparée sous 2.2.3). La règle monétaire du module tient en une phrase : **les totaux s'additionnent en centimes entiers** — c'est la leçon du §5.8, elle explique le « **± 2 FCFA** » que vous allez voir partout. Les repères canoniques : CA brut 7 908 259 732 (`m08p_ca_brut`), sans retours 7 876 320 164, panier 158 140, top magasin 4 = 1 596 813 264, plus gros mois 78 965 529.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Agréger avec `groupby` + `agg` : le `GROUP BY` de M07 en une ligne (les 5 magasins, CA 1 564 937 883 à 1 596 813 264, top magasin 4).
2. Remettre l'agrégat **sur chaque ligne** avec `transform` (le « delta vs la moyenne de son magasin » de C06, en une ligne — ce que `GROUP BY` ne fait pas).
3. Joindre avec `merge` : le `JOIN` de M07, `how="left"` (zéro ligne perdue) — et la table **vide** `objectif_magasin` qui rend toute la colonne en `NaN` (mesuré : 50 008).
4. Reconstruire la **vue** `v_ca_mensuel_magasin` de M07 avec `pivot/ groupby` : 120 lignes, plus gros mois 78 965 529 — **sans SQL**.
5. Manipuler les **dates** : `pd.to_datetime`, `dt.year/dt.month`, `resample("ME")` — et l'**alias 'M' est mort** (le `ValueError` réel).
6. Savoir **quand apply est légitime** — et le mesurer : la fonction de C05 sur une colonne coûte 4× la version vectorisée (mesuré), même résultat.
7. Exporter un **classeur Excel multi-feuilles** (`ExcelWriter`, feuilles nommées, largeurs) — le livrable E3 du projet.
8. Lire l'**annexe migration 2.3 → 3.x** : les 12 écritures obsolètes (avant/après) et les 4 écarts mesurés — la garantie « votre script de 2026 tournera en 2028 ».

2. Pourquoi cette notion est importante

Parce que la transition est posée : « SQL vous donne le chiffre. Python vous donne la méthode, la vitesse, et la capacité de refaire tout cela en une seconde le mois prochain. » — C08 est la preuve : les 30 questions de la direction (M07.P), la vue mensuelle, le classeur de synthèse — tout est **rejoué** en pandas, et le mois prochain le script tourne seul. Le SQL n'est pas remplacé, il est **complété** : le SGBD garde le rôle du moteur sur les jeux massifs, le script pandas prend celui de la méthode réutilisable, versionnable, commentée.

Parce que **le float a une mémoire** — et qu'elle s'appelle « centimes ». Les totaux de ce chapitre s'additionnent en **centimes entiers** (int64) : le float64 dérive de 1 à 2 FCFA sur 7,9 milliards (mesuré, \$5.8) — négligeable en métier, **visible en audit**. La règle du module : comptages **exacts**, totaux **± 2 FCFA**, dérive **documentée** dans le rapport — c'est ce que votre classeur E3 devra afficher, pas ce qu'il faudra cacher.

Parce que **les versions changent sous vos pieds** : les 4 écritures vues en C07 (append, inplace, chaînée, fillna(method=...)) ne sont que la 1^{re} salve — l'annexe du chapitre présente les **12** avec le code avant/après, et les 4 écarts de vérité terrain (103 → 85, 373 → 372, 18 → 0, 7 → 0) déjà mesurés en C07 §5.10. Un script qui branche sur la valeur plutôt que sur le dtype survit à la migration ; l'autre meurt silencieusement.

3. Explication simple — les classeurs et le classement

- **groupby** est le **classement en dossiers** : on trie les 50 008 fiches par magasin (5 dossiers), puis on pose la question **par dossier** (la somme, le compte, la moyenne) — le **GROUP BY** de M07, mais les dossiers sont en mémoire.
- **transform** est l'étiquette qu'on **recolle sur chaque fiche** : après avoir calculé la moyenne de son dossier, on la remet sur chacune des fiches du dossier — **GROUP BY** renvoie un résumé (5 lignes), **transform** renvoie la table d'origine (50 008 lignes) avec la valeur du groupe en plus.
- **merge** est la **correspondance de fiches** : deux classeurs, on les aligne sur le numéro de référence (id_magasin) — chaque fiche garde ses voisines, et si l'autre classeur est vide... toutes les cases opposées restent **vides** (NaN).
- **pivot_table** est le **tableau tourné** : les lignes deviennent des colonnes (ou l'inverse) — la vue « magasin × mois » de M07, en un geste.
- **resample** est le **classeur par mois** : on range chaque fiche dans son mois, puis on agrège — **dt.year/dt.month** sont les rubriques, **resample("ME")** est le tri par fin de mois.
- **apply** est l'**ascenseur** : on envoie la fonction fiche par fiche (c'est le **for** déguisé) — pratique quand la fiche a besoin d'un coup de pouce individuel (du texte « N/C », une décision par valeur), **lourd** quand un convoi vectorisé passerait à pleine vitesse.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
regroupement groupby	— la méthode qui trie les lignes par clé (un groupe par valeur de clé), avant l'agrégat.	croire qu'on agrège « tout d'un coup » : groupby renvoie un objet qu'il faut ensuite agréger (agg, sum...).
agrégation — agg	l'opération par groupe (sum, mean, count...) — la réduction d'un groupe à une valeur.	agréger des float64 pour un total monétaire : additionner en centimes entiers (\$5.8).
transformation transform	— remettre la valeur du groupe sur chaque ligne du groupe (la table garde sa taille).	confondre avec agg : agg réduit (5 lignes), transform ne réduit pas (50 008 lignes).

Français — English	Définition en une phrase	Piège à éviter
jointure — merge	l'alignement de deux DataFrames sur une clé commune — le JOIN de M07.	un merge sans clé explicite : il joint sur l'intersection des index — souvent au pire endroit.
table pivotée — pivot_table	la réorganisation lignes ↔ colonnes autour d'une clé et d'une valeur.	oublier la définition de la vue (sans retours, ici) : changer le filtre change le nombre.
rééchantillonnage — resample	le regroupement par période de dates (mois, trimestre) après un index daté.	l'alias 'M' est mort en 3.x : écrire 'ME' (le ValueError du \$5.5).

5. Cours approfondi

5.1 groupby + agg — le GROUP BY en une ligne

GROUP BY SQL et groupby pandas : la même chaîne de calcul

Deux syntaxes, une même sortie — les 5 magasins du socle, additionnés en centimes.

SQL — le moteur M07

```
SELECT id_magasin,
SUM(montant_ttc)
FROM vente
GROUP BY id_magasin
ORDER BY 2 DESC;
```

pandas — C08, en centimes entiers

```
ct = (v["montant_ttc"] * 100)
.round().astype("int64")
ct.groupby(v["id_magasin"])
.sum() // 100
```

la même sortie — CA par magasin (FCFA)

id	in	
4		1 596 813 264
1		1 595 223 423
2		1 578 987 709
5		1 572 297 451
3		1 564 937 883

top magasin : 4 — 1 596 813 264 FCFA (m08p_ca_top_magasin)

Les centimes : additionner en entiers, diviser après l'agrégat —

la dérive float (1 à 2 FCFA sur 7,9 milliards) est mesurée et documentée, pas cachée.

```
import pandas as pd

v = pd.read_csv("03_exercices/dossier_M08/vente.csv")
ct = (v["montant_ttc"] * 100).round().astype("int64") # les centimes ($5.8)
g = v.groupby("id_magasin")["montant_ttc"].agg(["sum", "count", "mean"])
print(g.to_string())
print()
print((ct.groupby(v["id_magasin"]).sum() // 100).to_string())
```

```

      sum  count      mean
id_magasin
1      1.595223e+09  10099  157958.552683
2      1.578988e+09  10033  157379.418820
3      1.564938e+09   9873  158506.825040
4      1.596813e+09  10061  158713.176035
5      1.572297e+09   9942  158146.997779

id_magasin
1      1595223423
2      1578987709
3      1564937883
4      1596813264
5      1572297451

```

pandas (§5.1)

```
v.groupby("id_magasin")
["montant_ttc"].agg([...])
```

SQL (M07, Q07)

```
SELECT id_magasin, ROUND(SUM(montant_ttc),0), COUNT(*),
AVG(montant_ttc) FROM vente GROUP BY id_magasin
```

Cinq groupes, chacun sa somme, son compte (10 099 + 10 033 + 9 873 + 10 061 + 9 942 = 50 008 — le contrôle de taille du C07), sa moyenne. Le **top magasin est le 4** (1 596 813 264 FCFA, `m08p_ca_top_magasin`) ; le plus petit est le 3 (1 564 937 883, `m08p_ca_min_magasin`). La 2^e sortie est la même somme, **additionnée en centimes** — la colonne de référence monétaire du chapitre.

5.2 transform — la moyenne du groupe, remise sur chaque ligne

```

v["delta"] = v["montant_ttc"] - v.groupby("id_magasin")["montant_ttc"].transform("mean")
print(v.head(3)[["id_magasin", "montant_ttc", "delta"]].to_string())
print("delta moyen magasin 1 :", v.loc[v["id_magasin"] == 1, "delta"].mean())

```

```

      id_magasin  montant_ttc      delta
0           2    213566.20  56186.781180
1           1    359109.80  201151.247317
2           2     85939.05  -71440.368820
delta moyen magasin 1 : -8.299735787245575e-12

```

Le **delta de chaque vente à la moyenne de son magasin** — en une ligne. Les trois valeurs de la sortie **sont** celles de C06 §5.6 (le `moyennes[ids - 1]` du broadcasting, calculé à la main) : `transform` fait le même travail sans alignement manuel, et la table garde ses 50 008 lignes (la colonne `delta` est une **colonne** comme une autre). Le contrôle de la dernière ligne redonne ≈ 0 (-8×10^{-12} , le bruit `float`) : si le delta moyen d'un groupe n'est pas zéro, la moyenne du groupe n'est pas celle du groupe.

Définition. transformation — `transform` — l'opération qui calcule une valeur **par groupe** et la **repose sur chaque ligne** du groupe : la table de départ garde sa forme (50 008 × 14), chaque ligne portant l'agrégat de son groupe. C'est l'outil des **comparaisons relatives** (« cette vente vs la moyenne de son magasin »), que ni `agg` ni le `GROUP BY` ne font.

5.3 merge — le JOIN de M07, et la table vide qui rend tout NaN

```

m = pd.read_csv("03_exercices/dossier_M08/magasin.csv")
o = pd.read_csv("03_exercices/dossier_M08/objectif_magasin.csv")
vm = v.merge(m, on="id_magasin")
print(vm.shape)
vo = v.merge(o, on="id_magasin", how="left")
print(vo.shape, "| NaN ca_objectif_ttc :", int(vo["ca_objectif_ttc"].isna().sum()))

```

```
(50008, 18)
(50008, 16) | NaN ca_objectif_ttc : 50008
```

pandas (§5.3)**SQL (M07, C07)**

```
v.merge(m, on="id_magasin")
```

```
SELECT v.*, m.nom FROM vente v JOIN magasin m ON v.id_magasin =
m.id_magasin
```

```
v.merge(o, on="id_magasin", how="left")
```

```
... LEFT JOIN objectif_magasin ... (la table vide)
```

Définition. jointure — merge — l'alignement de **deux DataFrames** sur une **clé commune explicite** (`on="id_magasin"`), avec le mode de conservat des lignes (`how=`) : le JOIN de M07, où la clé est une information **métier** et non un détail de syntaxe.

Attention. Une colonne **entière en NaN** après un merge est un **signal** à lire, pas un résultat : la table opposée est vide (ici `objectif_magasin`), ou la clé ne correspond pas — on l'inspecte avant d'exporter.

merge est le JOIN en nom : 13 + 5 colonnes = **18**, zéro ligne perdue (chaque `id_magasin` de vente existe dans `magasin`). Le 2^e exemple est le **piège du dossier** : `objectif_magasin` est **vide** (C07 §5.8), et la LEFT JOIN renvoie 50 008 lignes... dont **toutes** les cases `ca_objectif_ttc` sont NaN (50 008, mesuré) — la table est jointe, le LEFT sauve les lignes, mais il n'y a **rien** à rejoindre. Un merge « correct » qui produit une colonne entière de NaN est un signal à lire : la table opposée est vide, ou la clé ne correspond pas.

5.4 pivot — la vue mensuelle de M07, reconstruite sans SQL

La vue `v_ca_mensuel_magasin` de M07 est définie **sans retours** (`WHERE est_retour = FALSE`) — c'est la définition de la vue, pas un choix :

```
vs = v.loc[~v["est_retour"]]
vue = pd.DataFrame({
    "id_magasin": vs["id_magasin"],
    "annee": pd.to_datetime(vs["date_vente"]).dt.year,
    "mois": pd.to_datetime(vs["date_vente"]).dt.month,
    "ca_cents": ct[vs.index],
}).groupby(["id_magasin", "annee", "mois", as_index=False)["ca_cents"].sum()
vue["ca"] = vue["ca_cents"] // 100
print("lignes vue :", len(vue))
print(vue.sort_values("ca", ascending=False).head(3).to_string(index=False))
```

```
lignes vue : 120
id_magasin annee mois ca_cents ca
3 2025 8 7896552921 78965529
1 2025 3 7743800620 77438006
4 2025 2 7640325426 76403254
```

Définition. table pivotée — `pivot_table` — la réorganisation **lignes ↔ colonnes** d'une table autour d'une clé (ici `magasin × mois`) et d'une valeur agrégée (le CA) : la vue « tableau croisé » de M07, en un geste pandas.

Attention. Une vue a une **définition** — la vue M07 est définie **sans retours** (`WHERE est_retour = FALSE`) : changer le filtre change le **nombre**, pas les données (l'écart est mesuré en §8, n° 2). Avant un pivot, dire quelles lignes la vue contient — la question est un cinquième du travail.

120 lignes (`m08p_vue_lignes` : 5 magasins × 24 mois) — la vue de M07 est **reconstruite**, sans SQL — et le plus gros mois est le ****2025-08** du magasin 3

78 965 529 FCFA (`m08p_plus_gros_mois_ca`) — le même nombre que le `SELECT` de M07, au FCFA près. Remarquez l'ordre des opérations : on somme en centimes (`ca_cents`), on divise après** — l'erreur du §8 (n° 1) est de diviser d'abord.

5.5 Les dates — l'index qui porte un sens, et l'alias 'M' mort

```
dv = pd.to_datetime(v["date_vente"])
print("2025 :", int((dv.dt.year == 2025).sum()), "ventes | CA centimes :",
      ct[dv.dt.year == 2025].sum() // 100, "| CA float :", v.loc[dv.dt.year == 2025,
"montant_ttc"].sum())
s = pd.Series(ct.to_numpy() // 100, index=dv)
r = s.resample("ME").sum()
print("resample ME :", len(r), "mois | plus fort :", r.idxmax().date(), int(r.max()))
try:
    s.resample("M").sum()
except Exception as e:
    print(type(e).__name__)
    print(e)
```

```
2025 : 24920 ventes | CA centimes : 3925215672 | CA float : 3925215672.71
resample ME : 24 mois | plus fort : 2026-06-30 350948588
ValueError
Invalid frequency: M. Failed to parse with error message: ValueError("'M' is no longer supported
for offsets. Please use 'ME' instead.)")
```

`pd.to_datetime` transforme les `str` de C07 en **vraies dates** : `dt.year`, `dt.month`, `dt.dayofweek` (attention : 0 = lundi, 6 = **dimanche** — le `DOW` de DuckDB a 0 = dimanche, c'est l'inverse, le §8 n° 6) sont les `EXTRACT` de M07. 2025 : **24 920 ventes** (`m08p_nb_ventes_2025`), CA 3 925 215 672 en centimes (`m08p_ca_2025`) contre 3 925 215 672,71 en `float` — la dérive de 0,71 FCFA, le §5.8 la documente. `resample("ME")` range chaque vente dans son mois (24 mois), et l'alias **'M' est mort** : le `ValueError` cite la solution (`'ME'`) — le message d'erreur **est** la migration, l'annexe du §5.9 l'enseigne pour les 12 écritures.

5.6 `apply` — la fonction de C05 sur une colonne, et son prix mesuré

```
import time
import numpy as np

def classe_montant(x):
    if x > 100000:
        return "gros"
    if x > 10000:
        return "moyen"
    return "petit"

t0 = time.perf_counter()
r1 = v["montant_ttc"].apply(classe_montant)
t1 = time.perf_counter()
t2 = time.perf_counter()
r2 = np.where(v["montant_ttc"] > 100000, "gros",
              np.where(v["montant_ttc"] > 10000, "moyen", "petit"))
t3 = time.perf_counter()
print("apply      :", f"{{(t1 - t0) * 1000:.1f}} ms")
print("vecteur    :", f"{{(t3 - t2) * 1000:.2f}} ms")
print("rapport    :", f"{{(t1 - t0) / (t3 - t2):.0f}}x")
print("identiques :", bool((r1 == r2).all()))
```

```
apply      : 8.3 ms
vecteur    : 2.16 ms
rapport    : 4x
identiques : True
```

C'est la **fonction de C05** (un `if/elif/else` par valeur) appliquée aux 50 008 montants : `apply` la fait **fiche par fiche** (l'ascenseur du §3) — 8,3 ms — là où `np.where` la fait **en convoi** (2,16 ms) : **4×**, même résultat (identiques : `True`). La règle du chapitre : **vectoriser d'abord**, `apply` quand la valeur a besoin d'une **logique individuelle** que le convoi ne fait pas (du texte « N/C » à nettoyer, une décision par cas). Et quand on l'utilise, on **mesure** — 4× sur 50 000, ce sera 40× sur 5 millions.

5.7 L'export Excel — le classeur multi-feuilles, le livrable E3

```
audit = pd.DataFrame({"controle": ["formes", "doublons", "retours", "dates inverses",
    "manquants"],
    "valeur": [len(v), 8, 208, 21406, 0]})
with pd.ExcelWriter("/tmp/c07/synthese.xlsx", engine="openpyxl") as w:
    vue.drop(columns=["ca_cents"]).to_excel(w, sheet_name="ca_mensuel", index=False)
    audit.to_excel(w, sheet_name="audit", index=False)
    w.sheets["ca_mensuel"].column_dimensions["A"].width = 12
print({k: v.shape for k, v in pd.read_excel("/tmp/c07/synthese.xlsx", sheet_name=None).items()})
```

```
{'ca_mensuel': (120, 5), 'audit': (5, 2)}
```

Le `with pd.ExcelWriter(...)` est le **classeur** : chaque `to_excel(w, sheet_name=...)` est une **feuille nommée** (`ca_mensuel`, `audit`), et les paramètres `openpyxl` (`column_dimensions[...].width`) font la **mise en forme minimale** (largeurs, et les formats de nombres pour les colonnes monétaires). Le rechargement `sheet_name=None` renvoie **toutes** les feuilles (dictionnaire nom → `DataFrame`) : 120×5 et 5×2 , l'aller-retour sans perte — le CSV reste la porte d'entrée, l'Excel le **format de livraison** (M05), et E3 exige les comptages **identiques** au corrigé M07.P et les totaux **± 2 FCFA**, la dérive **documentée dans le rapport**.

Conseil professionnel. Un classeur de synthèse se juge à trois choses : les **feuilles nommées** (on les rechargera par nom), les **comptages identiques** au corrigé de référence, et les **totaux ± 2 FCFA documentés** — la dérive float n'est pas cachée dans le classeur, elle est **écrite dans le rapport d'audit** : c'est la différence entre un livrable et un fichier.

5.8 La leçon des centimes — 1 à 2 FCFA sur 7,9 milliards

```
print("float    :", v["montant_ttc"].sum())
print("centimes:", ct.sum() // 100)
```

```
float    : 7908259732.2
centimes: 7908259732
```

Addition	Résultat	Écart au Decimal exact (7 908 259 732,20)
Decimal exact (le vrai total)	7 908 259 732,20	—
DuckDB SUM (float, M07)	7 908 259 730,56 (publié arrondi : 731)	≈ 1,6 FCFA
pandas float64 (ci-dessus)	7 908 259 732,20	0
pandas centimes entiers (ci-dessus)	7 908 259 732	≤ 1 FCFA (l'arrondi)

Définition. addition en centimes — cent-based summation — la conversion des montants en **centimes entiers** (`(colonne * 100).round().astype("int64")`) avant toute somme : le centime est le plus petit multiple **exact** de FCFA, l'entier ne dérive pas, et le total final se convertit en FCFA par une seule division (`// 100`).

`montant_ttc` est un **DOUBLE** (C07, `dtypes`) : 50 008 additions de doubles cumulent l'erreur de représentation — 0,71 FCFA sur le CA 2025, 0,51 sur le CA 2026 (mesuré, \$5.5), 1,64 sur le total (DuckDB, M07). **Négligeable en métier, visible en audit** — et c'est **pour ça** que la vérification croisée SQL/pandas du module est **exacte sur les comptages** et à **± 2 FCFA sur les totaux monétaires** (écarts mesurés, sourcés `m08p_ecart_somme_float_*`) : la tolérance n'est pas une faute de goût, c'est la **physique des floats**, mesurée et documentée — comme E3 l'exige dans le rapport d'audit.

5.9 L'annexe migration 2.3 → 3.x — les 12 écritures, avant/après

Les 4 **1^{re} save** sont démontrées en C07 §5.10 (sorties réelles) ; les 8 restantes, exécutées ici sous pandas 3.0.6 :

#	Écriture (2.x)	Statut 3.0.6 (sortie réelle)	Écriture enseignée
1	<code>df.append(ligne)</code>	<code>AttributeError</code> (C07 §5.10)	<code>pd.concat([df, ligne])</code>
2	<code>inplace=True</code>	deux comportements (C07 §5.10)	réassigner : <code>s = s.fillna(0)</code>
3	<code>df[masque]["col"] = v</code>	changement perdu (C07 §5.10)	<code>df.loc[masque, "col"] = v</code>
4	<code>resample("M") / alias 'H', 'S'</code>	<code>ValueError : 'M' is no longer supported... use 'ME' (\$5.5)</code>	"ME", "h", "s"
5	<code>Series.iteritems()</code>	<code>AttributeError: 'Series' object has no attribute 'iteritems'</code>	<code>Series.items()</code>
6	<code>DataFrame.applymap(f)</code>	<code>AttributeError: 'DataFrame' object has no attribute 'applymap'</code>	<code>DataFrame.map(f)</code>
7	<code>fillna(method="ffill")</code>	<code>TypeError</code> (C07 §5.10)	<code>s.ffill() / s.bfill()</code>
8	<code>reindex_axis / swapaxes</code>	<code>AttributeError: 'DataFrame' object has no attribute 'reindex_axis'</code>	<code>reindex / transpose</code>
9	<code>df.mad()</code>	<code>AttributeError: 'DataFrame' object has no attribute 'mad'</code>	<code>(x - x.mean()).abs().mean()</code> (NumPy)
10	compter une cellule texte vide comme non-vidé	les 4 écarts mesurés (C07 §5.10 : 103 → 85, 373 → 372, 18 → 0, 7 → 0)	normaliser avant de compter
11	brancher sur le dtype object	le texte est en str (mesuré C07 §5.2/ §5.10)	brancher sur la valeur , pas le dtype
12	modifier une vue en croyant modifier la mère	silencieux : la vue change, la mère est intacte (§5.9 ci-dessous)	<code>copier : tranche = df[masque].copy()</code>

```
v2 = v[v["est_retour"] == True]
v2["montant_ttc"] = 0
print("vue modifiée :", v2["montant_ttc"].sum())
print("mère intacte :", v["montant_ttc"].sum() == 7908259732.2)
```

```
vue modifiée : 0.0
mère intacte : True
```

Le n° 12 est le **cas muet** de la liste : sous copy-on-write (3.x), `v[v["est_retour"]]` est une **copie** — modifier `v2` ne touche pas `v` (la mère est intacte, mesuré), **sans erreur ni avertissement** (plus de `SettingWithCopyWarning` : la protestation a été remplacée par le silence). L'écriture ensei-

gnée : **copier explicitement** (`tranche = df[masque].copy()`) quand on veut modifier la tranche, **df.loc** quand on veut modifier la mère.

Dans les faits. Les **4 écarts mesurés** 2.2.3 / 3.0.6 (C07 §5.10, fichier « attendu » de M03, 480 lignes) sont le **sujet E4** du module (« ce script tourne mais ses chiffres sont faux ») : même code, même fichier, 18 lignes changent de camp entre les deux versions — et rien ne proteste. L'annexe n'est pas une leçon, c'est un **repère** : les 12 lignes ci-dessus, et la règle qui les remplace toutes — brancher sur la valeur, réassigner, vectoriser, mesurer — « votre script de 2026 tournera en 2028 ».

6. Exemple concret — les 10 questions de M07.P, exécutées en pandas

Le fil rouge : les **30 questions** du projet M07.P (la direction commerciale, M07) **réécrites en pandas** — 10 exécutées dans l'atelier le 19/09/2026, comparées au corrigé SQL (les valeurs de 03_exercices/dossier_M07/ATTENDU.json), et 9 affichées avec le parallèle côte à côte. Les 10 exécutées :

Q	Question	pandas (exécuté)	SQL M07	Écart
Q01	Combien de clients ?	<code>len(c)</code> → 1200	<code>COUNT(*)</code> → 1200	0
Q02	Combien de produits actifs ?	<code>int(p["actif"].sum())</code> → 362	<code>WHERE actif = TRUE</code> → 362	0
Q04	CA TTC total (sans filtre)	<code>centimes</code> → 7 908 259 732	<code>ROUND(SUM(...))</code> → 1 FCFA en dessous (arrondi float)	1
Q07	Top magasin par CA	<code>par_mag.idxmax()</code> → 4 (1 596 813 264)	→ 4	0
Q10	Panier moyen (sans retours)	<code>round(...mean())</code> → 158159	→ 158 159	0
Q11	Top produit par quantité	<code>q.index[0]</code> → 98 (939 unités)	→ 98	0
Q13	Clients distincts par magasin	<code>nunique()</code> → toute la base client par magasin	→ même	0
Q15	Ventes le dimanche	<code>dayofweek == 6</code> → 7136	<code>EXTRACT(DOW) = 0</code> → 7136	0
Q18	Clients plafond > 100 000	<code>int((c["plafond_credit"] > 100000).sum())</code> → 490	→ 490	0
Q19	Ventes uniques (dédoublonnées)	<code>drop_duplicates</code> → 50000	→ 50 000	0

Le seul écart de la table est le Q04 : **1 FCFA**, la dérive `float` du §5.8 (SQL arrondi du `float`, pandas centimes, Decimal exact 7 908 259 732,20) — **comptages exacts, totaux ± 2 FCFA**, la règle de la croisée, tenue sur 10 questions. Les 9 affichées (le parallèle, sans exécution) :

Q	SQL (M07)	pandas (C08)
Q03	<code>SELECT id_magasin, nom, ville FROM magasin</code>	<code>m[["id_magasin", "nom", "ville"]]</code>
Q05	<code>SELECT * FROM mode_paiement</code>	<code>donnees["mode"]</code>
Q06	<code>SELECT MIN(date_vente), MAX(date_vente)</code>	<code>dv.min(), dv.max()</code>
Q09 (piège P2)	<code>SUM(montant_ttc)</code> sans filtrer les retours (gonflé)	<code>v["montant_ttc"].sum()</code> — le CA « brut » 7 908 259 732 includ les 208 retours
Q12	<code>GROUP BY id_client ORDER BY ca DESC LIMIT 5</code>	<code>v.groupby("id_client")["montant_ttc"].sum().sort_values(ascending=False).head(5)</code>

Q	SQL (M07)	pandas (C08)
Q14 (piège P3)	WHERE date_vente <= date_limite_remise (21 406 lignes inversées)	v.loc[dv <= dl] — même filtre, même 21 406
Q16	taux de retour par catégorie	v.groupby(c)["est_retour"].mean()
Q20	clients dans ≥ 3 magasins	v.groupby("id_client")["id_magasin"].nunique()
Q23 (piège P5)	COUNT(DISTINCT (client, produit))	len(v[["id_client", "id_produit"]].drop_duplicates())

7. Démonstration pas à pas — 6 étapes du bout en bout

Sortie réelle de chaque commande (exécutées le 19/09/2026, venv pandas 3.0.6).

Étape 1 — Le GROUP BY (les 5 magasins, centimes) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
ct = (v['montant_ttc'] * 100).round().astype('int64')
print((ct.groupby(v['id_magasin']).sum() // 100).to_string())"
1    1595223423
2    1578987709
3    1564937883
4    1596813264
5    1572297451
```

Interprétation : le GROUP BY de M07 — top magasin 4 (1 596 813 264, m08p_ca_top_magasin), plus petit 3 (1 564 937 883).

Étape 2 — Le transform (le delta vs la moyenne du magasin) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
d = v['montant_ttc'] - v.groupby('id_magasin')['montant_ttc'].transform('mean')
print('lignes :', len(d), '| delta moyen magasin 1 :', d[v['id_magasin'] == 1].mean())"
lignes : 50008 | delta moyen magasin 1 : -8.299735787245575e-12
```

Interprétation : 50 008 lignes conservées (pas de réduction), contrôle ≈ 0 — la moyenne du groupe est bien celle du groupe.

Étape 3 — Le merge (et la table vide) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
o = pd.read_csv('03_exercices/dossier_M08/objectif_magasin.csv')
vo = v.merge(o, on='id_magasin', how='left')
print(vo.shape, '| NaN :', int(vo['ca_objectif_ttc'].isna().sum()))"
(50008, 16) | NaN : 50008
```

Interprétation : le LEFT JOIN sauve les 50 008 lignes — mais la table opposée étant vide, toute la colonne rejointe est NaN : c'est le signal du §5.3.

Étape 4 — La vue mensuelle (120 lignes, sans retours) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
ct = (v['montant_ttc'] * 100).round().astype('int64')
vs = v.loc[~v['est_retour']]
vue = pd.DataFrame({'id_magasin': vs['id_magasin'],
                    'annee': pd.to_datetime(vs['date_vente']).dt.year,
                    'mois': pd.to_datetime(vs['date_vente']).dt.month,
                    'ca_cents': ct[vs.index]}).groupby(['id_magasin', 'annee', 'mois'], as_index=False)
['ca_cents'].sum()
print('lignes :', len(vue), '| plus gros :', (vue['ca_cents'] // 100).max())
lignes : 120 | plus gros : 78965529
```

Interprétation : la vue `v_ca_mensuel_magasin` de M07, reconstruite sans SQL — 120 lignes, plus gros mois 78 965 529 (`m08p_plus_gros_mois_ca`).

Étape 5 — Le resample (24 mois, alias mort) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
ct = (v['montant_ttc'] * 100).round().astype('int64')
dv = pd.to_datetime(v['date_vente'])
s = pd.Series(ct.to_numpy() // 100, index=dv)
print('mois :', len(s.resample('ME').sum()))"
mois : 24
```

Interprétation : 24 mois de 2025-2026 — et `resample("M")` lèverait le `ValueError` du §5.5 : l'alias 'M' est mort, 'ME' est l'écriture vivante.

Étape 6 — Le classeur (2 feuilles, rechargées) :

```
$ python3 -c "
import pandas as pd
print({k: v.shape for k, v in pd.read_excel('/tmp/c07/synthese.xlsx', sheet_name=None).items()})"
{'ca_mensuel': (120, 5), 'audit': (5, 2)}
```

Interprétation : l'aller-retour sans perte — le classeur E3 est lisible par `pandas` comme par Excel : feuilles nommées, comptages identiques, totaux ± 2 FCFA documentés.

8. Erreurs fréquentes

1. **Diviser avant de sommer — les 228 FCFA de l'erreur.** `ca_cents // 100` ligne par ligne avant le `groupby` donne **228 FCFA de moins** pour le plus gros mois au lieu de **78 965 529** (sorties réelles, §5.4) : l'arrondi **par ligne** perd des centimes que la somme aurait conservés. Remède : **sommer en centimes, diviser une fois** (`ca_cents` → `// 100` après l'agrégat) — l'ordre des opérations est la leçon.
2. **Oublier la définition de la vue — les retours qui gonflent.** Le pivot **avec** retours donne un plus gros mois **plus haut** que 78 965 529 (sorties réelles : l'écart est le CA des retours de ce mois) : la vue M07 est définie **sans retours** (`WHERE est_retour = FALSE`) — c'est la définition, pas un choix. Remède : avant un pivot, dire **quelles lignes** la vue contient (C07 §8, erreur n° 6) — la question est un cinquième du travail.
3. **L'alias de période mort.** `resample("M")` → `ValueError: Invalid frequency: M ... 'M' is no longer supported ... use 'ME'` (sortie réelle, §5.5). Remède : les alias courts sont morts en 3.x — 'ME' (fin de mois), 'h', 's' ; et le message d'erreur **cite la solution** : le lire.
4. **Le merge sans clé explicite.** `v.merge(m)` sans `on=` joint sur l'**intersection des index** (ici 0..49 807 des deux côtés) — la jointure « réussit » au pire endroit, sur des index qui ne veulent rien dire. Remède : **toujours** `on="id_magasin"` — la clé est une information métier, pas un détail de syntaxe.

5. **apply ce que le convoi fait en 4× moins.** La classification `classe_montant` en `apply` coûte 8,3 ms, en `np.where` 2,16 ms — **4×**, même résultat (mesuré, §5.6). Remède : **vectoriser d’abord** ; `apply` est réservé à la logique individuelle (texte « N/C », décision par valeur) — et quand on l’utilise, on mesure.
6. **Le dimanche qui n’est pas au même endroit.** `EXTRACT(DOW)` (DuckDB) a **0 = dimanche** ; `dt.dayofweek` (pandas) a **0 = lundi, 6 = dimanche** — la question Q15 répond 7 136 des deux côtés **à condition** de traduire (`dayofweek == 6`, pas `== 0`). Remède : quand un comptage « date » passe d’un moteur à l’autre, vérifier le **zéro** du calendrier avant le chiffre.

9. Bonnes pratiques professionnelles

1. **Centimes pour les totaux, jamais de float additionné** : `(col * 100).round().astype("int64")` avant tout `sum` monétaire (§5.8) — et la dérive, **mesurée**, documentée dans le rapport (E3 l’exige).
2. **La définition de la vue avant le pivot** : quelles lignes (retours ? doublons ?), quelles clés (magasin × mois ?) — la vue de M07 est sans retours, et un pivot « qui change le chiffre » a changé de définition, pas de données.
3. **merge toujours avec on= explicite** — et une colonne de `NaN` entière après jointure = signal (table vide, ou clé fausse) : on l’inspecte, on ne l’exporte pas.
4. **apply en dernier recours, mesuré** : la vectorisation d’abord (§5.6, 4×), et la mesure dans le rapport (E4 : « ce qui a failli tourner en boucle »).
5. **Écrire pour 2028** : les patterns valides 2.3 et 3.x (réassigner au lieu de `inplace`, `'ME'` au lieu de `'M'`, `.map` au lieu de `applymap`) — l’annexe du §5.9 est le repère ; la règle tient en quatre verbes : brancher sur la valeur, réassigner, vectoriser, mesurer.

10. Exercice guidé — « l’année 2025 du magasin 4 » (15 min, /10)

Énoncé. Écrire `magasin4.py` qui, sur la **vue mensuelle** reconstruite du §5.4 (120 lignes, sans retours) :

1. isole le **magasin 4** en **2025** (12 mois) ;
2. affiche la **liste des 12 CA mensuels** en centimes converties (une ligne par mois : `mois → ca`) ;
3. affiche le **plus fort mois** (numéro + CA) et le **total 2025** du magasin (en centimes entiers convertis, pas en `float`).

Barème. La vue reconstruite correctement (centimes, sans retours) (3 pts) · l’isolation magasin/année par **masques** (2 pts) · le plus fort mois par `idxmax/max` (2 pts) · le total en centimes (pas `float.sum()`) (2 pts) · la sortie alignée sur le modèle ci-dessous (1 pt). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```

1 -> 64948883
2 -> 76403254
3 -> 61294111
4 -> 62254879
5 -> 63589011
6 -> 61805710
7 -> 64979650
8 -> 65368579
9 -> 75836578
10 -> 68271344
11 -> 64272077
12 -> 66454383
plus fort mois : 2 = 76403254
total 2025 : 795478466

```

11. Exercices autonomes

Exercice 8.1 — Le GROUP BY des rayons. Le CA **par catégorie** (merge avec produit pour la catégorie, agrégat en centimes) : afficher les 8 valeurs et le top — le top doit redonner 1 236 006 464 (catégorie 7, m08p_ca_categorie_top). (15 min)

Exercice 8.2 — Le transform global. La colonne « delta vs la moyenne **générale** » : montant_ttc - montant_ttc.mean(), et le contrôle (la somme des deltas ≈ 0 , bruit float). (10 min)

Exercice 8.3 — Le merge des noms. Les **10 produits les plus vendus** (Q11) avec leur **désignation** (merge avec produit sur id_produit) : afficher id_produit, designation, quantité — le 1^{er} doit être le 98. (15 min)

Exercice 8.4 — Le resample annuel. Le CA mensuel 2025 par resample("ME") (sur l'index daté) : afficher les 12 valeurs, puis démontrer que resample("M") lève le ValueError (le message, tel quel). (10 min)

Exercice 8.5 — Le classeur. Un classeur ma_synthese.xlsx avec **3 feuilles** nommées (audit, par_magasin, par_mois) à partir des résultats du chapitre, largeurs réglées, rechargé par sheet_name=None et affiché (nom → forme). (15 min)

12. Correction détaillée

Exercice guidé. Le script (la vue est celle du §5.4, construite en centimes) :

```

m4 = vue[(vue["id_magasin"] == 4) & (vue["annee"] == 2025)].sort_values("mois")
for _, l in m4.iterrows():
    print(f"{int(l['mois'])} -> {int(l['ca_cents'] // 100)}")
print("plus fort mois :", int(m4.loc[m4['ca_cents'].idxmax(), 'mois']),
      "=", int(m4['ca_cents'].max() // 100))
print("total 2025 :", int(m4['ca_cents'].sum() // 100))

```

Les 14 lignes publiées **sont** la sortie de l'atelier (12 mois + plus fort + total). Le contrôle à faire en tête : la somme des 12 CA mensuels doit redonner le total affiché — ici c'est la **même** addition en centimes, donc l'égalité est exacte (pas d'arrondi intermédiaire).

Exercice 8.1. Le top de la sortie est la catégorie **7** : 1 236 006 464 FCFA (m08p_ca_categorie_top) — le merge apporte id_categorie, l'agrégat s'additionne en centimes, et le top redonne la clé du socle : la vérification croisée tient sur cette dimension aussi.

Exercice 8.2. La somme des deltas renvoie ≈ 0 (un résidu infinitésimal, le bruit float de l'addition) : chaque vente s'écarte de la moyenne générale d'autant qu'elle l'écarte **en sens inverse** — le contrôle du transform du §5.2, à l'échelle globale.

Exercice 8.3. Le 1^{er} de la sortie est le produit **98** (939 unités — Q11), sa désignation vient du merge avec produit : la jointure apporte le **nom**, l'agrégat apporte le **chiffre** — les deux dans la même ligne, comme le JOIN ... ORDER BY ... LIMIT 10 de M07.

Exercice 8.4. Les 12 valeurs mensuelles 2025 (environ 325,9 M FCFA en janvier, 340,0 M en février — les totaux **tous magasins, retours compris** : ce n'est pas la vue du §5.4, qui exclut les retours) ; resample("M") lève ValueError: Invalid frequency: M ... 'M' is no longer supported ... use 'ME' — le message cite la solution, l'annexe le détaille.

Exercice 8.5. Le rechargement renvoie 3 feuilles nommées (audit, par_magasin, par_mois) avec leurs formes — l'aller-retour sans perte : le classeur est **lisible par machine** (E3), pas seulement par les yeux.

13. Mini-projet M08.P — « Le script qui fait le travail de trois matinées » (2 h)

Énoncé. Écrire un seul script commenté, analyse_commerciale.py, qui, exécuté d'une traite, produit les 6 sections du projet :

1. **S1 — Import** des 8 CSV avec contrôle des formes (dictionnaire nom → lignes) ;
2. **S2 — Audit** (doublons, retours brut/net, dates inversées, deadlines aberrantes, manquants, table vide) avec **rapport imprimé** — les 4 défauts chiffrés **avec leur définition exacte** et la **méthode** qui les a trouvés (E2) ;
3. **S3 — Nettoyage** : dédoublonnage, exclusion des retours des indicateurs ;
4. **S4 — Indicateurs** : CA brut / CA propre / panier moyen / CA par magasin / par catégorie / plus gros mois — en centimes (les 30 questions M07 reformulées) ;
5. **S5 — Les 4 graphiques** : CA mensuel, CA par magasin, répartition par catégorie, histogramme des montants (matplotlib, Agg, 4 fichiers PNG) ;
6. **S6 — Export** du classeur synthese_commerciale.xlsx : 4 feuilles nommées (audit, indicateurs, par_magasin, par_mois), comptages identiques au corrigé M07.P, totaux **± 2 FCFA**, la dérive **documentée dans le rapport** (E3).

Le script du chapitre, exécuté d'une traite le 19/09/2026 (sorties publiées) — le script modèle est commité : 03_exercices/dossier_M08/analyse_commerciale_modele.py :

```
S1 formes : {'v': 50008, 'c': 1200, 'p': 380, 'cat': 8, 'm': 5, 'mode': 5, 'tva': 2, 'o': 0}
S2 audit : doublons=8 retours_brut=208 retours_net=200
           dates_naives=21406 deadlines_j20=15 manquants=0
           table vide : ['o']
S3 propres : 49800 lignes (doublons sortis, retours exclus)
S4 ca_brut=7908259732 ca_propre=7876320164 panier=158140
   top magasin : 4 1596813264
   top categorie : 7 1236006464
   plus gros mois : magasin 3 (2025-08) : 78965529
S5 graphiques : ['g1_ca_mensuel.png', 'g2_ca_magasin.png', 'g3_ca_categorie.png',
                 'g4_hist_montants.png']
S6 classeur : {'audit': (7, 2), 'indicateurs': (4, 2), 'par_magasin': (5, 1), 'par_mois': (120,
3)}
```

Chaque chiffre du S4 est une **clé du socle** : ca_brut = m08p_ca_brut, ca_propre = m08p_ca_sans_retours, panier = m08p_panier_moyen, top magasin 4 = m08p_ca_top_magasin, top catégorie 7 = m08p_ca_categorie_top, plus gros mois = m08p_plus_gros_mois_ca — et le S2 chiffre les 4 défauts avec leurs définitions (8 doublons → 50 000 uniques ; 208 retours bruts / 200 après dédoublonnage ; 21 406 dates sous la condition naïve + 15 deadlines aberrantes ; 0 manquant, 1 table vide) — **E2 et E3 sur la même sortie**. Le **E4** (« ce qui a failli tourner en boucle ») est votre page : les 2-3 blocs où vous avez hésité entre for et vectorisation, avec la **mesure de temps** — la démo du §5.6 (4×) en est le modèle.

Critères de réussite (grille /20, seuil 13) : E1 — le script commenté (6 sections numérotées, entêtes, aucune valeur en dur, `if __name__ == "__main__"`) (8 pts) · E2 — le rapport d’audit imprimé (les 4 défauts chiffrés + méthode) (4 pts) · E3 — le classeur (4 feuilles nommées, comptages identiques, totaux ± 2 FCFA documentés) (4 pts) · E4 — la note boucle/vectorisation (mesures fournies) (4 pts).

Note de correction. Le **seuil 13/20** est celui du projet : un script qui produit les bons chiffres **avec** les 4 défauts documentés passe ; un script aux bons chiffres **sans** l’audit non plus ne passe pas — le chiffre juste, obtenu **sans se faire piéger**, c’est la règle du projet M07.P, reprise ici : seul le chiffre juste compte, et la méthode qui l’a trouvé.

14. Boîte à outils du chapitre

Boîte à outils. Le venv de C01 (`/tmp/venv_m08`, pandas 3.0.6 + `matplotlib`) + le dossier `03_exercices/dossier_M08/` — la boîte du chapitre s’ajoute à celles de C02-C07 : le classement, la jointure, le pivot, la livraison.

Outil	Usage	Syntaxe
<code>groupby(...).agg([...])</code>	le GROUP BY	<code>v.groupby("id_magasin")["montant_ttc"].agg(["sum", "count", "mean"])</code>
<code>transform("mean")</code>	l’agrégat sur chaque ligne	<code>v["delta"] = v["x"] - v.groupby("g")["x"].transform("mean")</code>
<code>merge(..., on=..., how="left")</code>	le JOIN	<code>v.merge(m, on="id_magasin")</code>
<code>groupby([clés])</code> (vue)	la reconstruction de vue	120 lignes, sans retours (définition M07)
<code>pd.to_datetime</code> <code>dt.year</code>	/ l’EXTRACT	<code>dv.dt.year</code>
<code>resample("ME")</code>	le tri par fin de mois	<code>s.resample("ME").sum()</code>
<code>apply(f)</code>	la fonction par valeur (4x, mesuré)	<code>s.apply(classe_montant)</code>
<code>ExcelWriter</code> (multi-feuilles)	la livraison E3	<code>with pd.ExcelWriter(f) as w: df.to_excel(w, sheet_name=...)</code>
centimes entiers	la leçon monétaire	<code>(col * 100).round().astype("int64")</code>

15. Résumé du chapitre

- **groupby/agg** : le GROUP BY de M07 en une ligne — 5 magasins, 1 564 937 883 à 1 596 813 264 FCFA, top magasin 4 ; **transform** remet l’agrégat du groupe sur chaque ligne (50 008 conservées, delta moyen ≈ 0).
- **merge** : le JOIN en nom — 13 + 5 = 18 colonnes, zéro ligne perdue ; la table **vide** `objectif_magasin` rend **toute** la colonne jointe en NaN (50 008, mesuré) — le signal à lire.
- **La vue mensuelle** : 120 lignes **sans retours** (la définition M07), plus gros mois 78 965 529 (magasin 3, 2025-08) — la vue SQL reconstruite **sans SQL** ; sommer **en centimes**, diviser **après**.
- **Les dates** : `to_datetime`, `dt.year` (24 920 / 25 088), `resample("ME")` (24 mois) — l’alias `'M'` est **mort** (`ValueError` qui cite `'ME'`).
- **apply** : la fonction de C05 par valeur — 8,3 ms contre 2,16 ms en vectorisé (**4x**, même résultat) : vectoriser d’abord, mesurer, `apply` en dernier recours.

- **L'Excel** : ExcelWriter multi-feuilles nommées, largeurs, aller-retour sans perte — le livrable E3 (comptages identiques, totaux ± 2 FCFA, dérive documentée).
- **Les centimes** : 1 à 2 FCFA de dérive float sur 7,9 milliards (mesuré, sourcé) — négligeable en métier, visible en audit ; d'où « comptages exacts, totaux ± 2 FCFA ».
- **L'annexe migration** : les 12 écritures (4 en C07, 8 ici) avec le code avant/après et les 4 écarts mesurés — brancher sur la valeur, réassigner, vectoriser, mesurer ; « votre script de 2026 tournera en 2028 ».

16. À retenir

À retenir. Le chiffre de pandas se **définit** avant de se calculer : quelles lignes (retours ? doublons ?), quelle addition (float ou centimes), quelle jointure (on=, how=) — la vue de M07 et la vue « avec retours » ne sont pas les mêmes vues, et la dérive de 1 à 2 FCFA n'est pas une erreur, c'est la **physique des floats**, mesurée et documentée.

À retenir. La migration 2.3 → 3.x se fait **par écriture**, pas par panique : les 12 lignes de l'annexe (réassigner au lieu de inplace, 'ME' au lieu de 'M', .map au lieu de applymap, .copy() explicite), les 4 écarts mesurés (103 → 85, 373 → 372, 18 → 0, 7 → 0) — et la règle qui les remplace toutes : **brancher sur la valeur, pas sur le dtype**.

17. Évaluation formative (auto-correction, 8 min)

1. **agg et transform — en quoi les sorties diffèrent-elles sur groupby("id_magasin") ?** → agg **réduit** : 5 lignes (une par groupe) ; transform **ne réduit pas** : 50 008 lignes, chacune portant l'agrégat de son groupe (le delta vs la moyenne du magasin, \$5.2). (2 pts)
2. **v.merge(o, on="id_magasin", how="left") sur la table vide objectif_magasin — forme de résultat, et combien de NaN ?** → (50008, 16) : le LEFT sauve les 50 008 lignes, mais la table opposée est vide — les 50 008 cases ca_objectif_ttc sont NaN (mesuré) : le signal du \$5.3. (2 pts)
3. **Pourquoi la vue mensuelle reconstruit-elle 120 lignes « sans retours », et que renvoie le plus gros mois ?** → la vue M07 est **définie** WHERE est_retour = FALSE (c'est la définition, pas un choix) : 5 magasins × 24 mois = 120 lignes (m08p_vue_lignes) ; plus gros mois = 78 965 529 FCFA (magasin 3, 2025-08, m08p_plus_gros_mois_ca) — le même nombre que M07. (2 pts)
4. ****resample("M")** sous pandas 3.0.6 — que lève-t-il, et quelle est l'écriture vivante ?**
→ ValueError: Invalid frequency: M ... 'M' is no longer supported ... use 'ME'
l'alias court est mort en 3.x ; l'écriture vivante est "ME" (fin de mois) — le message cite la solution (§5.5). (1 pt)
5. **Pourquoi divise-t-on les centimes seulement APRÈS le groupby, et que coûte l'ordre inverse ?** → diviser ligne par ligne d'abord **arrondit chaque ligne** et perd des centimes que la somme conserverait : le plus gros mois sort à **228 FCFA** de son total exact (mesuré, \$8 n° 1) — on somme en centimes, on divise une fois. (1 pt)
6. **apply(classe_montant) et np.where — même résultat, combien de temps chacun (mesuré), et la règle d'usage ?** → True (identiques : True) ; 8,3 ms contre 2,16 ms (4×) : **vectoriser d'abord**, apply pour la logique individuelle (texte, décision par valeur), et mesurer quand on l'utilise (§5.6). (1 pt)
7. **Le cas muet n° 12 de l'annexe — que fait v2 = v[masque]; v2["x"] = 0 sous 3.0.6, et quelle est l'écriture enseignée ?** → il modifie la **vue** (copie, copy-on-write) et **perd** le chan-

gement pour la mère (mesuré : la mère intacte), **sans erreur** — l'écriture enseignée : copier explicitement (`tranche = df[masque].copy()`) ou agir sur la mère par `df.loc`. (1 pt)

8. **La règle de la croisée SQL/pandas du module — l'énoncer, et donner le seul écart des 10 questions exécutées.** → **comptages exacts, totaux monétaires ± 2 FCFA** (la dérive `float`, mesurée et sourcée `m08p_ecart_somme_float_*`) ; le seul écart des 10 questions est le Q04 : 1 FCFA (l'arrondi `float` de SQL, 1 FCFA en dessous des centimes de pandas) — documentée dans le rapport, pas cachée. (1 pt)